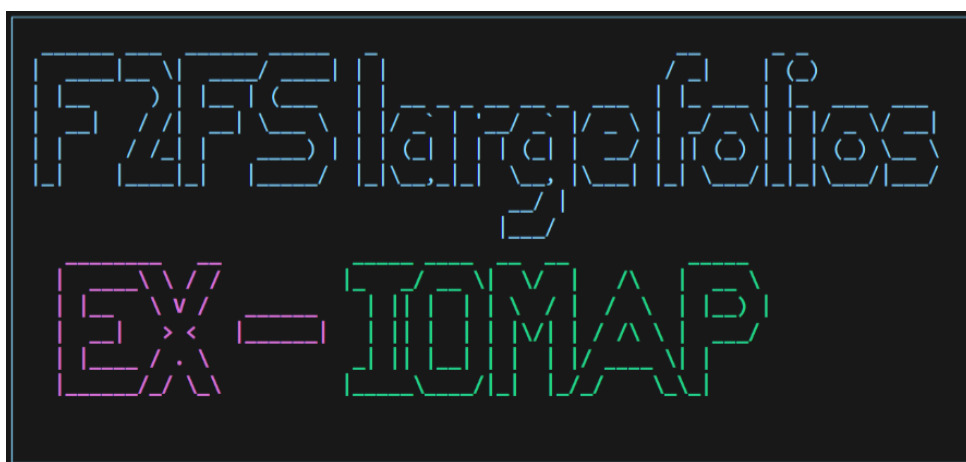




南京工业大学

NANJING TECH
UNIVERSITY

proj284: 基于扩展 iomap 的 F2FS 多 类型文件 large folios 支持



团队名称 : 蒸蒸日上队

参赛队员 : 赵南哲

指导老师 : 万夕里

项目老师 : 宋宝华

目录

1 项目概述	3
1.1 项目背景	3
1.2 项目意义	3
1.3 项目预期目标与实际完成度	4
1.4 技术路线选型与核心挑战	4
2 背景知识和数据结构介绍	6
2.1 通用背景知识	6
2.1.1 Linux 文件系统的四层抽象模型	6
2.2 核心数据结构定义与关系	7
2.2.1 struct folio: 新一代内存管理单元	7
2.2.2 folio 基本背景与支持挑战	8
2.2.3 iomap 迭代框架: 支持任意变长 folios 的关键	8
2.2.4 f2fs 与 iomap 的 I/O 桥梁	10
2.2.5 struct f2fs_io_info: f2fs 的 I/O 提交描述符	11
2.2.6 数据结构的转换流程	13
3 逐功能深度解析	13
3.1 普通文件 buffered read	13
3.1.1 功能背景	13
3.1.2 设计方案	14
3.2 普通/原子文件 buffered write	27
3.2.1 功能背景	27
3.2.2 方案设计	28
3.2.3 框架级对比: Folio 生命周期与脏区标记的精细化	33
3.2.4 写入失败与短写处理: 精准的“空间回收”机制	36
3.3 f2fs_iomap_folio_state	37
3.3.1 功能背景	37
3.3.2 重大问题分析	38
3.3.3 方案设计	40
3.4 压缩文件 buffered read	52
3.4.1 功能背景	52
3.4.2 重大问题分析	52
3.4.3 方案设计	53
3.5 压缩文件 buffered write	62
3.5.1 功能背景	62
3.5.2 重大问题分析	62
3.5.3 方案设计	63
3.6 脏页写回 Page Writeback	66
3.6.1 功能背景	66
3.6.2 核心挑战与关键问题	67

3.7	垃圾回收 GC	79
3.7.1	功能背景	79
3.7.2	重大问题分析	80
3.7.3	解决方案	82
4	项目性能测试	84
4.1	测试环境与方法论	84
4.1.1	核心测试调度脚本: <code>run_benchmark.py</code>	84
4.1.2	底层测试执行脚本: <code>fio_test.sh</code> & <code>clear_cache.sh</code>	90
4.2	性能测试结果与分析	94
4.2.1	QEMU 虚拟机性能测试	94
4.2.2	树莓派 5 性能测试	98
4.3	数据处理与可视化脚本	102
4.3.1	日志解析脚本: <code>parse_fio_logs.py</code>	102
4.3.2	结果可视化脚本: <code>visualize_results.py</code>	105
5	开发环境搭建与工作流	109
5.1	QEMU 虚拟开发环境	109
5.1.1	基本工作流搭建	109
5.1.2	高级工作流与遇到的问题	111
5.2	Git 版本控制工作流	117
5.2.1	从挣扎到规范: 工作流的演进	117
5.2.2	高级技巧: 在动态变化中维护代码	117
5.3	树莓派物理开发环境	118
5.3.1	基本工作流搭建	118
5.3.2	遇到的重大问题: 动态 IP 与网络穿透	122
6	项目总结和未来展望	123
A	改动函数一览表	125

1 项目概述

1.1 项目背景

- **Large Folios 的演进**: large folios 是内核自从 5.15 版本起引入的对 page 结构体的替代, 同时也是内核对原有的复合大页 (Compound Page) 实现方案的全新升级, 由 Linux 内存管理子系统维护者 Matthew Wilcox 先生发明。
 - **传统方案的痛点**: 传统的复合页面依靠特殊的 flag 来判断头页面 (head page) 和尾页面 (tail page), 但是其操作接口全部都是基于 page 结构体。这会导致接口使用者感到迷惑, 因为仅从函数签名来看, 无法直观地区分一个 page 参数究竟应该传入头页面还是尾页面。
 - **Folio 的革新**: folio 的出现则彻底将接口清晰化和规范化。一个 folio 结构体自身即代表一个巨大的、完整的复合页面整体, 不再需要进行头尾页面的区分。同时, 为了保持向后兼容, 原先基于头尾区分的 API 也可以轻松地将头页面指针类型转换为 folio 指针, 因为 folio 在内存布局上复用了头页面的位置。基于 folio 的引入, 可动态调节大小的大页面自此终于能广泛地在内核中得以使用, 原有的 4KB 小页面、固定大小死板僵硬的巨页 (HugeTLB)、以及使用场景比较局限的复合页面的尴尬局面被彻底打破。
- **iomap 框架的崛起**: iomap 框架是 Linux 内核现在主推的新一代文件系统 I/O 框架。它基于迭代器 (iterator) 实现, 提供了一个现代化的、统一的文件系统 I/O 接口, 并且 iomap 框架自身已经实现了文件系统所有的主要 I/O 路径。其它文件系统只需实现其暴露出来的核心钩子函数 (hook function), 如 iomap_begin 和 iomap_end, 就能以极为优雅的方式, 通过一个函数调用完成复杂的 I/O 工作。例如, 使用 iomap_readahead 完成缓冲读, 或用 iomap_file_buffered_write 完成缓冲写。
- **f2fs 的现状与需求**: f2fs (Flash-Friendly File System) 是当前安卓 (Android) 系统最广泛使用的文件系统。它因为其针对闪存 (Flash Memory) 友好的设计特性——例如通过日志结构 (Log-structured) 的设计来聚合写操作, 将随机写转换为顺序写, 从而有效减少写放大并延长闪存寿命——而广泛应用于手机这种使用闪存作为主要存储介质的移动式/嵌入式设备中。近年来, 随着硬件性能的提升和应用负载的加重, f2fs 社区也开始了对 large folios 的推进工作, 以期获得更好的性能提升。

1.2 项目意义

为 f2fs 引入 Large Folios 与 iomap 支持, 不仅是顺应内核演进趋势的必要举措, 更是解锁下一代存储性能、赋能前沿应用场景的关键一步。

- **内核层面的性能收益**:
 - **Large Folios 的直接优势**: 作为更大的内存管理单元, Folio 能够一次性覆盖更多的虚拟地址空间, 从而大幅降低因页表缓存 (TLB) 未命中而导致的性能损失。在内存

回收路径上, 由于 LRU 链表管理的内存单元数量急剧减少 (例如, 一个 64KB 的 Folio 在 LRU 上只占一个节点, 而过去是 16 个), 扫描和迁移的成本显著降低, 系统在高内存压力下的响应能力得到增强。

- **iomap 带来的 I/O 效率革命:** iomap 以“区间 (extent)”为单位进行批量处理, 彻底改变了传统文件系统“逐块 (block-by-block)”的 I/O 模式。在处理大文件时, iomap 能够一次性完成大段连续逻辑块到物理块的映射, 并生成更少、更大的 bio 请求, 这极大地减少了元数据查找的开销和 I/O 提交的 CPU 消耗, 是实现高性能 I/O 的核心。

- **对真实世界应用的深远影响:**

- **赋能 AI 与大数据:** 在 AI 训练和推理场景中, 模型权重和海量训练数据的加载是典型的 I/O 密集型任务。更高的 I/O 吞吐和更低的 CPU 开销, 意味着可以更快地将数据送入计算单元, 缩短“data-to-compute”的延迟, 从而加速整个训练和推理流程。
- **提升大型游戏与应用体验:** 现代 3A 游戏和专业图形软件需要加载庞大的高清纹理、模型和场景数据。本项目带来的性能提升, 可以直接转化为更快的应用启动速度、更短的游戏关卡加载时间, 为用户提供无缝、流畅的沉浸式体验。
- **优化数据库与数据中心负载:** 对于数据库系统和云存储服务, I/O 延迟和吞吐量是核心性能指标 (KPI)。通过本项目技术, 可以有效降低读写延迟, 提升 IOPS 和带宽, 同时节省下来的 CPU 资源可以服务于更多的计算任务, 最终帮助数据中心降低总拥有成本 (TCO)。

- **紧跟业界前沿技术浪潮:**

- **本项目所探索的技术是绝对的业界前沿。** 在移动端领域, Large Folios 带来的性能红利已成为各大厂商竞相追逐的焦点。目前, 国内如华为、OPPO、vivo 等领先的手机厂商均在其旗舰产品的操作系统内核中, 投入大量研发力量推进这项技术的适配与落地, 以构建其高端产品的核心技术壁垒。本项目的成功实践, 不仅与业界顶级厂商的技术路线保持一致, 也为该技术在更广泛场景下的应用积累了宝贵的实践经验。

1.3 项目预期目标与实际完成度

- **预期目标与实际完成情况:** 本项目的预期目标的原表述是能在 f2fs 普通文件的 I/O 路径上支持 large folios, 并让 f2fs 的垃圾回收机制能正确识别出 large folios, 最好能实现使用 iomap 支持 large folios。
- **超额完成的成果:** 而我 ** 不但直接使用了 iomap 来完成 f2fs 文件的 I/O 工作, 还扩展了 iomap, 使得压缩文件也支持了 large folios, 同时还解决了 f2fs 支持 large folios 许多独有的难点, 属于超额完成了预期目标 **。

1.4 技术路线选型与核心挑战

问题一：我们该如何为 f2fs 引入 Large Folios 支持？

理论上, 文件系统可以直接在原始的 bufferedread/write 以及 pagewriteback 路径中, 手动添加对 LargeFolios 的支持。然而, 经过我的深入调研, 发现这几乎是一条不可行的道路。真正的选择只有一个: 拥抱 iomap 框架。并且, 我很快意识到, 仅仅“使用”iomap 是不够的, 我们还必须对其进行专有的扩展。

为什么 iomap 是必由之路? 因为它为我们解决了原生 f2fs 在支持 LargeFolios 时面临的一系列根本性难题:

挑战 1: 性能瓶颈: 逐块映射 vs. 区间映射

- **f2fs 原生方式:** 以单个 Page (4KB) 为单位进行文件块地址的查找和 I/O 请求的构建, 在处理大文件时存在大量的、重复的元数据查找开销。
- **iomap 解决方案:** iomap 以区间 (extent) 为核心, 能够一次性映射一段连续的逻辑块区间, 并将其直接关联到覆盖该区间的整个 Folio。这在块映射和 I/O 请求提交两个层面, 都带来了数量级的性能提升。

挑战 2: 写放大问题: 模糊的脏页 vs. 精准的追踪

- **f2fs 原生方式:** 对 LargeFolio 的脏页标记是“模糊”的, 一旦 Folio 中有任何一个 Page 变脏, 整个 Folio 都会被标记为脏。在回写时, 这会导致大量未修改的干净数据被无效地写回, 对闪存寿命造成严重影响, 这就是“写放大”问题。
- **iomap 解决方案:** 内核 6.6 版本为 iomap 引入了 iomap_folio_state 结构, 它实现了逐块脏页追踪 (per-blockdirtytracking)。这使得我们能精准地知道一个 LargeFolio 中哪些块 (而非整个 Folio) 是脏的, 从而只回写真正需要回写的数据, 从根本上解决了写放大问题。

挑战 3: 并发 I/O 问题: 混乱的回调 vs. 原子计数

- **f2fs 原生方式:** 其代码基于“一个 Page 等于一个块”的陈旧假设, 完全没有考虑一个大的 Folio 内部可能包含多个并发 I/O 的情况。这会导致 bio 的结束回调函数 end_io 被过早调用, 错误地解锁 Folio 或结束其 writeback 状态, 引发数据竞争和一致性问题。
- **iomap 解决方案:** iomap_folio_state 中的 read_bytes_pending 和 write_bytes_pending 原子计数器, 正是为解决此问题而生。它能精确记录一个 Folio 中还有多少字节的数据“在途”, 只有当所有 I/O 都完成, 计数器归零时, 才会执行最后的回调操作。

挑战 4: 生命周期管理: 繁琐的手动 vs. 优雅的自动

- **f2fs 原生方式:** 其对 Folio (或 Page) 的加锁、解锁、标记状态等生命周期管理逻辑非常繁琐, 且只具备处理 0 阶 Folio 的意识, 直接改造以支持高阶 Folio 的成本极高。
- **iomap 解决方案:** iomap 框架本身没有任何历史包袱, 其设计原生支持高阶 Folio, 并能自动、优雅地处理其复杂的生命周期管理。

问题二：为什么仅仅使用 `iomap` 还不够？ 在将 `iomap` 用于 `f2fs` 的过程中，我很快发现，虽然 `iomap` 解决了通用性的问题，但 `f2fs` 作为一个特性丰富的文件系统，其“特殊性”是通用框架无法完全覆盖的。这引出了我设计的核心：对 `iomap` 进行扩展和适配。具体来说，挑战主要来源于以下几点：

挑战 1：压缩文件的特殊性： `f2fs` 支持文件压缩，这意味着内存中存在两种页：原始页和压缩页。压缩页位于一个特殊的 `COMPRESS_MAPPING` 缓存中，并且其真正落盘与否、与 I/O 实体 `bio` 的映射关系，都与普通数据页完全不同。而通用的 `iomap` 对 `address_mapping` 的机制完全不同，无法直接处理。

挑战 2：folio->private 字段的冲突： 我调研发现，`f2fs` 会司空见惯地将特有的私有标志位（如 `FI_ATOMIC_FILE`）存储在 `page->private` 字段中。当 `Page` 被整合为 `Folio` 后，`folio->private` 继承了该字段。这与 `iomap` 期望用该字段存储 `io map_folio_state` 指针的行为产生了直接冲突。为了解决这个问题，我设计了 `f2fs_iomap_folio_state` 结构体，它巧妙地扩展了 `io map_folio_state`，为其增加了一个存储 `f2fs` 私有标志的空间。

挑战 3：日志结构文件的本质： `f2fs` 是一个日志结构文件系统，其大量的写入操作是“异地更新”（out-of-place）。这意味着在脏页回写时，我们写入的是一个新的物理块地址，而原始数据块的地址没有任何意义。这使得 `iomap` 框架中依赖于稳定块映射的 `f2fs_map_blocks` 函数几乎无法编写。

挑战 4：Checkpoint 与重试机制： `f2fs` 依赖 Checkpoint 机制来保证文件系统的一致性。当 Checkpoint 进程启动时，它会阻塞所有新的文件操作。因此，`f2fs` 的脏页回写路径必须能够优雅地处理这种情况，支持可重试的回写（retryablewriteback）。然而，通用的 `iomap` 框架并未提供文件系统定制该重试逻辑的接口。

综上所述，我意识到，要真正实现一个健壮、高效的 `LargeFolios` 支持，我必须化用 `iomap` 的核心功能和思想，重写针对 `f2fs` 压缩文件的 `bufferedread` 和 `pagewriteback`。

2 背景知识和数据结构介绍

2.1 通用背景知识

2.1.1 Linux 文件系统的四层抽象模型

要理解 `iomap` 和 `folio` 带来的改进，我们首先需要建立一个关于文件在 Linux 内核中如何被看待的四层抽象模型。这套模型精妙地解耦了不同层级的关注点。可以将其类比为计算机网络的协议栈，每一层都只关注自己视角的索引单元，并定义了如何将单元转换（或映射）到下一层的单元，对其他层的复杂性保持透明。

1. **用户视角 (User Space):** 用户日常接触到的文件，其内容看起来是连续的。当我们使用 `read()` 或 `write()` 系统调用时，我们提供的是一个文件内的字节偏移量 `loff_t`

`pos` 和一个长度 `size_t len`。这一层操作的是逻辑上连续的字节流。

2. **内存视角 (Memory Layer)**: 内核为了加速访问, 会将文件的内容缓存到内存中。在这一层, 文件的字节流被分割成固定大小的块, 即页 (Page) 或更大的页 (Folio)。这一层通过页索引 (`pgoff_t`) 来寻址。关键的转换公式是: $\text{pgoff_t} = \text{pos} / \text{PAGE_SIZE}$ 。
3. **文件系统视角 (Filesystem Layer)**: 文件系统自身有一套逻辑块 (FS Blocks) 的概念, 大小通常为 4KB。这一层维护着核心的元数据, 负责将内存中的页索引映射到文件系统自己的逻辑块号。在页大小等于文件系统块大小的情况下, 这个转换是一对一的: $\text{fs_block_no} = \text{block_mapping}(\text{index})$ 。
4. **块设备视角 (BIO Layer)**: 这是最底层的 I/O 抽象。文件系统的逻辑块号最终需要被转换为块设备驱动能理解的地址。这一层将文件系统逻辑块号转换为逻辑扇区地址 (LBA), 并封装成 `bio` 结构体提交给块设备驱动。需要注意的是, 这里的”逻辑扇区”并非磁盘硬件上的真实物理扇区, 而是操作系统构建的一个虚拟的、连续的扇区地址空间。转换公式通常是: $\text{LBA} = \text{fs_block_no} * \text{FS_BLOCK_SIZE}$ 。

完整的转换示例: 假设用户要读取文件的第 20480 个字节开始的 8192 个字节 (即 2 个页), 在一个页大小为 4KB、文件系统块大小为 4KB、扇区大小为 512B 的系统中:

- **用户层**: `loff_t pos = 20480`, `size_t len = 8192`
- **内存层**: `pgoff_t index = 20480 / 4096 = 5`, 即从第 5 个页开始, 需要读取 2 个页 (页 5 和页 6)。
- **文件系统层**: 对应文件系统逻辑块号 5 和 6。文件系统通过其元数据 (如 extent tree) 查找这两个逻辑块在磁盘上的物理位置, 假设它们被映射到物理块号 1000 和 1001。
- **BIO 层**: 物理块号 1000 对应的 LBA 为 $1000 * 4096 = 4096000$, 物理块号 1001 对应的 LBA 为 $1001 * 4096 = 4100096$ 。最终生成的 `bio` 请求会将区间 $[4096000, 4100096 + 4096 - 512]$ 提交给块设备驱动。

这种分层的核心在于, 每一层都有自己的”视角协议”: 用户层看到的是字节偏移, 内存层看到的是页索引, 文件系统层看到的是逻辑块号, 而 BIO 层看到的是逻辑扇区地址。每一层只需要知道如何与上下相邻层进行转换, 无需了解其他层的复杂性。

2.2 核心数据结构定义与关系

2.2.1 `struct folio`: 新一代内存管理单元

`struct folio` 是 LargeFolios 机制的核心, 它代表一个物理上连续、大小为 2 的幂次个页的内存块。以下是其关键成员的精简定义, 以突出其核心功能:

struct folio 核心成员

```

struct folio {
    /* public: */
    unsigned long flags;           // 存储 folio 的各种状态位 (PG_locked,
    PG_dirty ...)
    struct list_head lru;          // 用于将 folio 链入 LRU 链表进行内存回收
    struct address_space *mapping; // 指向所属的地址空间 (通常是文件的 inode->
    i_mapping)
    pgoff_t index;                 // folio 在 mapping 内的起始页索引
    void *private;                 // 文件系统或驱动的私有数据指针
    atomic_t _mapcount;            // 被页表映射的次数
    atomic_t _refcount;            // 引用计数

    /* ... folio-specific members ... */
    atomic_t _pincount;            // Folio 被 pin 住的次数,防止被换出
#ifdef CONFIG_64BIT
    unsigned int _folio_nr_pages; // Folio 包含的 page 数量 (即 2^order)
#endif
    /* ... other members ... */
};

```

2.2.2 folio 基本背景与支持挑战

从四层模型视角来看, folio 和 page 一样, 都是使用 `pgoff_t` 作为索引单元。因此, 似乎直观上来看, 我们对于高阶 folio, 也可以将其抽象成单个单个的 `pgoff_t` 索引, 然后逐个和 `fs_block_no` 进行映射。

然而, 结合 folio 的设计来看, 会发现有很大的问题。首先 folio 逻辑上表示一整个大页。这意味着所有对一块内存元数据的各种操作, 比如锁、引用计数, 都是针对 folio 来进行的。如果将其拆成按照逐个 `pgoff_t` 索引的 page, 那 folio 的整体性信息就丢失了。并且我们将很难进行引用计数和上锁的维护。从 folio 拆分成的多个 page, 它们的头 page 都将指向 folio。这意味着如果说我们要进行上锁等各种操作, 我们只能使用老式的 page 相关接口。这将增加上锁解锁和引用计数的使用次数。使用 folio 相关接口几乎是一定会造成多重上锁/解锁这样的 bug 的。这和 folio 的设计哲学完全是背道而驰的。

那么, 我们最好从代码的概念上, 就应该抛弃对 folio 按逐个 `pgoff_t` 索引的设计思想。转而一次将一个 folio 视为一个具有起始 `pgoff_t` 和以页单位为长度的整体。而 folio 最大的特性是可以变化 order, 也就是变化它的页面数量。这种情况下, 我们该怎么解决支持 largefolios 的挑战呢?

2.2.3 iomap 迭代框架: 支持任意变长 folios 的关键

iomap 框架通过其巧妙的迭代器设计, 完美解决了支持任意变长 folios 的挑战。其核心思想是将 ** 区间映射 ** 与 ** 迭代处理 ** 相结合, 使得无论 folio 的阶数如何变化, 都能以统一的方式处理。

iomap 结构体承上而启下, 它同时存储了内存中连续区间信息和磁盘 LBA 地址字节映射:

iomap 结构体的区间映射能力

```

struct iomap {
    u64 addr;           // 映射到的磁盘字节偏移量（支持大区间）
    loff_t offset;      // 映射在文件中的字节偏移量
    u64 length;         // 映射的字节长度（关键：支持任意长度）
    u16 type;           // 映射类型（IOMAP_MAPPED, IOMAP_HOLE, ...）
    /* ... other members ... */
};

```

这种以 ** 字节为粒度的区间描述 ** 使得 iomap 能够天然地适配任何大小的 folio。无论是 4KB 的 0 阶 folio，还是 64KB 的 4 阶 folio，iomap 都能通过其 length 字段准确描述整个区间的映射信息。

iomap 的迭代器 iomap_iter 实现了这一切的自动化处理：

iomap_iter 的核心迭代逻辑

```

int iomap_iter(struct iomap_iter *iter, const struct iomap_ops *ops)
{
    // 检查是否需要开始新的映射
    if (!iter->iomap.length)
        goto begin;

    // 计算已处理的字节数
    advanced = iter->pos - iter->iter_start_pos;
    olen = iter->len + advanced;

    // 调用文件系统的结束回调（如果需要）
    if (ops->iomap_end) {
        ret = ops->iomap_end(iter->inode, iter->iter_start_pos,
                             iomap_length_trim(iter, iter->iter_start_pos, olen),
                             advanced, iter->flags, &iter->iomap);
        if (ret < 0 && !advanced)
            return ret;
    }

    // 判断是否继续迭代
    if (iter->status < 0)
        ret = iter->status;
    else if (iter->len == 0 || (!advanced && !stale))
        ret = 0; // 完成迭代
    else
        ret = 1; // 继续迭代

    // 重置 iomap 状态，准备下次映射
    iomap_iter_reset_iomap(iter);
    if (ret ≤ 0)

```

```

        return ret;

begin:
    // 调用文件系统的开始回调, 获取新的映射
    ret = ops->iomap_begin(iter->inode, iter->pos, iter->len, iter->flags,
                          &iter->iomap, &iter->srcmap);

    if (ret < 0)
        return ret;
    iomap_iter_done(iter); // 更新迭代器状态
    return 1;
}

```

这种设计的核心优势在于 ** 自适应的区间处理能力 **:

1. **统一的接口**: 上层调用者只需编写特定功能的 `iomap_begin` 函数将映射信息放入 `io` `map` 结构体中返回, 然后使用简洁的 `while(iomap_iter) + 内部执行函数` 的形式就能完成所有工作。
2. **自动区间前进**: `iomap_iter` 会根据每次处理的字节数自动前推 `iter.pos`, 无论每次处理的是 4KB 还是 64KB 的区间。
3. **变长 folio 适配**: 当遇到高阶 folio 时, 文件系统的 `iomap_begin` 可以返回一个覆盖整个 folio 的大区间映射, 迭代器会自动处理这个大区间, 避免了将 folio 拆分成多个 page 的复杂操作。
4. **性能优化**: 对于连续的磁盘块, 可以在一次 `iomap_begin` 调用中返回一个大区间, 减少了反复调用的开销。

例如, 在处理一个 16KB 的 2 阶 folio 时: - 传统方法: 需要 4 次独立的页映射调用 - `iomap` 方法: 一次 `iomap_begin` 调用返回 16KB 的区间映射, 迭代器自动处理整个区间

这种 ** 区间化的思想 ** 使得 `iomap` 框架能够无缝支持从传统的 4KB 页到未来可能出现的更大的 folios, 为文件系统的性能优化提供了坚实的基础。

2.2.4 f2fs 与 iomap 的 I/O 桥梁

为了将内存中的 folio 映射到磁盘上的物理位置, f2fs 和 `iomap` 框架通过一系列数据结构进行协作。理解它们之间的转换关系是理解本项目实现的关键。

struct extent_info: 磁盘连续区间的缓存

```

struct extent_info {
    unsigned int fofs;    /* start offset in a file (in blocks) */
    unsigned int len;     /* length of the extent (in blocks) */
    block_t blk;         /* start block address of the extent */
};

```

`extent_info` 是 `f2fs` 内部用于缓存已知的、磁盘上物理连续块区段信息的数据结构。它以 ** 文件系统块 ** 为单位记录了文件内的起始偏移量 (`fofs`)、连续长度 (`len`) 以及对应的起始物理块号 (`blk`)。

struct f2fs_map_blocks: 文件系统块映射请求

```
struct f2fs_map_blocks {
    block_t m_pblk;           // [out] 映射到的物理块起始地址
    block_t m_lblk;           // [in] 请求映射的逻辑块起始地址
    unsigned int m_len;        // [in/out] 请求映射的长度/实际映射到的连续长度
    unsigned int m_flags;      // [in] 映射标志 (e.g., F2FS_GET_BLOCK_PRECACHE)
    /* ... other members ... */
};
```

`f2fs_map_blocks` 是 `f2fs` 文件系统内部进行块映射操作时使用的核心载体。调用者通过填充 `m_lblk` 和 `m_len` (以块为单位) 来发起一个映射请求, `f2fs` 则将查找到的物理块起始地址和实际连续的长度填入 `m_pblk` 和 `m_len` 返回。

2.2.5 struct f2fs_io_info: f2fs 的 I/O 提交描述符

`f2fs_io_info` (简称 `fio`) 是 `f2fs` 将内存中的页 (`page`) 或页簇 (`folio`) 与其对应的旧块地址和新块地址打包在一起, 为向块设备提交 I/O 提供准备信息的描述符。它的设计能够同时支持 `f2fs` 的原地更新 (`in-place-update`) 和异地更新 (`out-of-place-update`)。同时, `f2fs` 中的特殊数据页, 如压缩和加密数据页, 也会被记录在该数据结构中, 从而影响最终提交 I/O 时所使用的具体数据。

原始 f2fs_io_info 结构体定义

```
struct f2fs_io_info {
    struct f2fs_sb_info *sbi;    /* f2fs_sb_info pointer */
    nid_t ino;                   /* inode number */
    enum page_type type;         /* contains DATA/NODE/META/META_FLUSH */
    enum temp_type temp;         /* contains HOT/WARM/COLD */
    enum req_op op;              /* contains REQ_OP_ */
    blk_opf_t op_flags;          /* req_flag_bits */
    block_t new_blkaddr;         /* new block address to be written */
    block_t old_blkaddr;         /* old block address before Cow */
    struct page *page;           /* page to be written */
    struct page *encrypted_page; /* encrypted page */
    struct page *compressed_page; /* compressed page */
    struct list_head list;        /* serialize IOs */
    unsigned int compr_blocks;    /* # of compressed block addresses */
    unsigned int need_lock:8;     /* indicate we need to lock cp_rwsem */
    unsigned int version:8;       /* version of the node */
    unsigned int submitted:1;     /* indicate IO submission */
    unsigned int in_list:1;       /* indicate fio is in io_list */
    unsigned int is_por:1;        /* indicate IO is from recovery or not */
};
```

```

unsigned int encrypted:1;    /* indicate file is encrypted */
unsigned int meta_gc:1;     /* require meta inode GC */
enum iostat_type io_type;   /* io type */
struct writeback_control *io_wbc; /* writeback control */
struct bio **bio;           /* bio for ipu */
sector_t *last_block;       /* last block number in bio */
};

```

在主线内核 6.17 版本 Matthew Wilcox 先生引入的补丁中, `fio` 中的 `page` 成员已经被替换为 `folio`。但是, 此时的 `fio` 并不具备处理高阶 `folio` 内部子范围的意识。因此, 为了真正支持 `largefolios`, 我将 `f2fs_io_info` 结构体定义变更如下:

支持 Large Folio 的 `f2fs_io_info` 结构体 (核心改动)

```

struct f2fs_io_info {
    ...
    block_t new_blkaddr;    /* new block address to be written */
    block_t old_blkaddr;    /* old block address before Cow */

    pgoff_t idx; /*start page index within current active folio in this fio*/
    unsigned int cnt; /*block cnts of the active folio, we assume they are continuous.*/
    union {
        struct page *page;    /* page to be written */
        struct folio *folio;
    };

    struct page *encrypted_page; /* encrypted page */
    ...
};

```

核心改动解析

- **引入子范围追踪:** 由于 `f2fs_io_info` (下文简称 `fio`) 已经被转换为使用 `fio->folio`, 并且旧的 `fio->page` 接口已被废弃, 为了支持 `largefolios`, 我们现在必须追踪一个 `folio` 的哪个子部分正在被提交到 `bio` 中。
- **新增 `idx` 和 `cnt` 字段:** 为了实现这一点, 我们向 `fio` 结构体中添加了 `idx` 和 `cnt` 两个字段。
 - `fio->idx`: 代表本次 I/O 操作在当前 `folio` 内的起始偏移量 (以页为单位)。
 - `fio->cnt`: 代表本次 I/O 操作需要处理的连续块的数量。
- **重释地址字段:** 随着这两个字段的引入, `fio` 中现有的 `old_blkaddr` 和 `new_blkaddr`

字段的含义被重新解释。它们现在代表与 `fio->idx` 相对应的 **起始** 新旧块地址。因此，一个 `fio` 不再代表从一个 `old_blkaddr` 到一个 `new_blkaddr` 的单一映射，而是代表一个**范围映射**：从块地址范围 `[old_blkaddr, old_blkaddr+fio->cnt-1]` 映射到 `[new_blkaddr, new_blkaddr+fio->cnt-1]`。

- **确保向后兼容**：在 `bio` 提交路径中，对于 `fio->cnt` 未被显式初始化的情况，我们将其默认设置为 1，并将 `fio->idx` 默认设置为 0。这确保了与所有现存的、仅在单个页面上操作的 `f2fs` 逻辑的向后兼容性。

2.2.6 数据结构的转换流程

这些结构体构成了从 `f2fs` 内部缓存到通用 I/O 框架的转换链条，其核心逻辑关系如下：

1. **Extent Cache 命中**：当 `f2fs` 需要进行块映射时，首先会使用请求的逻辑块区间查询 `extent cache`。如果命中，一个 `extent_info` 结构体会被找到。
2. **`extent_info` → `f2fs_map_blocks`**：`f2fs` 会将 `extent_info` 中的信息（以块为单位的 `fofs`, `len`, `blk`）直接转换为 `f2fs_map_blocks` 结构体中的 `m_lblk`, `m_len`, `m_pblk`。这一步是在文件系统内部完成的，单位仍然是 **** 文件系统块 ****。
3. **`f2fs_map_blocks` → `iomap`**：这是最关键的转换。`iomap` 框架调用文件系统的 `map_blocks` 接口（在 `f2fs` 中即 `f2fs_map_blocks` 函数）后，会得到一个填充好的 `f2fs_map_blocks` 结构。随后，通过 `f2fs_set_iomap` 函数将这个以 **** 块 **** 为单位的结果，转换为通用的、以 **** 字节 **** 为单位的 `iomap` 结构，从而建立起文件逻辑字节区间到磁盘 LBA 区间的完整映射。最终，上层的 I/O 逻辑（如 `iomap_readahead`）便可直接使用这个与具体文件系统无关的 `iomap` 结构来处理数据。

这个流程清晰地展示了 `iomap` 如何通过一层抽象，将 `f2fs` 特有的、基于块的 `extent_info` 和 `f2fs_map_blocks`，优雅地转换为通用的、基于字节的 I/O 描述，从而实现了文件系统与 I/O 逻辑的解耦。

3 逐功能深度解析

3.1 普通文件 buffered read

3.1.1 功能背景

普通文件的 buffered read 是文件系统中基础和最频繁的操作之一。传统的 `f2fs` 实现采用逐页处理的方式，每次只能处理一个 4KB 的页面，这在处理大文件或高阶 `folio` 时会产生显著的性能瓶颈。

本节介绍的基于 `iomap` 框架的 buffered read 实现，是我们项目中第一个要分析的核心功能。通过 `iomap` 的区间化处理思想，我们能够一次性处理大片连续的内存区间和磁盘区间，从而完美支持任意阶数的 `largefolios`。

iomap 的基本骨架与迭代流程 iomap 框架的核心是其优雅的迭代器设计。上层调用者的基本编程模式非常简洁：编写特定功能的 `iomap_begin` 函数将映射信息放入 `iomap` 结构体中返回，然后使用 `while(iomap_iter) + 内部执行函数` 的形式就能完成所有工作。

让我们通过 `iomap_readahead` 的实际用法来理解这个基本骨架：

iomap_readahead 的标准用法模式

```
void iomap_readahead(struct readahead_control *rac, const struct iomap_ops *ops)
{
    struct iomap_iter iter = {
        .inode    = rac->mapping->host,
        .pos      = readahead_pos(rac),
        .len      = readahead_length(rac),
    };
    struct iomap_readahead_ctx ctx = {
        .rac = rac,
    };

    // 核心：简洁的 while 循环 + iomap_iter
    while (iomap_iter(&iter, ops) > 0)
        iter.status = iomap_readahead_iter(&iter, &ctx);
}
```

这种设计的精妙之处在于将 **** 映射获取 **** 与 **** 数据处理 **** 完全解耦：

- **外层循环**: `iomap_iter` 负责调用 `ops->iomap_begin`，获取当前位置的区间映射信息
- **内层处理**: `iomap_readahead_iter` 负责处理该区间内的所有 folio
- **自动前进**: 迭代器根据处理的字节数自动推进位置，直到处理完所有请求的数据

3.1.2 设计方案

f2fs_map_blocks 的增强：支持区间映射 为了支持 `iomap` 的区间化处理，我对 `f2fs_map_blocks` 进行了重要扩展，新增了 `F2FS_GET_BLOCK_IOMAP` 标志位支持：

新增的空洞处理逻辑 `f2fs` 中的空洞不仅有传统的 `hole` 类型，还有 `dnode` 根本未分配的情况。为了正确处理这两种空洞类型，我新增了 `F2FS_MAP_NODNODE` 标志位：

增强的空洞检测逻辑

```
// 在 f2fs_map_blocks 中处理 dnode 未分配的情况
if (err) {
    if (flag == F2FS_GET_BLOCK_BMAP)
        map->m_pblk = 0;
    if (err == -ENOENT)
```

```

        err = f2fs_map_no_dnode(inode, map, &dn, pgofs);

        map->m_flags |= F2FS_MAP_NODNODE; // 新增: 标记为无 dnode 的空洞

        goto unlock_out;
    }

    // 在处理已分配 dnode 但块为空洞的情况
} else if (is_hole) {
    switch (flag) {
        case F2FS_GET_BLOCK_PRECACHE:
            goto sync_out; // 原有逻辑: 直接跳出

        case F2FS_GET_BLOCK_IOMAP:
            if (map->m_next_pgofs)
                *map->m_next_pgofs = pgofs + 1;
            break; // 新增: 继续处理以支持区间映射

    }
}

```

extent cache 更新策略 对于 F2FS_GET_BLOCK_IOMAP 标志, 我们启用了与 F2FS_GET_BLOCK_PRECACHE 相同的 extent cache 更新策略, 确保后续访问能够受益于缓存加速:

统一的 extent cache 更新

```

if (flag == F2FS_GET_BLOCK_PRECACHE || flag == F2FS_GET_BLOCK_IOMAP) {
    if (map->m_flags & F2FS_MAP_MAPPED && map->m_len > F2FS_MIN_EXTENT_LEN)
    {
        unsigned int ofs = start_pgofs - map->m_lblk;

        f2fs_update_read_extent_cache_range(&dn, start_pgofs,
                                             map->m_pblk + ofs,
                                             map->m_len - ofs);
    }
}

```

统一的 iomap 设置函数 为了避免代码重复并提供统一的 iomap 设置逻辑, 我设计了 `f2fs_set_iomap` 函数, 它综合了原先为 DIO 设计的逻辑, 并扩展支持 buffered I/O:

统一的 iomap 设置函数

```

int f2fs_set_iomap(struct inode* inode, struct f2fs_map_blocks* map,
                  struct iomap* iomap, unsigned int flags,
                  loff_t offset, loff_t length, bool dio)
{
    iomap->offset = F2FS_BLK_TO_BYTES(map->m_lblk);

    if (map->m_flags & F2FS_MAP_MAPPED) {
        if (dio) {
            if (WARN_ON_ONCE(map->m_pblk == NEW_ADDR))
                return -EINVAL;
        }
        iomap->length = F2FS_BLK_TO_BYTES(map->m_len);
        iomap->bdev = map->m_bdev;

        if (map->m_pblk != NEW_ADDR) {
            iomap->type = IOMAP_MAPPED;
            iomap->flags |= IOMAP_F_MERGED;
            iomap->addr = F2FS_BLK_TO_BYTES(map->m_pblk);
        } else {
            iomap->type = IOMAP_UNWRITTEN;
            iomap->addr = IOMAP_NULL_ADDR;
        }
    } else {
        if (dio && flags & IOMAP_WRITE)
            return -ENOTBLK;

        if (map->m_pblk == NULL_ADDR) {
            // 关键: 区分两种空洞类型
            if (map->m_flags & F2FS_MAP_NODNODE)
                iomap->length = F2FS_BLK_TO_BYTES(*map->m_next_pgofs) -
                    iomap->offset;

            else
                iomap->length = F2FS_BLK_TO_BYTES(map->m_len);
            iomap->type = IOMAP_HOLE;
        } else if (map->m_pblk == NEW_ADDR) {
            iomap->length = F2FS_BLK_TO_BYTES(map->m_len);
            iomap->type = IOMAP_UNWRITTEN;
        } else {
            f2fs_bug_on(F2FS_I_SB(inode), 1);
        }
        iomap->addr = IOMAP_NULL_ADDR;
    }

    if (map->m_flags & F2FS_MAP_NEW)
        iomap->flags |= IOMAP_F_NEW;
    if ((inode->i_state & I_DIRTY_DATASYNC) ||
        offset + length > i_size_read(inode))

```

```

        iomap->flags |= IOMAP_F_DIRTY;

    return 0;
}

```

简洁的 buffered read iomap_begin 实现 基于统一的设置函数, f2fs_buffered_read_iomap_begin 的实现变得非常简洁:

f2fs_buffered_read_iomap_begin 实现

```

static int
f2fs_buffered_read_iomap_begin(struct inode *inode, loff_t offset,
                               loff_t length, unsigned int flags,
                               struct iomap *iomap, struct iomap *srcmap)
{
    pgoff_t next_pgofs = 0;
    int err;
    struct f2fs_map_blocks map = {};

    map.m_lblk = F2FS_BYTES_TO_BLK(offset);
    map.m_len = F2FS_BYTES_TO_BLK(offset + length - 1) - map.m_lblk + 1;
    map.m_next_pgofs = &next_pgofs;
    map.m_seg_type = f2fs_rw_hint_to_seg_type(F2FS_I_SB(inode), inode->
        i_write_hint);
    map.m_may_create = false;

    if (is_sbi_flag_set(F2FS_I_SB(inode), SBI_IS_SHUTDOWN))
        return -EIO;

    if (flags & IOMAP_WRITE)
        return -EINVAL;

    err = f2fs_map_blocks(inode, &map, F2FS_GET_BLOCK_IOMAP);
    if (err)
        return err;

    if (WARN_ON_ONCE(map.m_pblk == COMPRESS_ADDR))
        return -EINVAL;

    return f2fs_set_iomap(inode, &map, iomap, flags, offset, length, false);
}

```

f2fs_map_blocks 的两条关键路径 f2fs_buffered_read_iomap_begin 利用 f2fs_map_blocks 函数来实现区间映射, 而 f2fs_map_blocks 内部有两条关键的查找路径:

f2fs_map_blocks 的双路径优化

路径 1: Extent Cache 快速查找 当 f2fs 启用了 extent cache 并且要查找的区间足够大时, 会首先尝试从 extent cache 中快速获取映射信息。

优点: 不用产生 IO, 可以直接不用循环, 一次得到起始块地址和长度

extent_info 结构体的核心成员:

- **fofs:** 文件内的起始逻辑偏移量
- **len:** extent 的长度 (以块为单位)
- **blk:** 对应的起始物理块号

```
/* f2fs_map_blocks_cached */
if (!f2fs_lookup_read_extent_cache(inode, pgoff, &ei))
    return false;
// 直接从 extent_info 计算映射
```

```
map->m_pblk = ei.blk + pgoff - ei.fofs;
map->m_len = min((pgoff_t)maxblocks,
                 ei.fofs + ei.len - pgoff);
```

路径 2: dnode 常规查找 当 extent cache 未命中时, f2fs 会通过查找 dnode of data 的方式逐步定位数据块。这是一个相对较慢的路径, 可能会产生 IO。

```
/* f2fs_map_blocks 中的常规路径 */
next_dnode:
set_new_dnode(&dn, inode, NULL, NULL, 0);
err = f2fs_get_dnode_of_data(&dn, pgofs, mode);
start_pgofs = pgofs; /*start_pgofs是循环的初始计数器*/
end_offset = ADDRS_PER_PAGE(&dn.node_folio->page, inode);
do {
    switch (flag) {
```

```
        case F2FS_GET_BLOCK_PRECACHE:
            goto sync_out;
        case F2FS_GET_BLOCK_IOMAP:
            if (map->m_next_pgofs)
                *map->m_next_pgofs = pgofs + 1;
            break;
```

```
    default:
        /* for defragment case */
        if (map->m_next_pgofs)
            *map->m_next_pgofs = pgofs + 1;
```

```

        goto sync_out;

    }
    if (map->m_len == 0) { /*如果m_len还是处在刚初始化的阶段*/
        map->m_pblk = blkaddr;
        map->m_len = 1;
    } else if (map_is_mergeable(sbi, map, blkaddr, flag, bidx, ofs)) {
        /*重点代码，f2fs会尝试进行块映射合并，对读来说最重要*/
        ofs++;
        map->m_len++;
    } else {
        goto sync_out; /*不能合并直接break循环*/
    }
skip:
    dn.ofs_in_node++;
    pgofs++;
} while(dn.ofs_in_node < end_offset)

```

当得到了 `f2fs_map_blocks` 中的块映射信息后，它能直接转换成 `iomap` 结构体中连续的一段 LBA 字节地址。

而 f2fs 原生的读 API `f2fs_read_single_page` 只能一次将一个页索引映射到一个文件系统块地址上。我们还是对比着看两端代码实现：

page-by-page VS. extent-based 映射

旧： `f2fs_read_single_page` 一次只把一个 4 KB page 的逻辑块号丢进 `f2fs_map_blocks`，得回单块结果。缺点：持续 syscall/read 时要循环 N 次调用，CPU 和锁开销高。

```

/* drivers/f2fs/data.c */
map.m_lblk = block_in_file;
map.m_len = 1; /* 固定 1 */
f2fs_map_blocks(inode, &map, F2FS_GET_BLOCK_DEFAULT);

```

新： `f2fs_buffered_read_iomap_begin` 一次把 (pos, len) 整段传给 `f2fs_map_blocks`，然后“翻译”为 `iomap`

优点：只要 extent 连续，就能一次拿完整的区段——随后外层 `iomap_iter` 会不断前推 `iter.pos`。

```

map.m_lblk = F2FS_BYTES_TO_BLK(offset);
map.m_len = F2FS_BYTES_TO_BLK(offset+length-1) - map.m_lblk + 1;
f2fs_map_blocks(inode, &map, F2FS_GET_BLOCK_IOMAP);

```



```
iomap->offset = F2FS_BLK_TO_BYTES(map.m_lblk);
iomap->length = F2FS_BLK_TO_BYTES(map.m_len);
iomap->addr   = F2FS_BLK_TO_BYTES(map.m_pblk);
```

块合并策略的关键作用 我们分析到这里，有个问题是，如果一开始 extent cache 里什么信息都没有，那 iomap 拿到的还是一段连续的块号吗？会不会只能拿到单一的块号呢？这里我们就要引出 `f2fs_map_blocks` 中另一个关键的逻辑——它对连续块的合并策略。

f2fs_map_blocks 块合并策略

```
/* 摘自 f2fs_map_blocks */
static bool map_is_mergeable(struct f2fs_sb_info *sbi,
                             struct f2fs_map_blocks *map,
                             block_t blkaddr, unsigned int flag,
                             int bidx, unsigned int ofs)
{
    /* 物理 & 逻辑必须连续 */
    return (map->m_pblk + map->m_len == blkaddr) &&
        (map->m_next_extent == pgofs);
}

/* 每找到一个连续块就累计到 map->m_len */
if (map_is_mergeable(sbi, map, blkaddr, flag, bidx, ofs)) {
    ofs++;
    map->m_len++;           /* 把长度做大，留给 iomap 一次吞掉 */
} else {
    goto sync_out;         /* 不能合并就退出循环 */
}
```

并且，当我们合并了连续的块之后，就可以将这次得到的信息给缓存起来，存到 extent cache 之中。这里面我们就要重点关注之前 switch 里的标志 `F2FS_GET_BLOCK_IOMAP` 了。正是这个标志位的设置，启用了 f2fs 的 extent cache 更新。

精确的读取区间调整: `iomap_adjust_read_range` iomap 框架的一个重要优势是能够精确地计算和调整实际需要读取的区间，避免不必要的 I/O 操作。这个功能主要由 `iomap_adjust_read_range` 函数实现。

iomap 调整读取区间策略

```
/*
 * 计算 folio 内实际需要读取的区间
 * 这个函数会考虑：
 * 1. 已经 uptodate 的块（避免重复读取）
 * 2. 超过文件大小的部分（需要特殊处理）
 */
```

```

* 3. 块大小与页大小的关系
*/
static void iomap_adjust_read_range(struct inode *inode, struct folio *folio
,
    loff_t *pos, loff_t length, size_t *offp, size_t *lenp)
{
    struct iomap_folio_state *ifs = folio->private;
    loff_t orig_pos = *pos;
    loff_t isize = i_size_read(inode);
    unsigned block_bits = inode->i_blkbits;
    unsigned block_size = (1 << block_bits);
    size_t poff = offset_in_folio(folio, *pos);
    size_t plen = min_t(loff_t, folio_size(folio) - poff, length);

    // 关键优化: 跳过已经 uptodate 的前导块
    if (ifs) {
        unsigned int i;
        for (i = first; i ≤ last; i++) {
            if (!ifs_block_is_uptodate(ifs, i))
                break;
            *pos += block_size;
            poff += block_size;
            plen -= block_size;
            first++;
        }

        // 截断已经 uptodate 的尾部块
        while (++i ≤ last) {
            if (ifs_block_is_uptodate(ifs, i)) {
                plen -= (last - i + 1) * block_size;
                last = i - 1;
                break;
            }
        }
    }

    // 处理跨越文件末尾的情况
    if (orig_pos ≤ isize && orig_pos + orig_plen > isize) {
        unsigned end = offset_in_folio(folio, isize - 1) >> block_bits;
        if (first ≤ end && last > end)
            plen -= (last - end) * block_size;
    }

    *offp = poff;
    *lenp = plen;
}

```

关键优势:

- **避免重复 I/O**: 精确跟踪 folio 中每个块的状态, 跳过已经是最新的部分
- **EOF 边界处理**: 正确处理跨越文件末尾的读取请求
- **块级粒度控制**: 即使在高阶 folio 中也能实现块级的精确控制

iomap 的双循环模型与并发 I/O 处理 `iomap_readahead` 的实现采用了一个优雅的”外循环 + 内循环”模型, 这种设计为并发 I/O 处理奠定了基础。

循环结构对比

旧架构: 单循环 `f2fs_mpage_readpages` 只有一个简单的循环, 逐个处理 `readahead_control` 中的所有页。

```
/* f2fs_mpage_readpages */
for (; nr_pages; nr_pages--) {
    folio = readahead_folio(rac);
    // ...
    ret = f2fs_read_single_page( ... );
    /*begin f2fs_read_single_page*/
    if (!bio_add_folio(bio, folio, blocksize, 0))
        goto submit_and_realloc;
    /*end f2fs_read_single_page*/
}
```

新架构: 双循环 `iomap_readahead` 通过内外两层循环, 将块映射和页处理解耦。

```
/* iomap_readahead & iomap_readahead_iter */
// 外循环: 获取 iomap 区间
while (iomap_iter(&iter, ops) > 0)
    // 内循环: 处理该区间内的所有 folio
    iter.status = iomap_readahead_iter(&iter, &ctx);

// --- 内循环实现 ---
static int iomap_readahead_iter( ... )
{
    while (iomap_length(iter)) {
        // ...
        ctx->cur_folio = readahead_folio(ctx->rac);
        ret = iomap_readpage_iter(iter, ctx);
        // ...
    }
}
```

read_bytes_pending 的完整并发控制机制 这种双循环模型引出了对并发 I/O 的优雅处理。当一个高阶 folio 需要多次 bio 提交才能读完时, 如果第一次 bio 完成后就解锁 folio, 会导致数据竞争。iomap 通过 `read_bytes_pending` 和巧妙的提交流程解决了这个问题。

让我们看看这个机制的完整流程:

第一步：提交前的计数增加

read_bytes_pending 机制

```
/* iomap_readpage_iter 中的关键代码 */
ctx->cur_folio_in_bio = true;
if (ifs) {
    spin_lock_irq(&ifs->state_lock);

    ifs->read_bytes_pending += plen; // 关键：先增加待读字节数

    spin_unlock_irq(&ifs->state_lock);
}

// 然后才添加到 bio 并可能提交，并且能一次性添加folio的大段区间到bio

bio_add_folio_nofail(ctx->bio, folio, plen, poff);
```

这个顺序非常关键：我们必须在将 folio 添加到 bio 之前就增加 read_bytes_pending，这样即使 bio 立即完成，也不会导致 folio 被提前解锁。

第二步：I/O 完成时的计数减少

I/O 完成时的并发控制

```
/* I/O 完成回调链 */
static void iomap_read_end_io(struct bio *bio)
{
    int error = blk_status_to_errno(bio->bi_status);
    struct folio_iter fi;

    bio_for_each_folio_all(fi, bio)
        iomap_finish_folio_read(fi.folio, fi.offset, fi.length, error);
    bio_put(bio);
}

static void iomap_finish_folio_read(struct folio *folio, size_t off,
    size_t len, int error)
{
    struct iomap_folio_state *ifs = folio->private;
    bool uptodate = !error;
    bool finished = true;

    if (ifs) {
        unsigned long flags;
```

```
spin_lock_irqsave(&ifs->state_lock, flags);
if (!error)
    uptodate = ifs_set_range_uptodate(folio, ifs, off, len);

ifs->read_bytes_pending -= len; // 关键：减少待读字节数

finished = !ifs->read_bytes_pending; // 检查是否全部完成
spin_unlock_irqrestore(&ifs->state_lock, flags);
}

if (finished)
    folio_end_read(folio, uptodate); // 只有全部完成才解锁
}
```

关键的流水线安全性论证：

考虑这样一个场景：一个 16KB 的高阶 folio 需要通过两个 8KB 的 bio 来读取。

1. **T1 时刻**：第一个 bio 创建，read_bytes_pending=8192
2. **T2 时刻**：第一个 bio 提交到块设备
3. **T3 时刻**：第二个 bio 创建，read_bytes_pending=16384
4. **T4 时刻**：第二个 bio 提交到块设备
5. **T5 时刻**：第一个 bio 完成，read_bytes_pending=8192（仍 > 0，不解锁）
6. **T6 时刻**：第二个 bio 完成，read_bytes_pending=0（解锁 folio）

这种设计保证了无论 bio 完成的顺序如何，folio 都只会在所有相关的 I/O 都完成后才被解锁，完美解决了并发问题。

空洞文件处理的改进 空洞文件（sparse file）是指文件中某些区域没有实际分配物理存储空间的文件。iomap 框架在处理空洞文件时展现了其设计的优越性。

在 f2fs 的 `f2fs_map_blocks` 中，空洞的检测其实和普通块有点像，但是目前对于 `F2FS_GET_BLOCK_IOMAP` 会根据不同情况进行处理。另外，这里面会区分 `dnode` 结构体是否被分配过。未分配的话，空洞的长度会由 `f2fs_get_next_page_offset` 计算出来：

f2fs_map_blocks 中的空洞检测关键分支

```

/* 如果 dnode 结构体已经分配过 */
} else if (is_hole) { /* 不创建新块但是是空洞 */
    switch (flag) {
        case F2FS_GET_BLOCK_PRECACHE: /* 在碰到文件空洞的情况下直接跳出循环 */
            goto sync_out;
        case F2FS_GET_BLOCK_IOMAP:
            if (map->m_next_pgofs) /* 还没设置 f2fs_block_map 中的 next_pgofs 的话将其
            其设置为 +1 */
                *map->m_next_pgofs = pgofs + 1;
            break;
    }
}
map->m_pblk = blkaddr;

/* 如果没分配过 */
if (err == -ENOENT) {
    err = 0;
    if (map->m_next_pgofs)
        *map->m_next_pgofs = f2fs_get_next_page_offset(&dn, pgofs);
    if (map->m_next_extent)
        *map->m_next_extent = pgofs;

    // 关键: 不设置 F2FS_MAP_MAPPED 标志
    // 这样 iomap 就知道这是一个空洞区域
    goto unlock_out;
}

// 在 f2fs_set_iomap 中:
if (map->m_pblk == NULL_ADDR) {
    if (map->m_flags & F2FS_MAP_NODNODE)
        iomap->length = F2FS_BLK_TO_BYTES(*map->m_next_pgofs) - iomap->offset
        ;
    else
        iomap->length = F2FS_BLK_TO_BYTES(map->m_len);
    iomap->type = IOMAP_HOLE; // 标记为空洞
}

```

f2fs 与 iomap 空洞文件处理对比

旧架构: 分散的空洞判断 `f2fs_read_single_page` 需要在多个地方判断和处理空洞情况, 逻辑分散且容易出错。并且其实现只能对 0 阶 folio (通常是 4KB) 进行清零操作。

```

/* f2fs_read_single_page 中的空洞处理 */
} else if ((map->m_flags & F2FS_MAP_MAPPED)) {
    // 正常的映射块处理 ...
} else {

```

```

// 空洞处理, 此时只有单块长度
zero_out:
    folio_zero_segment(folio, 0, folio_size(folio));
    if (!folio_test_uptodate(folio))
        folio_mark_uptodate(folio);
    folio_unlock(folio);
    goto out;
}

```

新架构：统一的区间化处理 iomap 通过 `iomap_block_needs_zeroing` 提供统一的空洞判断接口，并能一次性处理大片区间。

关键优势：

- **区间化**：一次性处理 `plen` 长度的区间
- **高性能**：能够处理高阶 `folio` 的大片连续区域
- **统一性**：空洞、EOF 后区域、预分配块等都同一套逻辑

```

/* iomap_readpage_iter 中的统一处理 */
/* plen 是已经根据 iomap->length 和 iomap_adjust_read_range 调整过的大片区间 */
if (iomap_block_needs_zeroing(iter, pos)) {
    folio_zero_range(folio, poff, plen);
    iomap_set_range_uptodate(folio, poff, plen);
    goto done;
}

```

性能优势总结 基于 iomap 框架的 buffered read 实现相比传统的逐页处理方式，在多个方面都有显著的性能提升：

1. **减少函数调用开销**：一次 `iomap_begin` 调用可以处理多个连续页，而不是每页都要调用一次映射函数。
2. **提高 extent cache 命中率**：通过 `F2FS_GET_BLOCK_IOMAP` 标志位启用的 extent cache 更新策略，使后续访问能从缓存中快速获取映射信息。
3. **优化 bio 构建**：能够一次性将大片连续区间添加到 bio 中，减少 bio 的数量和提交次数。
4. **支持任意阶数 folios**：通过区间化处理天然支持高阶 folios，无需将其拆分为单个页面处理。
5. **精确的并发控制**：通过 `read_bytes_pending` 机制确保高阶 folio 的 I/O 完整性，避免部分读取导致的数据竞争。

这些改进使得基于 iomap 的 buffered read 实现能够充分发挥 largefolios 的性能潜力，为文

件系统的整体性能提升奠定了坚实的基础。

3.2 普通/原子文件 buffered write

3.2.1 功能背景

在我们深入代码细节之前，必须明确一个良好的缓冲写入（Buffered Write）机制需要实现的核心目标。这不仅是所有文件系统的基础功能，也是理解 iomap 框架带来优越性的关键。

1. **支持文件创建与顺序追加 (Append)**: 我们来思考一下，很多时候我们 touch 了一个新的文件，然后，我们可能用 vim 或者 echo 了很多文本进去。这个时候我们 vim 保存或者 cat 一下，是能看到文件内容的。但是，我们知道，touch 创建的新文件，其内容是空的。或者，我们往 vim 里追加了一段文字，但是我们知道，原本的文件没有这段内容，这点可以通过看之前的文件大小进行验证。这同时意味着，磁盘上原先是没有这段内容的。但是对于用户来说，我们就是能正常看到这段内容。这就是 buffer write 需要满足的第一个目标，能支持文件的顺序追加的写入，同时扩展文件的大小。当用户创建一个新文件并写入内容，或者在一个已有文件的末尾追加内容时，文件系统必须能够：

- 在逻辑上文件末尾的位置，按需分配新的物理块。
- 将用户数据写入到内存中的页缓存 (Page Cache / Folio)。
- 更新文件大小元数据。
- 最终由后台进程将这些“脏”页回写到磁盘上新分配的位置。

对用户而言，这个过程是透明的，他们只看到文件变长了，内容增加了。

2. **支持文件覆盖写 (Overwrite)**: 好，现在我们的文件有内容了，我们重启了一下系统，此时我们想编辑一下文件内容，很容易，打开 vim 然后到我们想改的地方改一下，立刻就能看到效果。但是我们细想一下，因为我们重启过系统，所以一开始内存中是没有缓存的文件数据的。那我们怎样将文件内容精确写到正确的位置的呢？毕竟我们在 buffered write 的场景下不能直接对着磁盘写入对吧。所以这时候 buffered write 要满足的第二个目标是同时要触发 buffered read，将文件内容读到内存中，然后我们修改内存中的内容，最后将内存中的内容写入磁盘。因此，文件系统必须：

- 首先，像读操作一样，找到待修改数据所在的物理块。
- 将这部分数据从磁盘读入页缓存（如果尚未缓存）。这个过程通常被称为“Read-Modify-Write”。
- 用户在内存中修改数据。
- 将修改后的脏页标记起来，等待回写。

这两个目标，特别是它们对块映射（找到已有块或分配新块）的依赖，是传统实现与 iomap 实现产生巨大差异的核心所在。接下来，我们将围绕这两个目标，逐一剖析 F2FS 在引入 iomap 前后的实现差异。

覆盖写：从手动挡到自动挡的块映射与锁管理 覆盖写操作的核心在于 `write_begin` 这个钩子函数。它负责在数据从用户空间拷贝到页缓存之前，完成所有准备工作：找到或分配块、读入旧数据、处理锁等。这正是传统 `f2fs_write_begin` 与新 `f2fs_buffered_write_iomap_begin` 分道扬镳的地方。

3.2.2 方案设计

`f2fs_write_begin` VS. `f2fs_buffered_write_iomap_begin`

旧： `prepare_write_begin` (`f2fs_write_begin` 的核心) 传统实现中，`prepare_write_begin` 像一个事必躬亲的“手动挡”司机。它为单个 folio 的写入，执行了一系列繁琐的底层操作。

缺点：

- **手动 dnode 查找：** 直接调用底层的 `f2fs_get_dnode_of_data`，处理复杂的 dnode 定位逻辑。
- **显式锁管理：** 需要自己调用 `f2fs_lock_op` 获取段锁，增加了逻辑复杂度和死锁风险。
- **手动资源平衡：** 需要显式检查 `need_balance` 标志，并可能调用 `f2fs_balance_fs` 来整理磁盘空间，这打断了核心写流程。
- **单块映射：** 其核心逻辑是围绕单个块地址 `blkaddr` 展开的，缺乏区间化思想。

```
/* f2fs/data.c -> prepare_write_begin */
static int prepare_write_begin( ... )
{
    // ...
```

```
set_new_dnode(&dn, inode, NULL, NULL, 0);
```

```
err = f2fs_get_dnode_of_data(&dn, index, ALLOC_NODE);
```

```
if (err)
    goto out;

*blkaddr = f2fs_data_blkaddr(&dn);
// ...
if (is_hole || ... ) {
    // ...
```

```
err = f2fs_reserve_new_block(&dn);
```

```
if (err)
```

```

        goto out;
        *blkaddr = NEW_ADDR;

```

```

*need_balance = true;

```

```

    }
    // ...
}

```

新：f2fs_buffered_write_iomap_begin iomap 版本的实现则像一个聪明的“自动挡”系统。它将所有复杂的块映射和资源管理工作，都委托给了我们之前分析过的 f2fs_map_blocks 函数。

优点：

- 优雅封装：无需关心 dnode、段锁等底层细节，只需调用高层抽象 f2fs_map_blocks_iomap。
- 逻辑复用：复用了 f2fs_map_blocks 中成熟的块查找、合并、缓存及锁管理逻辑。
- 区间化处理：天然地以区间 (start_blk, len_blks) 为单位进行操作，为处理高阶 Folio 奠定基础。
- 代码简洁：回调处理本身变得极其简短和清晰，只负责调用映射函数并转换结果。

```

/* 我的 f2fs_buffered_write_iomap_begin 实现 */
static int f2fs_buffered_write_iomap_begin( ... )
{
    // ...
    block_t start_blk = F2FS_BYTES_TO_BLK(pos);
    block_t len_blks = F2FS_BYTES_TO_BLK(pos + length - 1) - start_blk + 1;

```

```

err = f2fs_map_blocks_iomap
      (inode, start_blk, len_blks, &map);

```

```

if (err && err != -ENOSPC)
    return err;

```

```

// 如果是空洞，则尝试预分配
if (map.m_pblk == NULL_ADDR) {

```

```
err = f2fs_map_blocks_preallocate
    (inode, map.m_lblk, map.m_len, &map);

    if (err && err != -ENOSPC)
        return err;
}

// ... 将 map 结果转换为 iomap 结构体
return 0;
}
```

顺序追加写：从单点预留到区间预分配的飞跃 对于顺序追加写，性能的关键在于能否高效地预留 (pre-allocate) 连续的空闲块。在这里，传统方法和 iomap 方法的差距被进一步拉大。F2FS 使用一个特殊的地址 NEW_ADDR 来标记一个被延迟分配 (delalloc) 的块。

f2fs_map_blocks 对 NEW_ADDR 的合并支持

```
/* 在 f2fs_map_blocks 中, map_is_mergeable 可以合并连续的 NEW_ADDR */
static inline bool map_is_mergeable(struct f2fs_sb_info *sbi, struct
f2fs_map_blocks *map, block_t blkaddr, int flag, int bidx, int &ofs)
{
    // ...
    /* 关键逻辑：如果当前块和之前的块都是 NEW_ADDR, 则可以合并 */
    if (map->m_pblk == blkaddr && blkaddr == NEW_ADDR)
        return true;

    // 或者物理上连续
    if (map->m_pblk + map->m_len == blkaddr)
        return true;

    // ...
    return false;
}
```

这个合并能力是实现区间化预分配的基础，但传统方法由于其“单点”思维，未能充分利用它。

单点预留 VS. 区间预分配

旧：prepare_write_begin 由于 prepare_write_begin 的设计是面向单个 folio，并且其内部调用 f2fs_reserve_new_block 一次只分配一个块，因此在连续写入时，它实际上是在进行“分配一个，写入一个，再分配一个...”的循环。

缺点：

- **性能瓶颈:** 每次追加写入都需要重新查找 dnode 并分配一个新块, 元数据操作开销大。
- **无法利用合并:** 即使底层有合并能力, 但上层每次只请求一个块, 使其英雄无用武之地。

```
/* prepare_write_begin 只能返回单个 NEW_ADDR */
if (is_hole) {
    // ...
    err = f2fs_reserve_new_block(&dn);
    // ...
}
```

```
*blkaddr = NEW_ADDR;
```

```
// 一次只有一个
}
```

新: `f2fs_buffered_write_iomap_begin` iomap 方案通过一个专门的辅助函数 `f2fs_map_blocks_preallocate`, 将预分配的意图直接传递给 `f2fs_map_blocks`。它使用了 `F2FS_GET_BLOCK_PRE_AIO` 标志, 这会触发 `f2fs_map_blocks` 内部的批量预分配逻辑。

优点:

- **批量分配:** 一次性为整个写入区间 (可能跨越多个 folio) 请求预分配, 大大减少了元数据操作。
- **减少碎片:** 批量分配更有可能获得物理上连续的大块空间。
- **充分利用合并:** 底层的 `map_is_mergeable` 逻辑被完全激活, 可以返回一个横跨多个逻辑块的、由 `NEW_ADDR` 组成的巨大映射。

```
/* f2fs_map_blocks_preallocate -> f2fs_map_blocks */
int f2fs_map_blocks_preallocate(struct inode *inode, block_t start, block_t
    len, struct f2fs_map_blocks *map)
{
    // ...
    map->m_len = len; // 传入期望的预分配长度
}
```

```
err = f2fs_map_blocks(inode, map, F2FS_GET_BLOCK_PRE_AIO);
```

```
return err;
}
```

```

/* f2fs_map_blocks 内部的批量预留逻辑 */
if (flag == F2FS_GET_BLOCK_PRE_AIO &&
    (pgofs == end || dn.ofs_in_node == end_offset)) {
    dn.ofs_in_node = ofs_in_node;

    err = f2fs_reserve_new_blocks(&dn, prealloc);

    // 批量预留
    if (err)
        goto sync_out;

    map->m_len += dn.ofs_in_node - ofs_in_node;
    map->m_pblk = dn.data_blkaddr; // 可能是 NEW_ADDR
    map->m_flags |= F2FS_MAP_MAPPED;
}

```

原子文件: 逻辑复用的威力 对于原子文件 (Atomic Files), 其写操作遵循写时复制 (Copy-on-Write) 原则, 写入时会先操作一个临时的 `cow_inode`。iomap 的优势在这里体现为代码的极致复用。

f2fs_buffered_write_atomic_iomap_begin 的优雅实现

```

/* 我的 f2fs_buffered_write_atomic_iomap_begin 实现 */
static int f2fs_buffered_write_atomic_iomap_begin( ... )
{
    struct inode *cow_inode = F2FS_I(inode)->cow_inode;
    // ...

    // 1. 先尝试从 cow_inode 映射块

    err = f2fs_map_blocks_iomap(cow_inode, start_blk, len_blks, &map);

    // 2. 如果 cow_inode 中没有, 则尝试预分配
    if (map.m_pblk == NULL_ADDR && ... ) {

        err = f2fs_map_blocks_preallocate(cow_inode, ... );

        goto out;
    } else if (map.m_pblk != NULL_ADDR) {
        return 0;
    }

    // 3. 如果 cow_inode 中没有且无需预分配 (例如覆盖写), 则从原 inode 找

```

```
err = f2fs_map_blocks_iomap(inode, start_blk, len_blks, &map);
```

```
out:
    // ... 将 map 结果转换为 iomap
    return 0;
}
```

可以看到，无论是操作 `cow_inode` 还是原 `inode`，无论是查找已有块还是预分配新块，所有操作都统一到了 `f2fs_map_blocks_iomap` 和 `f2fs_map_blocks_preallocate` 这两个高层 API 上。相比之下，传统的 `prepare_atomic_write_begin` 需要在内部重复大量的、与 `prepare_write_begin` 类似的 `dnode` 手动查找和分配逻辑，显得非常冗余。iomap 框架通过良好的抽象，让复杂的原子写逻辑变得清晰而健壮。

3.2.3 框架级对比: Folio 生命周期与脏区标记的精细化

除了块映射，iomap 框架在更高层面上也带来了革命性的改进，尤其是在 Folio 的生命周期管理和脏区标记方面。

Folio 生命周期管理: 手动 VS. 自动

旧: `generic_perform_write` 框架 在旧的 VFS 写入框架中，文件系统扮演着繁重的角色。

- 手动管理: `a_ops->write_begin` 必须手动查找或创建 folio，并返回一个锁定的 folio。
- 手动释放: `a_ops->write_end` 在完成工作后，必须手动解锁并释放 folio。
- 竞争风险: 在 `write_begin` 中，必须手动检查 folio 是否被并发截断 (`folio->mapping!=mapping`)，这种竞争条件处理起来非常棘手，容易出错。

```
/* f2fs_write_begin */
repeat:
    folio = __filemap_get_folio(mapping, index, FGP_LOCK | ... );
    // ...
    if (need_balance && ...) {
```

```
folio_unlock(folio);
```

```
f2fs_balance_fs(sbi, true);
```

```
folio_lock(folio);
```

```
if (folio->mapping != mapping) { // 手动处理竞争
    folio_unlock(folio);
```



```

        folio_put(folio);
        goto repeat;
    }
}
// ...
return 0;
put_folio:

```

```
folio_unlock(folio);
```

```
folio_put(folio);
```

```
return err;
```

新: `iomap_file_buffered_write` 框架 `iomap` 框架将 `Folio` 的生命周期管理完全自动化, 文件系统只需关注自己的核心逻辑。

- **自动管理:** `iomap` 框架在调用 `iomap_write_begin` 之前, 就已经通过 `__iomap_get_folio` 自动完成了 `folio` 的查找、创建和加锁。
- **安全上下文:** 文件系统回调函数在一个安全的、已锁定 `folio` 的上下文中执行, 无需担心并发截断等竞争问题。
- **逻辑简化:** 文件系统只需实现核心的块映射和数据准备逻辑, 大大降低了出错的可能性。

```

/* iomap_write_begin (VFS generic helper) */
static int iomap_write_begin( ... )
{
    // ...

```

```
folio = __iomap_get_folio(iter, pos, len);
```

```

// iomap 框架负责
if (IS_ERR(folio))
    return PTR_ERR(folio);

// ... 检查 iomap 是否陈旧

// 调用文件系统回调, 此时 folio 已锁定且安全

```

```
status = __iomap_write_begin(iter, pos, len, folio);
```

```

    if (unlikely(status))
        goto out_unlock; // 出错后框架负责解锁

    *foliop = folio;
    return 0;
}

```

脏区标记: 粗放 VS. 精准

旧: `f2fs_write_end` 传统的 `write_end` 实现对于脏区的标记是粗放式的。

缺点: 无论实际写入了多少字节, 它都会将 **整个 folio** 标记为脏。如果在一个 64KB 的高阶 folio 中只修改了 100 字节, 这种做法会导致整个 64KB 在回写时都被写入磁盘, 造成严重的写放大 (Write Amplification)。

```

/* f2fs_write_end */
if (!copied)
    goto unlock_out;

```

```
folio_mark_dirty(folio);
```

// 将整个 folio 标记为脏

```

if (pos + copied > i_size_read(inode) && ...) {
    f2fs_i_size_write(inode, pos + copied);
}
// ...

```

新: `__iomap_write_end` iomap 框架提供了精细化的脏区标记能力。

优点: iomap 框架的 `write_end` 知道写入的起始位置 `pos` 和实际拷贝的长度 `copied`, 因此它可以只将 folio 内被修改的 **精确区间**标记为脏。这极大地减少了不必要的 I/O, 提高了效率, 并延长了闪存设备的使用寿命。

```

/* __iomap_write_end (VFS generic helper) */
static bool __iomap_write_end(struct inode *inode, loff_t pos, size_t len,
    size_t copied, struct folio *folio)
{
    // ...
    if (unlikely(copied < len && !folio_test_uptodate(folio)))
        return false;

    // ...
}

```

```
iomap_set_range_dirty(folio, offset_in_folio(folio, pos), copied);

filemap_dirty_folio(inode->i_mapping, folio);
return true;
}
```

3.2.4 写入失败与短写处理: 精准的“空间回收”机制

当一次写入操作未能完全成功时（例如，`copy_from_user` 中途失败），一个健壮的文件系统必须能够回收为这次失败的写入而预留的磁盘空间。这是 `iomap` 框架展现其设计深度的另一个领域。

错误处理: 粗暴截断 VS. 精准回收

旧: `f2fs_write_failed` F2FS 的传统错误处理函数 `f2fs_write_failed` 是一种非常“粗暴”的机制。

缺点:

- 操作过重: 它会直接将文件截断回写入前的大小，并丢弃所有相关的页缓存和块。
- 缺乏灵活性: 它无法处理发生在文件中间的短写失败。如果一个大文件中间的写入失败，它不能只回收那一部分空间，而是可能导致后续数据丢失。
- 不适用于复杂场景: 对于已经有脏页和预留空间交织的复杂情况，这种简单的截断是无能为力的。

```
/* f2fs_write_failed */
void f2fs_write_failed(struct inode *inode, loff_t to)
{
    loff_t i_size = i_size_read(inode);

    if (to > i_size && ...) {
        // ...

truncate_pagecache(inode, i_size);

f2fs_truncate_blocks(inode, i_size, true);

        // ...
    }
}
```

新: `iomap_write_delalloc_release` `iomap` 框架提供了一套极其精密的“延迟分配空间回收”机制，通过 `iomap_end` 回调触发。

优点:

- **外科手术式回收:** 它能精确地找出为写入预留 (DELALLOC) 但最终没有被脏数据覆盖的区域。
- **感知页缓存:** 它通过 `mapping_seek_hole_data` 扫描页缓存，只保留那些真正有脏数据支撑的预留空间，而将其余的回收。
- **原子性与安全性:** 整个过程在 `invalidate_lock` 的保护下进行，确保了与页缓存操作的原子性，避免了竞争。

```
/* iomap_write_delalloc_release (VFS generic helper) */
void iomap_write_delalloc_release(struct inode *inode, loff_t start_byte,
    loff_t end_byte, ..., iomap_punch_t punch)
{
    loff_t punch_start_byte = start_byte;
    // ...
    lockdep_assert_held_write(&inode->i_mapping->invalidate_lock);

    // 1. 在预留空间内, 查找有真实数据的区域
    while (start_byte < scan_end_byte) {
        start_byte = mapping_seek_hole_data(..., SEEK_DATA);
        // ...

        // 2. 对于数据区域之间的空隙, 调用 punch 回收
        iomap_write_delalloc_scan(inode, &punch_start_byte, start_byte,
            data_end, iomap, punch);

        start_byte = data_end;
    }

    // 3. 回收末尾剩余的预留空间
    if (punch_start_byte < end_byte)

        punch(inode, punch_start_byte, end_byte - punch_start_byte, iomap);
}
```

3.3 f2fs_iomap_folio_state

3.3.1 功能背景

在 Linux 内核的传统设计中, `page->private` (现在是 `folio->private`) 字段是一个 `void*` 类型的指针, 被文件系统广泛用于存储与特定内存页相关的私有状态。F2FS 也不例外, 它利用这个字

段的各个比特位来存储一系列标志，如 `PAGE_PRIVATE_ATOMIC_WRITE`（原子写）、`PAGE_PRIVATE_ONGOING_MIGRATION`（GC 迁移中）等。

然而，当 `iomap` 框架引入后，它也需要使用 `folio->private` 来存储一个指向 `iomap_folio_state` 结构体的指针。这个结构体对于实现高阶 Folio 的逐块脏状态跟踪和并发 I/O 控制至关重要。

核心冲突 (The Core Conflict)

- **F2FS 的需求**: 需要在 `folio->private` 中存储多个布尔型标志位。
- **iomap 的需求**: 需要在 `folio->private` 中存储一个指向 `iomap_folio_state` 结构体的指针。

这两者在 `folio->private` 这个有限的资源上产生了直接冲突。更复杂的是，我们为 F2FS 压缩文件设计的 `cc_bytes_pending` 计数器也需要一个存储位置。我们不能简单地抛弃任何一方，必须找到一种方法，将 F2FS 的私有状态与 `iomap` 的状态跟踪机制融合到一个统一的结构中。

3.3.2 重大问题分析

我们已经在问题背景中分析过 `f2fs_iomap_folio_state` 的必要性。若没有它，导致的问题现象，就是在任何需要使用 `f2fs_iomap_folio_state` 的地方，以及 `iomap` 框架中需要访问 `iomap_folio_state` 的代码，会将其（`folio->private` 中存储的 F2FS 标志位）误认为是一个指针值，导致内核在形式上抛出空指针解引用的异常。

当我们将 `f2fs_iomap_folio_state` 集成到代码中时，我发现真是牵一发而动全身。任何原先使用旧的 `set_page_private/get_page_private` 的代码路径全部都需要更改。这里面涉及到的代码路径不光有普通数据 `folio`，还有元数据 `folio` 中存储自己 `private` 字段的地方也需要替换成我们新设计的接口。

举个例子，Page Cache 中的 `f2fs_release_folio` 和 `f2fs_is_compressed_folio`。它们看似是 F2FS 内部函数，但其调用栈可能深入到通用的 VFS 和 `iomap` 框架中，如下所示：

f2fs_release_folio 调用栈示例

```
Call trace:
show_stack+0x18/0x24 (C)
dump_stack_lvl+0x78/0x90
dump_stack+0x18/0x24
f2fs_release_folio+0x18/0x50 <---期望private flag
filemap_release_folio+0x78/0xbc
mapping_evict_folio+0x8c/0xac
mapping_try_invalidate+0xbc/0x1a0
invalidate_mapping_pages+0x14/0x20
drop_pagecache_sb+0xa4/0x128
iterate_supers+0x88/0x100
drop_caches_sysctl_handler+0x7c/0x190
proc_sys_call_handler+0x17c/0x258
proc_sys_write+0x14/0x20
vfs_write+0x2b0/0x35c
ksys_write+0x68/0xfc
```

```

__arm64_sys_write+0x1c/0x28
invoke_syscall+0x48/0x110
el0_svc_common.constprop.0+0x40/0xe8
do_el0_svc+0x20/0x2c
el0_svc+0x30/0xd8
el0t_64_sync_handler+0x144/0x168
el0t_64_sync+0x198/0x19c

```

同时，我最终还惊讶地发现，初赛时我认为本可以和 `iomap` 框架自身保持一致地不为 0 阶 folio 分配 `f2fs_iomap_folio_state`，然而在我遭遇了下面这个问题之后，我的想法彻底改变了。

问题显现：0 阶 Folio 的致命冲突

GC 代码路径中的 `set_page_private_gcing` 调用

```

/* move_data_page in gc.c */
static int move_data_page(struct inode *inode, block_t bidx, int gc_type,
                          unsigned int segno, int off)
{
    struct folio *folio;
    // ...
    folio = f2fs_get_lock_data_folio(inode, bidx, true);
    // ...
    if (gc_type == BG_GC) {
        // ...
        folio_mark_dirty(folio);

```

```
set_page_private_gcing(&folio->page);
```

```

// <- 问题根源
} else {
    // ...

```

```
set_page_private_gcing(&folio->page);
```

```

// <- 问题根源
    err = f2fs_do_write_data_page(&fio);
    // ...
}
out:
    f2fs_folio_put(folio, true);
    return err;
}

```

下面我们详细地描述一下整个过程：

1. F2FS 的垃圾回收 (GC) 进程在 `move_data_page` 中处理一个 0 阶 Folio。
2. 代码调用 `set_page_private_gcing`, 这个宏将 `folio->private` 设置为一个包含了 `ONGOING_MIGRATION` 标志的小整数值。
3. 稍后, 用户或系统进程可能对同一个文件发起写操作, 进入了 `iomap_buffered_write` 路径。
4. 在 `iomap` 框架内部, 代码检查到 `folio->private` 非空, 于是它假定这一定是一个指向 `iomap_folio_state` 结构体的有效指针。
5. `iomap` 代码尝试解引用这个“指针”去访问其成员 (例如 `ifs->state_lock`)。
6. 然而, `folio->private` 此时的值是一个类似 `0x4` 的小整数, 并非一个合法的内存地址。
7. **内核崩溃!** 系统抛出空指针解引用 (NULL pointer dereference) 或页错误 (Page Fault) 异常。

这个发现迫使我们重新设计方案, 确保即使是 0 阶 Folio, 在与 `iomap` 交互的场景下, 也必须使用与高阶 Folio 一致的状态结构。

3.3.3 方案设计

f2fs_iomap_folio_state 的诞生 经过 large folios 创始人 Matthew Wilcox 先生的指点, 我决定创建一个 F2FS 专属的、与 `iomap_folio_state` 兼容但功能针对 F2FS 需求进行了扩展的结构体: `f2fs_iomap_folio_state`。

f2fs_iomap_folio_state 的结构设计

```
/* f2fs_ifs.h */
struct f2fs_iomap_folio_state {
    spinlock_t state_lock;
    unsigned int read_bytes_pending;
    atomic_t write_bytes_pending;

    /*
     * Flexible array member.
     * Holds [0...iomap_longs-1] for iomap uptodate/dirty bits.
     * Holds [iomap_longs] for F2FS private flags/data (unsigned long).
     * Holds [iomap_longs+1] for dirty_bytes_pending (cc_bytes_pending)
     */
    unsigned long state[];
};
```

这个设计的精妙之处在于：

1. **内存布局兼容性**: 它的前三个成员 (`state_lock`, `read_bytes_pending`, `write_bytes_pending`) 与 `iomap_folio_state` 的定义完全一致。
这意味着，任何操作 `iomap_folio_state` 的通用 `iomap` 函数，都可以无缝地操作一个 `f2fs_iomap_folio_state` 实例的头部，因为它们的内存布局是相同的。
2. **柔性数组扩展**: 我们利用 C 语言的柔性数组成员 `state[]`，在 `iomap_folio_state` 所需的内存空间之后，额外分配了两个 `unsignedlong` 的空间。
 - 第一个额外空间用于存储 F2FS 自己的所有私有标志位。
 - 第二个额外空间被用作我们为压缩文件设计的 `cc_bytes_pending` 原子计数器。

核心实现: 智能分发的 Folio 私有数据 API 为了取代旧的 `page_private` 宏，我们设计了一整套新的宏，用于生成对 `folio->private` 进行获取 (Get)、设置 (Set) 和清除 (Clear) 操作的函数。这套新 API 的精髓在于其 ** 智能的、不依赖 Folio 阶数的多级分发策略 **。

我们利用了 F2FS 原有标志位设计中的 `PAGE_PRIVATE_NOT_POINTER` 位，它天生就能区分 `folio->private` 中存储的是“标志位 **”还是一个“指针”。我们的 `f2fs_iomap_folio_state` 恰好就是一个指针，这使得新旧两种模式可以完美共存和区分。

Get/Clear 操作的多级判断逻辑: 对于获取 (Get) 或清除 (Clear) 标志位的操作，函数会首先直接测试 `PAGE_PRIVATE_NOT_POINTER` 标志位。

- 如果标志位被设置: 说明 `folio->private` 存储的是旧式的标志位。函数会直接在此值上进行位操作并返回。这是一个高效的快速路径，完美兼容了元数据 Folio 或其他不与 `iomap` 交互的场景。
- 如果标志位未被设置: 说明 `folio->private` 是一个指针 (或为 NULL)。函数则会继续检查该指针指向的结构体是否包含我们的魔数 `F2FS_IFS_MAGIC`。如果匹配，就在 `f2fs_iomap_folio_state` 的扩展区域进行操作。

Set 操作的智能分配逻辑: 设置 (Set) 操作的逻辑更为精妙，它需要决定是否要分配 `f2fs_iomap_folio_state` 结构体。

- 它会先通过 `f2fs_should_use_buffered_iomap` 判断当前 Folio 是否可能进入 `iomap` 的缓冲写路径。
- 如果不会进入，且 `folio->private` 当前为空，那么它会沿用旧逻辑，直接在 `folio->private` 中设置标志位。
- 但如果会进入 `iomap` 路径 (由 `force_alloc` 参数体现)，或者 `folio->private` 已经有值 (可能是标准的 `iomap_folio_state`)，那么它必须走 `f2fs_ifs_alloc` 的完整路径。

这个设计确保了即使是 0 阶 Folio，一旦它处于可能引发冲突的代码路径上，也会被强制分配一个 `f2fs_iomap_folio_state` 作为占位，从而从根本上解决了问题。

私有标志位设置函数对比

旧: `PAGE_PRIVATE_SET_FUNC` 宏 旧的宏直接操作 `page->private` 指针, 通过位操作来设置标志。缺点:

- Page-centric: 完全基于 `structpage`, 无法适应 Folio。
- 冲突: 与 `iomap` 的 `iomap_folio_state` 指针存储需求直接冲突。
- 不安全: 没有对并发访问的保护机制。

`PAGE_PRIVATE_SET_FUNC` (原始)

```
/* f2fs.h (原始) */
#define PAGE_PRIVATE_SET_FUNC(name, flagname) \
static inline void set_page_private_##name(struct page *page) \
{ \
    if (!PagePrivate(page)) \
        attach_page_private(page, (void *)0); \
    set_bit(PAGE_PRIVATE_NOT_POINTER, &page_private(page)); \
\
    set_bit(PAGE_PRIVATE_##flagname, &page_private(page)); \
\
}
```

新: `F2FS_FOLIO_PRIVATE_SET_FUNC` 宏 我们全新的宏, 实现了对不同场景的智能分发处理。

优点:

- Folio-centric: 统一了 folio 的私有数据访问接口。
- 智能分发: 通过检查 `NOT_POINTER` 标志和 `force_alloc` 条件, 智能地在“直接位操作”和“分配 `f2fs_iomap_folio_state`”两种模式间切换, 而非盲目依赖 Folio 阶数。
- 并发安全: 对 `f2fs_iomap_folio_state` 的操作利用了其内部的自旋锁或原子操作, 保证了并发安全。

F2FS_FOLIO_PRIVATE_SET_FUNC (我的实现)

```

/* f2fs_ifs.h (我的实现) */
#define F2FS_FOLIO_PRIVATE_SET_FUNC(name, flagname) \
static inline int f2fs_set_folio_private_##name(struct folio *folio) \
{ \
    if (folio_order(folio) == 0) { \
        // 0 阶 Folio 的处理逻辑 ...
    } \
    // 高阶 Folio 的处理逻辑

    struct f2fs_iomap_folio_state *fifs = f2fs_ifs_alloc(folio, GFP_NOFS);

    \
    if(unlikely(!fifs)) return -ENOMEM; \
    unsigned long *private_p = f2fs_ifs_private_flags_ptr(fifs, folio); \
    \

    set_bit(PAGE_PRIVATE_##flagname, private_p);

    \
    return 0; \
}

```

辅助函数: `folio_get_f2fs_ifs` 在需要访问 `f2fs_iomap_folio_state` 结构体实例的地方，我们现在可以利用一个专门的辅助函数 `folio_get_f2fs_ifs`。这个函数能够安全地检查 `folio->private` 字段，判断其是否指向一个有效的 `f2fs_iomap_folio_state` 结构体，并返回一个实体指针或 `NULL`。这极大地简化了代码中的判断逻辑，避免了手动进行多重检查，例如早期方案中常见的 `if(folio_order(folio)>0&&fifs)` 这种冗余且不够精确的判断。通过统一的接口，代码变得更加简洁和健壮。

folio_get_f2fs_ifs 实现

```

struct f2fs_iomap_folio_state *folio_get_f2fs_ifs(struct folio *folio)
{
    if (!folio_test_private(folio))
        return NULL;

    if (test_bit(PAGE_PRIVATE_NOT_POINTER, (unsigned long *)&folio->private)
    )
        return NULL;
}

```

```

/* Note we assume folio->private can be either ifs or f2fs_ifs here.
Compressed folios
should not call this function */
f2fs_bug_on(F2FS_F_SB(folio),
    *((u32 *)folio->private) == F2FS_COMPRESSED_PAGE_MAGIC);
return folio->private;
}

```

类型识别: 魔数 (Magic Number) 的引入 由于 `folio->private` 是一个 `void*`, 当我们在代码中拿到这个指针时, 如何区分它指向的是一个标准的 `iomap_folio_state` 还是我们特化的 `f2fs_iomap_folio_state`?

答案是引入一个魔数 (Magic Number)。我们巧妙地滥用了 `read_bytes_pending` 字段。在 `f2fs_ifs_alloc` 中, 一旦我们分配了 `f2fs_iomap_folio_state`, 就会将一个特殊的魔数 `F2FS_IFS_MAGIC` 写入到 `read_bytes_pending` 字段。

利用魔数进行类型检查

```

/* f2fs_ifs.h */
static inline bool is_f2fs_ifs(struct folio *folio)
{
    // ...
    struct f2fs_iomap_folio_state *fifs =
        (struct f2fs_iomap_folio_state *)folio->private;
    // ...
    // 通过检查这个特殊值, 我们就能确定这是一个我们自己管理的结构体
    if (READ_ONCE(fifs->read_bytes_pending) == F2FS_IFS_MAGIC) {
        return true;
    }
    return false;
}

/* f2fs_ifs.c -> f2fs_ifs_alloc */
// 在分配和初始化 fifs 后 ...

```

```
WRITE_ONCE(fifs->read_bytes_pending, F2FS_IFS_MAGIC);
```

这个魔数的设计非常关键, 因为它允许我们在运行时安全地进行类型转换和访问我们扩展的私有字段, 而不会意外地破坏一个由通用 `iomap` 框架分配的标准 `iomap_folio_state`。

无缝迁移: `f2fs_ifs_alloc` 的兼容逻辑 在实际运行中, 一个 `Folio` 的 `private` 字段可能已经被通用 `iomap` 框架设置为一个标准的 `iomap_folio_state`。当 `F2FS` 需要设置自己的私有标志时, `f2fs_ifs_alloc` 必须能够处理这种情况。

f2fs_ifs_alloc 的迁移逻辑

```

/* f2fs_ifs.c */
struct f2fs_iomap_folio_state *f2fs_ifs_alloc(struct folio *folio, gfp_t gfp)
{
    // ...
    old_private = folio_get_private(folio);

    if (old_private) {
        // 检查魔数, 如果已经是我们的类型, 直接返回
        if (READ_ONCE(...) == F2FS_IFS_MAGIC) {
            return (struct f2fs_iomap_folio_state *)old_private;
        } else {
            // 不是我们的类型, 但非空 -> 假定是 iomap_folio_state
            // 1. 分配一个新的、更大的 f2fs_iomap_folio_state
            fifs = kmalloc(alloc_size, gfp);
            // ...

            // 2. 将旧 iomap_folio_state 的内容拷贝到新结构体的对应位置

            ifs_to_f2fs_ifs(old_private, fifs, folio);

            // 3. 设置我们的魔数和私有字段初始值
            WRITE_ONCE(fifs->read_bytes_pending, F2FS_IFS_MAGIC);
            atomic_set(f2fs_ifs_cc_pending_bytes_ptr(fifs, folio), 0);

            // 4. 原子地替换 folio->private 指针, 并释放旧的结构体
            folio_change_private(folio, fifs);
            kfree(old_private);
            return fifs;
        }
    } else {
        // private 为空, 直接分配并初始化新的 f2fs_iomap_folio_state
        // ...
    }
}

```

这个分配-拷贝-替换的无缝迁移机制, 确保了 F2FS 可以在不丢失 `iomap` 已有状态 (如逐块脏位图) 的情况下, 将 `folio->private` 的所有权安全地转移到我们自己的、功能更丰富的 `f2fs_iomap_folio_state` 结构体上。

功能扩展: 感知压缩簇的脏区查找 因为我需要让我们的设计也能纳入针对压缩文件的支持, 因此我对 `iomap` 原先的帮助函数也进行了扩展。例如, `iomap_find_dirty_range` 用于在回写时查找 Folio 内的脏区间。我们通过包装它, 为其增加了感知 F2FS 压缩簇边界的能力。

f2fs_iomap_find_dirty_range 的扩展实现

```

/* f2fs_ifs.c */
unsigned f2fs_iomap_find_dirty_range(struct folio *folio, u64 *range_start,
                                   u64 range_end)
{
    struct inode* inode = folio->mapping->host;

    if (folio_order(folio) == 0)
        return range_end - *range_start; // 0阶folio总是全脏

    if (f2fs_compressed_file(inode)) {
        /* 将查找范围的末尾, 限制在当前压缩簇的边界内 */
        \begin{tabular}
        {
        l}
        range_end = min(range_end, \\\
            (cluster_i_end_idx(inode, *range_start >> PAGE_SHIFT) + 1) \\\
            << PAGE_SHIFT);
        \end{tabular}}
    }

    // 调用 iomap 的原始实现来处理调整后的区间
    return iomap_find_dirty_range(folio, range_start, range_end);
}

```

这个扩展确保了在回写一个横跨多个压缩簇的高阶 Folio 时，我们的处理逻辑总是能以簇为单位进行，避免了跨簇的写操作，维持了 F2FS 压缩机制的完整性。

迁移至上游所做的兼容工作 F2FS 最新的上游 Matthew 先生已经将其为 folio 的 private 字段设计的新的 API 和相关补丁放到了 F2FS 社区。为了不破坏 F2FS 旧有的逻辑并且不和 Matthew 先生的补丁冲突，我使用了 #ifdef 条件编译灵活控制。

上游兼容性宏定义

```

#ifdef CONFIG_F2FS_IOMAP_FOLIO_STATE
#define F2FS_FOLIO_PRIVATE_GET_FUNC(name, flagname)
    static inline bool folio_test_f2fs_##name(const struct folio *folio)
    {
        /* First try direct folio->private access for meta folio */
        if (folio_test_private(folio) &&
            test_bit(PAGE_PRIVATE_NOT_POINTER,
                    (unsigned long *)&folio->private)) {
            return test_bit(PAGE_PRIVATE_##flagname,

```

```

        (unsigned long *)&folio->private);
    }
    /* For higher-order folios, use iomap folio state */
    struct f2fs_iomap_folio_state *fifs =
        (struct f2fs_iomap_folio_state *)folio->private;
    unsigned long *private_p;
    if (unlikely(!fifs || !folio->mapping))
        return false;
    /* Check magic number before accessing private data */
    if (READ_ONCE(fifs->read_bytes_pending) != F2FS_IFS_MAGIC)
        return false;
    private_p = f2fs_ifs_private_flags_ptr(fifs, folio);
    if (!private_p)
        return false;
    /* Test bits directly on the 'private' slot */
    return test_bit(PAGE_PRIVATE_##flagname, private_p);
}
static inline bool page_private_##name(struct page *page)
{
    return PagePrivate(page) &&
        test_bit(PAGE_PRIVATE_NOT_POINTER, &page_private(page)) &&
        test_bit(PAGE_PRIVATE_##flagname, &page_private(page));
}
#define F2FS_FOLIO_PRIVATE_SET_FUNC(name, flagname)
static inline int folio_set_f2fs_##name(struct folio *folio)
{
    /* For higher-order folios, use iomap folio state */
    if (unlikely(!folio->mapping))
        return -ENOENT;
    bool force_alloc=f2fs_should_use_buffered_iomap(folio_inode(
folio));
    if (!force_alloc && !folio_test_private(folio)) {
        folio_attach_private(folio, (void *)0);
        set_bit(PAGE_PRIVATE_NOT_POINTER,
            (unsigned long *)&folio->private);
        set_bit(PAGE_PRIVATE_##flagname,
            (unsigned long *)&folio->private);
        return 0;
    }
    struct f2fs_iomap_folio_state *fifs =
        f2fs_ifs_alloc(folio, GFP_NOFS,true);
    if (unlikely(!fifs))
        return -ENOMEM;
    unsigned long *private_p;
    WRITE_ONCE(fifs->read_bytes_pending, F2FS_IFS_MAGIC);
    private_p = f2fs_ifs_private_flags_ptr(fifs, folio);
    if (!private_p)
        return -EINVAL;

```

```

        /* Set the bit atomically */
        set_bit(PAGE_PRIVATE_##flagname, private_p);
        /* Ensure NOT_POINTER bit is also set if any F2FS flag is set */
        if (PAGE_PRIVATE_##flagname != PAGE_PRIVATE_NOT_POINTER)
            set_bit(PAGE_PRIVATE_NOT_POINTER, private_p);
        return 0;
    }

    static inline void set_page_private_##name(struct page *page)
    {
        if (!PagePrivate(page))
            attach_page_private(page, (void *)0);
        set_bit(PAGE_PRIVATE_NOT_POINTER, &page_private(page));
        set_bit(PAGE_PRIVATE_##flagname, &page_private(page));
    }

#define F2FS_FOLIO_PRIVATE_CLEAR_FUNC(name, flagname)
    static inline void folio_clear_f2fs_##name(
        struct folio *folio)
    {
        /* First try direct folio->private access */
        if (folio_test_private(folio) &&
            test_bit(PAGE_PRIVATE_NOT_POINTER,
                (unsigned long *)&folio->private)) {
            clear_bit(PAGE_PRIVATE_##flagname,
                (unsigned long *)&folio->private);
            folio_detach_private(folio);
            return;
        }
        /* For higher-order folios, use iomap folio state */
        struct f2fs_iomap_folio_state *fifs =
            (struct f2fs_iomap_folio_state *)&folio->private;
        unsigned long *private_p;
        if (unlikely(!fifs || !folio->mapping))
            return;
        /* Check magic number before clearing */
        if (READ_ONCE(fifs->read_bytes_pending) != F2FS_IFS_MAGIC)
            return; /* Not ours or state unclear */
        private_p = f2fs_ifs_private_flags_ptr(fifs, folio);
        if (!private_p)
            return;
        clear_bit(PAGE_PRIVATE_##flagname, private_p);
    }

    static inline void clear_page_private_##name(struct page *page)
    {
        clear_bit(PAGE_PRIVATE_##flagname, &page_private(page));
        if (page_private(page) == BIT(PAGE_PRIVATE_NOT_POINTER))
            detach_page_private(page);
    }

```

```

// Generate the accessor functions using the macros
F2FS_FOLIO_PRIVATE_GET_FUNC(nonpointer, NOT_POINTER);
F2FS_FOLIO_PRIVATE_GET_FUNC(inline, INLINE_INODE);
F2FS_FOLIO_PRIVATE_GET_FUNC(gcing, ONGOING_MIGRATION);
F2FS_FOLIO_PRIVATE_GET_FUNC(atomic, ATOMIC_WRITE);
F2FS_FOLIO_PRIVATE_GET_FUNC(reference, REF_RESOURCE);

F2FS_FOLIO_PRIVATE_SET_FUNC(reference, REF_RESOURCE);
F2FS_FOLIO_PRIVATE_SET_FUNC(inline, INLINE_INODE);
F2FS_FOLIO_PRIVATE_SET_FUNC(gcing, ONGOING_MIGRATION);
F2FS_FOLIO_PRIVATE_SET_FUNC(atomic, ATOMIC_WRITE);

F2FS_FOLIO_PRIVATE_CLEAR_FUNC(reference, REF_RESOURCE);
F2FS_FOLIO_PRIVATE_CLEAR_FUNC(inline, INLINE_INODE);
F2FS_FOLIO_PRIVATE_CLEAR_FUNC(gcing, ONGOING_MIGRATION);
F2FS_FOLIO_PRIVATE_CLEAR_FUNC(atomic, ATOMIC_WRITE);
static inline int folio_set_f2fs_data(struct folio *folio, unsigned long
data)
{
    if (unlikely(!folio->mapping))
        return -ENOENT;

    struct f2fs_iomap_folio_state *fifs = f2fs_ifs_alloc(folio, GFP_NOFS
, true);
    if (unlikely(!fifs))
        return -ENOMEM;

    unsigned long *private_p;

    private_p = f2fs_ifs_private_flags_ptr(fifs, folio);
    if (!private_p)
        return -EINVAL;

    *private_p &= GENMASK(PAGE_PRIVATE_MAX - 1, 0);
    *private_p |= (data << PAGE_PRIVATE_MAX);
    set_bit(PAGE_PRIVATE_NOT_POINTER, private_p);

    return 0;
}
static inline unsigned long folio_get_f2fs_data(struct folio *folio)
{
    struct f2fs_iomap_folio_state *fifs =
        (struct f2fs_iomap_folio_state *)folio->private;
    unsigned long *private_p;
    unsigned long data_val;
    if (!folio->mapping)
        return 0;
    f2fs_bug_on(F2FS_I_SB(folio_inode(folio)), !fifs);

```



```

        if (READ_ONCE(fifs->read_bytes_pending) ≠ F2FS_IFS_MAGIC)
            return 0;

        private_p = f2fs_ifs_private_flags_ptr(fifs, folio);
        if (!private_p)
            return 0;

        data_val = READ_ONCE(*private_p);

        if (!test_bit(PAGE_PRIVATE_NOT_POINTER, &data_val))
            return 0;

        return data_val >> PAGE_PRIVATE_MAX;
    }
#else
#define PAGE_PRIVATE_GET_FUNC(name, flagname)
static inline bool folio_test_f2fs_##name(const struct folio *folio)
{
    unsigned long priv = (unsigned long)folio->private;
    unsigned long v = (1UL << PAGE_PRIVATE_NOT_POINTER) |
        (1UL << PAGE_PRIVATE_##flagname);
    return (priv & v) == v;
}
static inline bool page_private_##name(struct page *page)
{
    return PagePrivate(page) &&
        test_bit(PAGE_PRIVATE_NOT_POINTER, &page_private(page)) &&
        test_bit(PAGE_PRIVATE_##flagname, &page_private(page));
}

#define PAGE_PRIVATE_SET_FUNC(name, flagname)
static inline void folio_set_f2fs_##name(struct folio *folio)
{
    unsigned long v = (1UL << PAGE_PRIVATE_NOT_POINTER) |
        (1UL << PAGE_PRIVATE_##flagname);
    if (!folio->private)
        folio_attach_private(folio, (void *)v);
    else {
        v |= (unsigned long)folio->private;
        folio->private = (void *)v;
    }
}
static inline void set_page_private_##name(struct page *page)
{
    if (!PagePrivate(page))
        attach_page_private(page, (void *)0);
    set_bit(PAGE_PRIVATE_NOT_POINTER, &page_private(page));
    set_bit(PAGE_PRIVATE_##flagname, &page_private(page));
}

```

```

}

#define PAGE_PRIVATE_CLEAR_FUNC(name, flagname)
static inline void folio_clear_f2fs_##name(struct folio *folio)
{
    unsigned long v = (unsigned long)folio->private;

    v &= ~(1UL << PAGE_PRIVATE_##flagname);
    if (v == (1UL << PAGE_PRIVATE_NOT_POINTER))
        folio_detach_private(folio);
    else
        folio->private = (void *)v;
}
static inline void clear_page_private_##name(struct page *page)
{
    clear_bit(PAGE_PRIVATE_##flagname, &page_private(page));
    if (page_private(page) == BIT(PAGE_PRIVATE_NOT_POINTER))
        detach_page_private(page);
}

PAGE_PRIVATE_GET_FUNC(nonpointer, NOT_POINTER);
PAGE_PRIVATE_GET_FUNC(inline, INLINE_INODE);
PAGE_PRIVATE_GET_FUNC(gcing, ONGOING_MIGRATION);
PAGE_PRIVATE_GET_FUNC(atomic, ATOMIC_WRITE);

PAGE_PRIVATE_SET_FUNC(reference, REF_RESOURCE);
PAGE_PRIVATE_SET_FUNC(inline, INLINE_INODE);
PAGE_PRIVATE_SET_FUNC(gcing, ONGOING_MIGRATION);
PAGE_PRIVATE_SET_FUNC(atomic, ATOMIC_WRITE);

PAGE_PRIVATE_CLEAR_FUNC(reference, REF_RESOURCE);
PAGE_PRIVATE_CLEAR_FUNC(inline, INLINE_INODE);
PAGE_PRIVATE_CLEAR_FUNC(gcing, ONGOING_MIGRATION);
PAGE_PRIVATE_CLEAR_FUNC(atomic, ATOMIC_WRITE);

static inline unsigned long folio_get_f2fs_data(struct folio *folio)
{
    unsigned long data = (unsigned long)folio->private;

    if (!test_bit(PAGE_PRIVATE_NOT_POINTER, &data))
        return 0;
    return data >> PAGE_PRIVATE_MAX;
}

static inline void folio_set_f2fs_data(struct folio *folio, unsigned
long data)
{

```

```
data = (1UL << PAGE_PRIVATE_NOT_POINTER) | (data << PAGE_PRIVATE_MAX);

if (!folio_test_private(folio))
    folio_attach_private(folio, (void *)data);
else
    folio->private = (void *)((unsigned long)folio->private | data);
}
#endif
```

3.4 压缩文件 buffered read

3.4.1 功能背景

F2FS 作为专为闪存设备设计的文件系统, 其压缩功能对于节省存储空间至关重要。然而, F2FS 压缩文件的读取流程与普通文件截然不同。当内核需要读取一个被压缩的簇时, 它不能直接将磁盘上的数据读入目标 Folio, 而必须执行一个多阶段的“间接读取”流程:

1. **读取到临时缓冲区**: 先将压缩数据块读入由 `compress_page_pool` 分配的缓冲页中。
2. **解压**: CPU 调用解压算法 (如 `lzo`, `lz4`, `zstd` 等), 将数据解压到目标 Folio 中。

这个流程意味着, 通用的 `iomap_readahead` 假设的“磁盘到 Folio”的直接数据通路在此失效。更复杂的是, F2FS 文件支持部分压缩, 即一个文件内可能同时包含压缩数据和非压缩数据。当引入 Large Folios 机制后, 一个大的 Folio 完全可能跨越一个或多个压缩簇以及非压缩数据块, 这就对 I/O 的管理和并发控制提出了严峻的挑战。

3.4.2 重大问题分析

在为 F2FS 的压缩读路径引入 Large Folios 支持时, 核心挑战源于 **I/O 模型的混合性与状态管理的复杂性**^{*}。一个 Large Folio 的不同部分可能需要通过完全不同的路径来填充数据, 导致了以下关键问题:

- **混合 I/O 模型**: Folio 的一部分可能通过标准的直接 I/O 路径完成 (非压缩部分), 而另一部分则需要走“读取到临时页 -> 解压”的间接路径 (压缩部分)。
- **并发与状态不一致**: 这两条路径的 I/O 完成时机完全不同。如果不能统一管理整个 Folio 的完成状态, 就可能出现一个线程已经完成了非压缩部分的读取, 而另一个解压任务仍在进行中。此时若错误地解锁 Folio, 就会导致数据损坏。
- **生命周期追踪**: 必须有一个精确的机制来追踪一个 Large Folio 上所有悬而未决 (pending) 的 I/O 请求。只有当所有相关的 I/O 操作 (无论是直接读还是解压) 都成功完成后, 才能将整个 Folio 标记为 `Uptodate` 并解锁。

为解决此问题, 我们引入了 `read_bytes_pending` 计数器机制, 对一个 Large Folio 发起的所有 I/O 请求进行统一追踪, 确保其生命周期管理的正确性。

3.4.3 方案设计

我们的核心思路是保留 `iomap_iter` 的外层双循环框架, 但将内层的 `iomap_readpage_iter` 替换为我们特化的、能够识别并处理压缩簇的 `f2fs_compress_readpage_iter`。

`iomap` 迭代框架的入口: `f2fs_buffered_read_compress_iomap_begin` 在 F2FS 压缩文件的读取过程中, 我们对 `iomap` 迭代框架的入口函数 `f2fs_buffered_read_compress_iomap_begin` 进行了关键的重写。这个函数是整个读取流程的起点, 它负责初始化 `iomap` 结构体, 并根据当前读取位置是否处于压缩簇来决定后续的处理逻辑。

f2fs_buffered_read_compress_iomap_begin 实现

```
static int f2fs_buffered_read_compress_iomap_begin(struct inode *inode,
                                                  loff_t pos, loff_t length,
                                                  unsigned flags,
                                                  struct iomap *iomap,
                                                  struct iomap *srcmap)
{
    struct f2fs_sb_info *sbi = F2FS_I_SB(inode);
    pgoff_t index = pos >> PAGE_SHIFT;
    loff_t map_len_bytes;
    int ret = 0;
    iomap->bdev = sbi->sb->s_bdev;
    iomap->private = NULL;
    iomap->flags = 0;
    iomap->offset = F2FS_BLK_TO_BYTES(F2FS_BYTES_TO_BLK(pos));
#ifdef CONFIG_F2FS_FS_COMPRESSION
    struct compress_ctx cc = {
        .inode = inode,
        .log_cluster_size = F2FS_I(inode)->i_log_cluster_size,
        .cluster_size = F2FS_I(inode)->i_cluster_size,
        .cluster_idx = NULL_CLUSTER,
        .rpages = NULL,
        .cpages = NULL,
        .nr_rpages = 0,
        .nr_cpages = 0,
    };
    /* For compress cluster, just fill iomap length with a cluster size
    then all block mapping logic left for f2fs_write_multi_folio
    to handle*/
    if (f2fs_compressed_file(inode)) {
        bool is_compressed = f2fs_is_compressed_cluster(inode, index);
        if (is_compressed) {
            pgoff_t cluster_start_idx = cluster_idx(&cc, index);
            map_len_bytes = min_t(loff_t, length,
                                  ((loff_t)(cluster_start_idx + 1) << cc.
                                   log_cluster_size) - index) << PAGE_SHIFT;
            iomap->length = map_len_bytes;
            iomap->type = IOMAP_MAPPED;
        }
    }
#endif
}
```

```

        return 0;
    }
}
#endif
return f2fs_buffered_read_iomap_ops.iomap_begin(inode, pos, length,
                                                flags, iomap, srcmap);
}

```

逻辑阐述: 该函数的核心在于对压缩文件的特殊处理:

- **初始化 iomap 结构:** 函数首先初始化 iomap 结构体的基本字段, 如设备、私有数据、标志和偏移量。
- **压缩簇长度限制:** 对于压缩文件, 如果当前位置 pos 位于一个压缩簇内, 函数会计算出当前簇的结束位置, 并将 iomap->length 设置为从 pos 到当前簇结束的字节数 (不超过原始请求的 length)。这意味着, 每次 iomap_iter 循环时, iomap_begin 返回的 iomap 结构体所描述的映射长度将精确地限制在当前压缩簇的边界内。
- **类型标记:** 将 iomap->type 设置为 IOMAP_MAPPED, 表示这是一个已映射的区域。
- **非压缩文件回退:** 如果文件未压缩, 或者当前位置不在压缩簇内, 则回退到通用的 f2fs_buffered_read_iomap_ops.iomap_begin 函数进行处理。

通过这种方式, 我们确保了 iomap 迭代框架在处理压缩文件时, 能够以簇为单位进行迭代, 从而将后续复杂的块映射和 I/O 提交逻辑留给 f2fs_read_multi_folios 等函数来统一处理, 简化了整体流程。

核心思想与状态扩展 为了聚合针对同一个压缩簇的 I/O 请求, 我们使用了 compress_ctx 结构体。当进行预读 (readahead) 时, 我们会遍历多个 Folio, 并将属于同一个压缩簇的 page 添加到 compress_ctx 中, 最后统一提交 I/O。

在此基础上, 我们增加了一个新的状态标志: cur_folio_in_compress_ctx。在预读循环中, 一个 Folio 的部分页面被添加到 compress_ctx 后, I/O 可能尚未提交。此时, 若没有此标志, 当预读逻辑移至下一个 Folio 时, 上一个 Folio 可能会被错误地解锁 (因为它暂时未被加入 bio), 从而引发竞态条件。

新增状态 cur_folio_in_compress_ctx

- **作用:** 明确标识一个 Folio 的页面已被暂存到 compress_ctx 中, 即使 I/O 尚未提交, 该 Folio 也处于“待处理”状态。
- **目的:** 防止在预读循环中, 一个已被部分处理但 I/O 未提交的 Folio 被提前解锁, 确保了状态的正确性和一致性。

数据读取与 pending 计数增加 当确定需要为压缩文件读取数据时, f2fs_do_read_multi_folios 函数成为处理入口。它负责初始化 compress_ctx、设置 cur_folio_in_compress_ctx 状态, 并调用核心处理函数 do_read_multi_folios。

压缩文件多 Folio 读取入口

```

int f2fs_do_read_multi_folios(struct f2fs_readpage_ctx* ctx, loff_t pos,
                             loff_t plen)
{
    struct folio *folio = ctx->cur_folio;
    struct compress_ctx*cc=&ctx->cc;
    struct bio** bio_ret = &ctx->bio;
    pgoff_t cur_idx = pos >> PAGE_SHIFT;
    struct readahead_control *rac = ctx->rac;
    int ret = 0;
    pgoff_t cluster_idx = cluster_idx(cc,cur_idx);
    if (cc->cluster_idx == NULL_CLUSTER)
    {
        ret = f2fs_init_compress_ctx(cc);
        if (ret < 0)
            return ret;
    }
    ctx->cur_folio_in_compress_ctx = true;
    ret =do_read_multi_folios(cc, folio, pos, plen,bio_ret,rac,false);
    return ret;
}

```

核心逻辑在 `do_read_multi_folios` 中实现。该函数首先调用 `f2fs_compress_ctx_add_folio` 将 Folio 的相关页面添加到 `compress_ctx`。紧接着, 它会为本次 I/O 请求增加 `read_bytes_pending` 计数。

执行多 Folio 读取与 pending 计数增加

```

int do_read_multi_folios(struct compress_ctx*cc, struct folio *folio, loff_t
    pos,
                        loff_t plen, struct bio** bio_ret,
                        struct readahead_control *rac,bool for_write)
{
    int ret = 0;
    f2fs_compress_ctx_add_folio(cc, folio, pos, plen);
    struct f2fs_iomap_folio_state* fifs = folio->private;
    if(folio_order(folio)>0&&fifs)
    {
        spin_lock_irq(&fifs->state_lock);

        fifs->read_bytes_pending += plen;

        spin_unlock_irq(&fifs->state_lock);
    }

    if (f2fs_cluster_is_full(cc) || (rac&&rac->_nr_pages == 0)) {

```

```

        ret = f2fs_read_multi_folios(cc, bio_ret, rac, for_write);
        f2fs_destroy_compress_ctx(cc, false);
    }
    return ret;
}

```

Folio 添加逻辑 (正确实现) `f2fs_compress_ctx_add_folio` 函数的正确实现至关重要。它负责精确地将一个 Folio 的一个或多个 page 添加到压缩上下文中。这与旧的、一次只能添加单个 page 的 `f2fs_compress_ctx_add_page` 形成对比。

向 compress_ctx 添加 Folio (正确版本)

```

void f2fs_compress_ctx_add_folio(struct compress_ctx *cc, struct folio *
    folio,
                                loff_t pos, loff_t rlen)
{
    unsigned int cluster_ofs;
    loff_t poff = offset_in_folio(folio, pos);
    unsigned int idx_in_folio = poff >> PAGE_SHIFT;
    if (!f2fs_cluster_can_merge_page(cc, folio->index+idx_in_folio))
        f2fs_bug_on(F2FS_I_SB(cc->inode), 1);
    cluster_ofs = offset_in_cluster(cc, folio->index+idx_in_folio);
    int num_pages_to_add =
        min_t(unsigned int, (unsigned int)folio_nr_pages(folio) -
            idx_in_folio,
              cc->cluster_size - cc->nr_rpages);
    num_pages_to_add = min_t(unsigned int, num_pages_to_add, rlen >>
        PAGE_SHIFT);
    for (int i = 0; i < num_pages_to_add; ++i) {
        cc->rpages[cluster_ofs] = folio_page(folio, idx_in_folio + i);
        cc->nr_rpages++;
        cluster_ofs++;
    }
    cc->cluster_idx = cluster_idx(cc, folio->index+idx_in_folio);
}

```

逻辑阐述: 该函数精确计算出本次可以添加的页面数量 `num_pages_to_add`，它取 Folio 内剩余页数、压缩上下文剩余容量、本次请求长度三者的最小值，然后通过一个循环将这些页面逐一添加到 `cc->rpages` 数组中。

IO 完成与 pending 计数减少 当磁盘 I/O 完成后，内核会调用注册的回调函数 `f2fs_read_end_io`。这个函数是所有读 I/O 完成路径的起点，它会根据 I/O 的类型（是否需要解压等）将任务分发到不同的处理函数。

常规页与压缩页的 I/O 完成路径

常规页完成路径: f2fs_finish_read_bio

```
static void f2fs_finish_read_bio(struct bio *bio, bool in_task)
{
    struct folio_iter fi;
    /* ... */
    bio_for_each_folio_all(fi, bio) {
        struct folio *folio = fi.folio;          if (f2fs_is_compressed_folio(
        folio)) {
            /* ... */
            continue;
        }
        /* ... */
        int error=!(bio->bi_status == 0);
        f2fs_iomap_finish_folio_read(folio,fi.offset, fi.length,error);
    }
    /* ... */
}
```

压缩页完成路径: f2fs_decompress_end_io

```
void f2fs_decompress_end_io(struct decompress_io_ctx *dic, bool failed, bool
    in_task)
{
    /* ... */
    int num_to_skip=0;
    for (int i = 0; i < dic->cluster_size; i+=num_to_skip) {
        struct folio *folio;
        num_to_skip=1;
        if (!dic->rpages[i])
            continue;
        folio = page_folio(dic->rpages[i]);
        while ((i + num_to_skip) < dic->cluster_size && dic->rpages[i +
        num_to_skip] &&
            page_folio(dic->rpages[i + num_to_skip]) == folio) {
            num_to_skip++;
        }
        /* ... */
        if(!failed)
        {
            f2fs_iomap_finish_folio_read(folio,0,num_to_skip<<PAGE_SHIFT,0);
        }
    }
    /* ... */
}
```

逻辑阐述: 可以看到, 无论是常规页的完成路径, 还是压缩页解压后的完成路径, 最终都调用了

我们抽象出的新函数 `f2fs_iomap_finish_folio_read`。特别是在 `f2fs_decompress_end_io` 中, 我们通过一个巧妙的循环, 一次性处理完连续属于同一个 Large Folio 的所有页面, 并将它们合并为单次调用。

统一的 Folio 完成处理与 pending 计数减少 这是我们方案的闭环部分。所有读 I/O 完成路径最终都会汇集到 `f2fs_iomap_finish_folio_read` 函数。此函数负责减少 `read_bytes_pending` 计数, 并在计数归零时, 最终完成 Folio 的解锁。

统一的 Folio 完成与 pending 计数减少

```
void f2fs_iomap_finish_folio_read(struct folio *folio, size_t off, size_t len, int error)
{
    struct f2fs_iomap_folio_state *fifs = folio->private;
    bool uptodate = !error;
    bool finished = true;
    if(folio_order(folio) > 0 && fifs)
    {
        unsigned long flags;
        spin_lock_irqsave(&fifs->state_lock, flags);

        fifs->read_bytes_pending -= len;

        if(!error)
        {
            uptodate = ifs_set_range_uptodate(folio, (struct iomap_folio_state *)fifs, off, len);
        }
        finished = (fifs->read_bytes_pending == 0 || fifs->read_bytes_pending == F2FS_IFS_MAGIC);
        spin_unlock_irqrestore(&fifs->state_lock, flags);
        if(finished)
        {
            folio_end_read(folio, uptodate);
        }
    }
    else
    {
        folio_end_read(folio, !error);
    }
}
```

逻辑阐述: 通过 `read_bytes_pending` 的统一“增加-减少”模型, 普通块 I/O 与解压 I/O 在同一个计数器下收敛。只有当计数器归零时, 才代表这个 Large Folio 的所有部分都已就绪, 此时才调用 `folio_end_read` 将其解锁, 实现了真正的“混合路径一致性”, 完美解决了并发控制问题。

核心迭代器对比：`f2fs_compress_readpage_iter` 与 `iomap_readpage_iter` 在 F2FS 压缩文件读取的 `iomap` 迭代框架中，`f2fs_compress_readpage_iter` 是我们针对压缩逻辑进行定制的核心函数。它取代了通用 `iomap` 框架中的 `iomap_readpage_iter`。尽管两者都负责处理单个 Folio 的读取，但它们在处理压缩数据和 I/O 提交方式上存在显著差异。

title=f2fs_compress_readpage_iter 与 iomap_readpage_iter 对比

F2FS 压缩文件迭代器: `f2fs_compress_readpage_iter`

```
static int f2fs_compress_readpage_iter(struct iomap_iter *iter, struct
f2fs_readpage_ctx *ctx)
{
    loff_t pos = iter->pos;
    loff_t length = iomap_length(iter);
    struct folio *folio = ctx->cur_folio;
    struct inode* inode=folio->mapping->host;
    struct f2fs_iomap_folio_state *fifs;
    size_t poff, plen;
    int ret;
#ifdef CONFIG_F2FS_FS_COMPRESSION
    bool is_compressed =
        f2fs_is_compressed_cluster(inode, pos >> PAGE_SHIFT);
#endif
    /* zero post-eof blocks as the page may be mapped */
    fifs = f2fs_ifs_alloc(folio, iter->flags, false);
    iomap_adjust_read_range(iter->inode, folio, &pos, length, &poff, &plen);
    if (plen == 0)
        goto done;
    if (!is_compressed)
    {
        if (iomap_block_needs_zeroing(iter, pos)) {
            folio_zero_range(folio, poff, plen);
            iomap_set_range_uptodate(folio, poff, plen);
            goto done;
        }
    }
    if (is_compressed) {
        ret = f2fs_do_read_multi_folios(ctx, pos, plen);
    } else {
        ret = f2fs_do_read_single_folio_iomap(iter, ctx, pos, plen, poff);
    }
    if (ret)
        return ret;
done:
    length = pos - iter->pos + plen;
    return iomap_iter_advance(iter, &length);
}
```

特点:

- 压缩感知: 核心逻辑是根据 `f2fs_is_compressed_cluster` 判断当前 Folio 区域是否属于压缩数据。
- 分发到专门函数: 如果是压缩数据, 则调用 `f2fs_do_read_multi_folios` 来处理, 该函数会聚合多个页面的 I/O 请求。如果是非压缩数据, 则调用 `f2fs_do_read_single_folio_iomap`。
- 延迟 I/O 提交: 对于压缩数据, I/O 不会立即提交。页会被添加到 `compress_ctx` 中, 直到整个压缩簇满或预读结束才统一提交。
- 职责分离: 自身不直接进行 bio 的分配和提交, 而是将这些操作委托给更底层的 `f2fs_do_read_multi_folios` 或 `f2fs_do_read_single_folio_iomap`。

通用 I/O 迭代器: `iomap_readpage_iter`

```
static int iomap_readpage_iter(struct iomap_iter *iter,
                              struct iomap_readpage_ctx *ctx)
{
    const struct iomap *iomap = &iter->iomap;
    loff_t pos = iter->pos;
    loff_t length = iomap_length(iter);
    struct folio *folio = ctx->cur_folio;
    struct iomap_folio_state *ifs;
    size_t poff, plen;
    sector_t sector;
    int ret;

    if (iomap->type == IOMAP_INLINE) {
        ret = iomap_read_inline_data(iter, folio);
        if (ret)
            return ret;
        return iomap_iter_advance(iter, &length);
    }

    /* zero post-eof blocks as the page may be mapped */
    ifs = ifs_alloc(iter->inode, folio, iter->flags);
    iomap_adjust_read_range(iter->inode, folio, &pos, length, &poff, &plen);
    if (plen == 0)
        goto done;

    if (iomap_block_needs_zeroing(iter, pos)) {
        folio_zero_range(folio, poff, plen);
        iomap_set_range_uptodate(folio, poff, plen);
        goto done;
    }

    ctx->cur_folio_in_bio = true;
```

```

    if (ifs) {
        spin_lock_irq(&ifs->state_lock);
        ifs->read_bytes_pending += plen;
        spin_unlock_irq(&ifs->state_lock);
    }

    sector = iomap_sector(iomap, pos);
    if (!ctx->bio ||
        bio_end_sector(ctx->bio) != sector ||
        !bio_add_folio(ctx->bio, folio, plen, poff)) {
        gfp_t gfp = mapping_gfp_constraint(folio->mapping, GFP_KERNEL);
        gfp_t orig_gfp = gfp;
        unsigned int nr_vecs = DIV_ROUND_UP(length, PAGE_SIZE);

        if (ctx->bio)
            submit_bio(ctx->bio);

        if (ctx->rac) /* same as readahead_gfp_mask */
            gfp |= __GFP_NORETRY | __GFP_NOWARN;
        ctx->bio = bio_alloc(iomap->bdev, bio_max_segs(nr_vecs),
                            REQ_OP_READ, gfp);

        /*
         * If the bio_alloc fails, try it again for a single page to
         * avoid having to deal with partial page reads. This emulates
         * what do_mpage_read_folio does.
         */
        if (!ctx->bio) {
            ctx->bio = bio_alloc(iomap->bdev, 1, REQ_OP_READ,
                                orig_gfp);
        }
        if (ctx->rac)
            ctx->bio->bi_opf |= REQ_RAHEAD;
        ctx->bio->bi_iter.bi_sector = sector;
        ctx->bio->bi_end_io = iomap_read_end_io;
        bio_add_folio_nofail(ctx->bio, folio, plen, poff);
    }

done:
    /*
     * Move the caller beyond our range so that it keeps making progress.
     * For that, we have to include any leading non-up-to-date ranges, but
     * we can skip trailing ones as they will be handled in the next
     * iteration.
     */
    length = pos - iter->pos + plen;
    return iomap_iter_advance(iter, &length);
}

```

特点:

- **通用性**: 不区分文件是否压缩, 直接处理 I/O 映射。
- **直接 I/O 提交**: 在函数内部直接分配和提交 bio 请求。如果当前的 bio 不连续或无法添加 Folio, 它会立即提交当前的 bio 并分配一个新的。
- **Inline 数据处理**: 包含对 IOMAP_INLINE 类型的特殊处理, 这些数据直接从 inode 中读取。
- **零填充**: 处理文件末尾或空洞区域的零填充逻辑。

总结差异: 最主要的区别在于 I/O 提交的策略。`iomap_readpage_iter` 倾向于立即或尽快提交 bio, 以确保通用文件系统的效率。而 `f2fs_compress_readpage_iter` 则采取了更复杂的策略, 它根据数据是否压缩来分发任务, 并且对于压缩数据, 它会延迟 I/O 提交, 直到一个完整的压缩簇数据准备就绪, 从而优化了 F2FS 压缩读的性能和正确性。这种分离使得 F2FS 能够更好地管理其特有的压缩 I/O 流程, 同时保持与 `iomap` 框架的兼容性。

3.5 压缩文件 buffered write

3.5.1 功能背景

我们已经决定将整个 bufferedwrite 路径迁移往 iomap 框架, 因为经过前面的分析我们知道, 这是最无历史包袱、能够原生支持 largefolios 的方式。

与 bufferedread 和 pagewriteback 不同的一点是, `iomap_buffered_write` 除了提供通用的 `iomap_begin` 和 `iomap_end` 接口外, 还允许文件系统通过 ``folio_ops`` 回调函数集, 提供自己定制化的 folio 获取逻辑 (`__iomap_get_folio`) 和释放逻辑 (`__iomap_put_folio`)。通过分析, 我发现利用 `iomap_buffered_write` 框架来实现 F2FS 压缩文件的缓冲写是完全可能的, 并且其函数调用栈可以做到比原有实现更少的开销。

3.5.2 重大问题分析

初赛方案的局限性

在初赛时, 我们的设计方案是将我们自己改造成支持 largefolios 的 `prepare_write_begin` 逻辑 (即原先 F2FS 的 bufferedwrite 逻辑) 直接拿过来填入到 `iomap_begin` 之中。然后, 在 `__iomap_get_folio` 中, 我们和原代码逻辑一样, 从压缩文件的上下文 (`compresscontext`) 中根据文件内偏移量取出对应的 page, 并将其转化为 folio 结构体。

这种方法的根本问题在于, 我们的操作视角被局限在了单个“压缩簇” (`compressioncluster`) 内。这意味着, 即使上层 VFS 希望写入一个横跨多个簇的大块数据, 我们的 `iomap_begin` 和 `__iomap_get_folio` 也只能以簇为单位、一次只处理一个簇大小的 folio。

这种限制的根源在于 VFS/MM 层的核心 folio 分配策略。当 `iomap_get_folio` 调用底层的 `__filemap_get_folio` 时, 虽然它会尝试分配一个较大阶数 (order) 的 folio, 但一旦失败, 它会立刻降低阶数并重试。

VFS/MM 核心分配逻辑 `__filemap_get_folio`

```

/* mm/filemap.c */
// ...
// 如果页缓存中没有找到 folio, 并且调用者希望创建
if (!folio && (fgp_flags & FGP_CREAT)) {
    // ...
    // 获取建议的分配阶数 order
    unsigned int order = max(min_order, FGF_GET_ORDER(fgp_flags));
    // ...

    do {
        // ...
        folio = filemap_alloc_folio(alloc_gfp, order);
        if (!folio)
            continue; // 分配失败, 继续循环, 阶数将减小
        // ...
        err = filemap_add_folio(mapping, folio, index, gfp);
        if (!err)
            break; // 成功, 跳出循环
        // ...
    } while (order-- > min_order); // <--- 关键: 如果失败, 则降低阶数 (order--)
    并重试

    // ...
}

```

这个逐级递减的分配策略意味着, 我们初赛的方案几乎无法真正利用到 `largefolios` 带来的性能优势。具体来说, 如果上层应用尝试写入一个 64KB 的数据块, 该数据块理论上可以由一个 `order-4` (16 个页面) 的 `largefolio` 表示。但在初赛方案中, 如果这个 64KB 跨越了两个 32KB 的压缩簇, 我们的实现会将其拆分成两次独立的、针对小 `folio` 的操作, 从而丧失了 `largefolio` 的聚合优势。我们大大地限制了单次 I/O 操作能分配的 `folio` 的尺寸上限, 无法将一次大的写操作聚合到一个大的 `folio` 中处理, 从而违背了支持 `largefolios` 的初衷。

3.5.3 方案设计

为了克服初赛方案的瓶颈, 我们设计了一套全新的算法。其核心思想是 **** 切换视角 ****: 我们不再是以“压缩簇”的视角出发, 一次次地用小 `folio` 去填充一个簇; 而是 **** 从一个“大 Folio”的视角出发, 一次性分配一个足够大的 Folio, 然后用一个或多个压缩簇的数据去填充它 ****。例如, 对于一个 128KB 的写入请求, 新算法会先尝试分配一个完整的 128KB `largefolio`, 然后并行地从磁盘读取覆盖这 128KB 范围所需的多个 (可能是 4 个 32KB 的) 压缩簇数据, 一次性填满这个大 `folio`。这与旧算法一次只处理一个簇的模式形成了鲜明对比。这是我们的原创核心算法。

决赛方案核心代码实现

iomap_begin: 准备大 Folio

```
/* f2fs_iomap.c */
static int f2fs_buffered_write_iomap_begin( ... )
{
    // ...
    // 1. 计算对齐后的总长度
    loff_t aligned_len = (end_idx - start_idx + 1) << PAGE_SHIFT;

    // 2. 一次性获取一个足够大的、锁定的 Folio
```

```
    struct folio* folio = iomap_get_Folio
(iter, start_idx << PAGE_SHIFT, aligned_len);
```

```
    // 3. 对 folio 的 read_bytes_pending 加一个偏置(bias)
    if (folio_order(folio) > 0 && fifs) {
        spin_lock_irqsave(&fifs->state_lock, flags);
```

```
        fifs->read_bytes_pending += 1;
```

```
        spin_unlock_irqrestore(&fifs->state_lock, flags);
    }
```

```
    // 4. 异步提交 BIO 读取数据填充 Folio
    for ( ... ) {
        // ...
        do_read_multi_folios(&cc, folio, ... );
```

```
        iomap->length += add_len;
```

```
    }
    f2fs_submit_read_bio(sbi, bio, DATA);
```

```
    // 5. 同步等待 Folio 完全 up-to-date
    if (folio_order(folio) > 0 && fifs) {
        for(;;) {
            // ... 轮询检查 fifs->read_bytes_pending
            if (done) break;
        }
        // 移除偏置
```

```
        fifs->read_bytes_pending -= 1;
```

```
    }
```

```
// 6. 将准备好的大 Folio 传给 iomap 核心
```

```
iomap->private = folio;
```

```
iomap->type = IOMAP_MAPPED;  
return 0;  
}
```

__iomap_get_folio: 传递大 Folio

```
/* f2fs_iomap.c */  
struct folio* f2fs_iomap_get_folio(  
    struct iomap_iter *iter, loff_t pos, unsigned len)  
{  
    struct inode *inode = iter->inode;  
  
    /*  
     * 初赛时的写法:  
     * struct page**rpages = iter->iomap.private;  
     * return page_folio(rpages[offset_in_cluster( ... )]);  
     */  
  
    // 决赛的写法:  
    // 如果 iomap_begin 已经准备好了大 Folio,  
    // 直接返回它。  
    if(iter->iomap.private && f2fs_compressed_file(inode))  
    {  
  
        return (struct folio*)iter->iomap.private;  
  
    }  
    else // 否则, 走标准流程  
        return iomap_get_folio(iter, pos, len);  
}
```

这个新算法的实现流程和关键设计点如下:

1. 准备阶段 (iomap_begin):

- 分配大 Folio: 我们首先根据用户请求的写入位置和长度, 计算出需要覆盖的所有压缩簇的起止范围, 并以此为据, 调用 `iomap_get_folio` 一次性分配一个能够容纳所有数据的、已锁定的 `largefolio`。

- **填充 Folio:** 我们启动 F2FS 异步的 I/O 逻辑 (`do_read_multi_folios`), 提交一个或多个 BIO, 从磁盘读取数据来填充这个 `largefolio`。在循环中, 我们根据实际读取的簇长度, 累加更新 `iomap->length`, 确保 `iomap` 框架能正确认知本次操作的范围。
 - **传递 Folio:** 当 Folio 被完全填充并处于 `up-to-date` 状态后, 我们并不立即解锁它, 而是将其指针保存在 `iomap->private` 字段中, 传递给 `iomap` 的核心处理逻辑。
2. **同步等待机制:** F2FS 内提交 I/O 的代码全部是异步的。但 `iomap_buffered_write` 必须等到整个 `folio` 的内容都从磁盘读上来 (即完全 `up-to-date`) 后, 才能对其进行写入, 否则会发生严重的数据不一致问题。内核虽提供了 `submit_bio_wait`, 但它只能等待单个 BIO, 无法满足等待一个可能横跨多个 BIO 的 `largefolio` 的需求。
- **巧用 `read_bytes_pending`:** 我们利用了 `f2fs_iomap_folio_state` 中的 `read_bytes_pending` 字段。在提交 BIO 前, 我们先给它加上一个偏置 (`bias`)。这个偏置会阻止 I/O 完成回调函数过早地唤醒其他等待者。
 - **显式轮询:** 在 `iomap_begin` 中, 我们显式地循环等待, 直到 `read_bytes_pending` 的值变为 `F2FS_IFS_MAGIC+1` (我们的魔数加上偏置), 这标志着所有为此 `folio` 提交的 BIO 都已完成。此时, `folio` 必然是完全 `up-to-date` 的。
 - 这种“加偏置-轮询-去偏置”的模式, 为我们实现了一个高效、可靠的、`folio` 粒度的同步等待机制。
3. **簇对齐与一致性:** 为了避免一个压缩簇中只有部分 `folio` 处于 `up-to-date` 的尴尬情况 (特别是当 `folio` 本身大小不足以覆盖整个簇时), 我们的代码还会显式地将 `folio` 最后一个页面所在的整个簇全部读入, 确保了数据的一致性。
4. **执行阶段 (`__iomap_get_folio`):** 我们的 `f2fs_iomap_get_folio` 实现被大大简化。当 `iomap` 核心在处理数据时调用它来获取 `folio` 时, 它只需检查 `iter->iomap.private` 是否已经被设置。如果是, 就直接返回我们在 `begin` 阶段精心准备好的那个 `largefolio`。

通过这一系列设计, 我们彻底解决了初赛方案的性能瓶颈, 使得 F2FS 的压缩文件缓冲写能够真正从 `largefolios` 中受益。

3.6 脏页写回 Page Writeback

3.6.1 功能背景

在深入探讨脏页回写之前, 我们必须首先理解 F2FS 压缩文件的工作机制。F2FS 并非逐块压缩, 而是将多个连续的逻辑块 (通常是 4 个, 即 16 KB) 组成一个压缩簇 (`CompressCluster`)。文件系统会尝试将整个簇作为一个单元进行压缩。如果压缩后的体积小于原始体积, F2FS 就会将这个压缩后的数据块作为一个整体写入磁盘, 并在磁盘上将簇的第一个块地址用一个特殊的 `COMPRESS_ADDR` 标记来指代这一个簇的起始。

核心概念: 压缩簇 (CompressCluster)

- **原子单元:** 压缩和解压总是以整个簇为单位进行。要读取簇中的任何一页, 都必须先将整个簇读入内存并解压。
- **延迟提交:** 当写入一个压缩文件时, 脏页不会立即被提交到bio 层。它们会被暂存到一个名为 `compress_ctx` 的压缩上下文中。只有当这个上下文被一个完整的簇填满时, F2FS 才会触发压缩和回写操作。

3.6.2 核心挑战与关键问题

f2fs 的压缩簇目前的实现是一个簇中的page 数量是固定的 $4 < n$, 默认情况下 $n=1$ 。然而, 从内存中分配的largefolio 阶数是无法预知的,(内存层有一套自适应阶数分配算法)。因此我们不能假设一个folio 的整体处于同一个簇内。这带来了第一组严峻的挑战:

固有的架构性挑战

- **多对多关系:** 一个高阶 Folio (如 64 KB) 可能横跨多个压缩簇。反之, 一个压缩簇 (如 16 KB) 也可能只包含一个高阶 Folio 的一部分。这种交错关系使得追踪和管理变得异常困难。
- **生命周期不匹配:** Folio 的锁和 writeback 状态是针对整个大内存块的。但它的不同部分可能属于不同的压缩簇, 这些簇的回写时机是完全独立的。如何在一个 Folio 的一部分还在压缩上下文中等待时, 正确处理另一部分的回写完成?
- **并发与截断风险:** 在旧的重试逻辑中, 对 Folio 的反复加锁和解锁, 为并发的部分截断 (PartialTruncation) 创造了巨大的风险窗口。一个被部分截断的 Folio 状态已变, 但旧的检测机制无法识别, 继续写入可能导致数据损坏。
- **重试逻辑失效:** 当写回因锁竞争而返回 `-EAGAIN` 时, 我们必须能从上次失败的精确字节位置重试, 而不是重写整个 Folio, 以避免灾难性的写放大和数据不一致。对于f2fs, 因为它异地分配新数据块写入的特性, 重复写回folio 的同一部分几乎一定会造成磁盘上的数据混乱。

在上述固有挑战的基础上, 我们进一步发现了由内核回写机制和上游补丁引入的更细微、但同样致命的问题:

决赛期间发现的新增挑战

- **后台回写的“空悬” Folio:** 内核的后台回写线程 (如 `pflush`) 会定期扫描并回写脏页。在它的扫描批次中, 可能包含大量已被清理脏位但仍处于回写状态的 Folio。这些 Folio 不会进入我们真正的 I/O 提交流程 (即`find_dirty_range`找不到脏范围), 因此它们不会被正常解锁, 也不会通过 bio 回调来结束回写状态 (`folio_end_writeback`), 导致其永久“挂起”, 阻塞后续操作。
- **子页索引错误:** 在 `f2fs_do_write_data_page` 中, 一个考虑欠妥的上游改动在查询块

地址时, 错误地使用了 folio->index (Folio 首页的索引), 而不是当前正在处理的子页的正确索引。这导致对 LargeFolio 的原地更新 (in-placeupdate) 逻辑彻底失效。

- **Bio 提交流程错误:** 在 `f2fs_submit_page_bio` 中, 另一个上游改动在将 Folio 的一部分添加到 bio 时, 错误地将长度设置为整个 `folio_size`, 而不是单个 `PAGE_SIZE`。这导致 bio 中记录的 I/O 请求范围远大于实际需要, 造成数据不一致风险。

为了同时解决上面诸多繁杂的问题, 我发现, 直接改造f2fs 旧的脏页回写函数`f2fs_write_cache_pages`过于复杂, 而直接使用iomap 现有的脏页回写接口,`iomap_writepages`, 又和f2fs 作为日志结构文件系统的异地写入和对重试写逻辑无法兼容。因此我决定借助iomap 的思想, 但是对其进行扩展和改造。

解决方案 (一): 核心框架改造

基础改造: 从 Page-Aware 到 Folio-Aware 的辅助函数 改造的第一步, 是将整个f2fs 压缩文件中所有处理 `struct page*` 数组的底层辅助函数, 升级为能够理解和高效处理高阶 Folio 的版本。这些函数虽然简单, 但却是整个新架构的基石。

压缩上下文page 添加函数改造对比

旧: `f2fs_compress_ctx_add_page` (逐页操作) 旧函数假设folio 阶数为 0 阶, 仅仅将folio 的第一个page 添加到压缩簇中。完全不具备largefolios 意识。

```
/* f2fs/compress.c (原始) */
void f2fs_compress_ctx_add_page(struct compress_ctx *cc, struct folio *folio
)
{
    unsigned int cluster_ofs;

    if (!f2fs_cluster_can_merge_page(cc, folio->index))
        f2fs_bug_on(F2FS_I_SB(cc->inode), 1);

    cluster_ofs = offset_in_cluster(cc, folio->index);
    cc->rpages[cluster_ofs] = folio_page(folio, 0);
    cc->nr_rpages++;
    cc->cluster_idx = cluster_idx(cc, folio->index);
}
```

新: `f2fs_compress_ctx_add_folio` (Folio 感知) 新代码首先精确接受一段逻辑文件的开始位置和长度组成的区间, 然后首先计算出逻辑文件在当前folio 的位置, 随后将添加长度限制在folio 末尾, 簇末尾和添加长度三者的最小值。

优点: 精准而正确地计算出folio 需要被添加到压缩上下文的的部分的长度的。不需要调用者

手动进行区间的clamp。

```
/* 我的改造版本 */
void f2fs_compress_ctx_add_folio(struct compress_ctx *cc, struct folio *
    folio,
    loff_t pos, loff_t rlen)
{
    unsigned int cluster_ofs;
    loff_t poff = offset_in_folio(folio, pos);
    unsigned int folio_cur_idx = poff >> PAGE_SHIFT;
    /* ... */
    cluster_ofs = offset_in_cluster(cc, folio->index + folio_cur_idx);

    int num_pages_to_add =
        min_t(unsigned int,
            (unsigned int)folio_nr_pages(folio) - folio_cur_idx,
            cc->cluster_size - cc->nr_rpages);
    num_pages_to_add = min_t(unsigned int, num_pages_to_add,
        rlen >> PAGE_SHIFT);

    for (int i = 0; i < num_pages_to_add; ++i) {
        cc->rpages[cluster_ofs] = folio_page(folio, folio_cur_idx + i);
        cc->nr_rpages++;
        cluster_ofs++;
    }
    cc->cluster_idx = cluster_idx(cc, folio->index);
}
```

簇状态管理函数对比举例

旧: `f2fs_drop_rpages` (逐页操作) 旧代码通过一个简单的 for 循环, 逐个处理 `rpages` 数组中的每一个页指针。

缺点: 当遇到一个高阶 Folio 时, 它会多次对同一个 Folio 执行 `unlock_page` 或 `put_page`, 这不仅效率低下, 而且是逻辑错误的根源。

```
/* f2fs/compress.c (原始) */
static void f2fs_drop_rpages(struct compress_ctx *cc, int len, bool unlock)
{
    int i;

    for (i = 0; i < len; i++) {
        if (!cc->rpages[i])
            continue;
        if (unlock)
```

```
unlock_page(cc->rpages[i]);
```

```
else
```

```
put_page(cc->rpages[i]);
```

```
}
```

```
}
```

新：f2fs_drop_rpages (Folio 感知) 新代码通过 `page_folio` 获取到 Folio，并计算出这个 Folio 在当前簇中占据的连续页数 `num_to_skip`。

优点： 对每个 Folio 只执行一次解锁或释放操作，然后一次性跳过它所包含的所有页，既正确又高效。

```
/* 我的改造版本 */
```

```
static void f2fs_drop_rpages(struct compress_ctx *cc, int len, bool unlock)
{
```

```
    int i;
```

```
    unsigned int num_to_skip = 0;
```

```
    for (i = 0; i < len;) {
```

```
        struct folio *rfolio = page_folio(cc->rpages[i]);
```

```
        if (unlock)
```

```
folio_unlock(rfolio);
```

```
else
```

```
folio_put(rfolio);
```

```
num_to_skip = min( ... );
```

```
/* ... logic to calculate num_to_skip ... */
```

```
i += num_to_skip;
```

```
}
```

```
}
```

类似地，`set_cluster_writeback`、`set_cluster_dirty`、`f2fs_put_rpages_wbc` 等一系列函数都遵循了相同的改造模式，将逐页循环优化为了按 Folio 区间跳跃处理。

回写主循环对比

旧: `f2fs_write_cache_pages` 这是一个典型的、复杂而脆弱的手动实现。

缺点:

- 手动聚合: 通过 `filemap_get_folios_tag` 手动从 `pagecache` 中抓取脏页, 并填充到一个本地数组 `pages` 中。
- 手动加锁: 在循环内部, 需要显式地对每个 `folio` 调用 `folio_lock`。
- 状态检查繁琐: 必须手动检查 `folio->mapping`、`folio_test_dirty`、`folio_test_writeback` 等多个状态, 逻辑复杂且容易出错。
- Folio 拆分: 它将高阶 Folio 强行拆分为单个 `page`, 丢失了整体性, 使得直接在 `f2fs_write_cache_pages` 上支持 `largefolios` 不可行。

```
/* f2fs_write_cache_pages */
while (!done && !retry && (index ≤ end)) {
    nr_folios = filemap_get_folios_tag(mapping, &index, end,
                                       tag, &fbatch);
    // ... 将 folio 拆成 page 填入 pages[] 数组 ...
    for (i = 0; i < nr_pages; i++) {
        struct page *page = pages[i];
        // ...
```

```
folio_lock(folio);
```

```
    if (unlikely(folio->mapping ≠ mapping)) {
        folio_unlock(folio);
        continue;
    }
    // ... 大量状态检查 ...
    ret = f2fs_write_single_data_page( ... );
}
}
```

新: `f2fs_write_cache_folios` 新实现完全拥抱了 VFS 的 `writeback_iter` 框架。

优点:

- 自动迭代: `writeback_iter` 优雅地处理了所有脏 Folio 的查找、锁定和状态检查。我们只需在循环中处理它返回的、已准备好的 Folio。
- 自动化状态管理: `writeback_iter` 返回的 Folio 保证是锁定的、脏的、且属于当前 mapping 的, 极大地简化了我们的逻辑。
- Folio 整体性: 我们始终以一个完整的 Folio 为操作单元, 完美契合 LargeFolio 的设计哲学。
- 职责清晰: 框架负责“找”, 我们负责“写”, 分工明确。

```

/* 我的 f2fs_write_cache_folios 实现 */
static int f2fs_write_cache_folios(struct address_space *mapping, struct
writeback_control *wbc, enum iostat_type io_type)
{
    struct folio *folio = NULL;
    int err;
    // ...
    while ((folio = writeback_iter(mapping, wbc, folio, &err))) {
        // 此时 folio 已被框架准备好, 直接处理
        u64 pos      = folio_pos(folio);
        u64 end_pos = pos + folio_size(folio);

        // 循环处理 folio 内的每一个脏区间
        while ((r_len = f2fs_iomap_find_dirty_range(folio, &pos, end_aligned
))) {
            if (is_compressed_file) {
                // 添加到压缩上下文
                f2fs_compress_ctx_add_folio(&cc, folio, pos, r_len);
            } else {
                // 直接写
                err = f2fs_write_single_data_folio( ... );
            }
            pos += r_len;
        }
        // ...
    }
    // ...
}

```

解锁生命周期控制: `cc_bytes_pending` 与延迟解锁 这是整个改造中最精妙的部分, 它解决了 Folio 生命周期与压缩簇延迟提交之间的核心矛盾。

cc_bytes_pending 的生命周期

```

/* 1. 在 f2fs_write_cache_folios 中, 当 folio 的一个脏区被加入压缩上下文时, 增加计数 */
if (is_compressed_file) {
    // ...
    f2fs_compress_ctx_add_folio(&cc, folio, pos, r_len);
    if (fifs) {

        atomic_add(r_len, f2fs_ifs_cc_pending_bytes_ptr(fifs, folio));

    }
}

/* 2. 当 f2fs_write_cache_folios 处理完一个 folio 后, 检查计数器 */
if (!err && fifs && atomic_read(f2fs_ifs_cc_pending_bytes_ptr(fifs, folio))
    > 0) {
    // 如果计数 > 0, 说明有部分数据被暂存, 不能立即解锁
    // 我们给它打上一个 "延迟解锁" 的标记

    f2fs_set_folio_private_deferred_unlock(folio);

} else {
    // 否则, 安全解锁
    folio_unlock(folio);
}

/* 3. 在最终写回函数 (如 f2fs_write_raw_pages) 中, 当暂存的数据被处理后, 减少计数并检查解锁条件 */
ret = f2fs_write_single_data_folio(folio, ...);
if (fifs) {
    // 减少计数, 并检查是否为0且有延迟解锁标记
    if (atomic_sub_and_test(submitted << PAGE_SHIFT,
        f2fs_ifs_cc_pending_bytes_ptr(fifs, folio))
        && f2fs_folio_private_deferred_unlock(folio)) {

        folio_unlock(folio);

    }
}

// 在这里最终解锁!
}

```


最终解锁逻辑对比

旧: f2fs_write_raw_pages 旧的实现逻辑混乱。它会在 f2fs_write_raw_pages 的开头, 简单粗暴地将簇中所有的 page 解锁, 这是为了防止与 prepare_compress_overwrite 死锁。但当 Folio 横跨多簇时, 这种做法会导致对同一个 Folio 的反复“解锁-加锁”, 是严重的逻辑 bug。

```
/* f2fs_write_raw_pages (原始) */
for (i = 0; i < cc->cluster_size; i++) {
    if (!cc->rpages[i])
        continue;

    redirty_page_for_writepage(wbc, cc->rpages[i]);
```

```
unlock_page(cc->rpages[i]);
```

```
// 危险的无条件解锁
}
// ...
for (i = 0; i < cc->cluster_size; i++) {
    // ...
```

```
lock_page(cc->rpages[i]);
```

```
// 重新加锁
// ...
}
```

新: f2fs_write_raw_pages 新的实现不再有这种危险的解锁/加锁循环。它完全依赖于 cc_bytes_pending 和延迟解锁标志来管理生命周期。它只负责处理数据, 而将解锁的决策权交给了计数器。

```
/* f2fs_write_raw_pages (改造后) */
// (开头的解锁循环被完全移除)
for (i = 0; i < cc->cluster_size; i++) {
    // ...
    // Folio 保持锁定状态进入
    ret = f2fs_write_single_data_folio(folio, ... );

    // 在 f2fs_write_single_data_folio 内部或其回调中
    // 通过 cc_bytes_pending 决定是否最终解锁
```

```

    if (atomic_sub_and_test( ... ) && f2fs_folio_private_deferred_unlock( ... ))
    {
        folio_unlock(folio);
    }
}

```

健壮的重试机制：集中化与无锁释放的权衡 重试背景: F2FS 有一个定期的写检查点 (WriteC heckpoint) 机制, 它会获取一把全局写锁。此时, 所有其他文件操作, 包括脏页回写, 在尝试获取读锁时会以非阻塞方式失败, 并返回 `-EAGAIN`。从逻辑上来看的话, 我们应该执行重试写回的逻辑而不应该把错误返回给文件系统上层, 因为这并非真正的 I/O 错误。

LargeFolio 的挑战: 当一个 64 KB 的 Folio 正在回写, 写到 32 KB 时, `f2fs_do_write_data_page` 返回 `-EAGAIN`, 我们应该怎么做? 简单地重写整个 64 KB Folio 是不可接受的。我们必须从失败的 32 KB 处精确地继续。

解决方案: 我们将所有重试逻辑从上层 (`f2fs_write_cache_pages`) 下沉并集中到 `f2fs_write_single_data_folio` 内部。

集中化的重试逻辑

```

/* f2fs_write_single_data_folio (改造后) */
int f2fs_write_single_data_folio(struct folio *folio, int *
    submitted_pages_count,
    struct bio **bio_ptr, sector_t *last_block_ptr,
    struct writeback_control *wbc,
    enum iostat_type io_type,
    unsigned int compr_blocks, bool is_reclaim,
    u64 start, u64 end
)
{
    /*写回folio的一整段区间*/
    // ...
    if (folio_order(folio) == 0) {
retry_single:
        err = f2fs_write_single_data_page( ... );
        if (err == -EAGAIN) {
            folio_unlock(folio);
            if (wbc->sync_mode == WB_SYNC_ALL || ... ) {
                f2fs_io_schedule_timeout(DEFAULT_IO_TIMEOUT);
                folio_lock(folio);
                // ...
                goto retry_single;
            }
        }
    }
}

```

```

        }
        return -EAGAIN;
    }
    return err;
}

/* 高阶 folio 的循环 */
for (pgoff_t i = folio_start_idx; i ≤ folio_end_idx; ++i) {
    // ...
retry_multi:
    err = f2fs_do_write_data_page(&fio); // 核心写操作
    if (err == -EAGAIN) {
        if (wbc->sync_mode == WB_SYNC_ALL || compr_blocks) {
            f2fs_io_schedule_timeout(DEFAULT_IO_TIMEOUT);
            fio.need_lock = LOCK_REQ;
            goto retry_multi;
        } else {
            err = -EAGAIN;
            goto handle_multi_err;
        }
    }

    // ...
}
return err;
}

```

解决方案 (二): 关键问题修复与细节完善

1. 修复后台回写中的“空悬”Folio 为了解决后台回写线程可能导致 Folio 被永久锁住的问题，我们在 `f2fs_write_cache_folios` 的主循环末尾增加了专门的收尾处理逻辑。

`f2fs_write_cache_folios` 中的收尾逻辑

```

static int f2fs_write_cache_folios( ... )
{
    // ... writeback_iter 主循环 ...
    while ((folio = writeback_iter( ... ))) {
        // ... 脏页查找与提交流程 ...
        // submitted 变量记录了是否有脏页被实际提交
    }

    // 循环结束后，对最后一个 folio 进行检查
    if (folio_test_locked(folio)) {
        // 如果 folio 在处理时仍然是锁住状态，说明它没有
        // 进入正常的 bio 提交流程，我们必须手动解锁。
        folio_unlock(folio);
    }
}

```

```

    if (!submitted) {
        // 如果 submitted 为 0, 说明这个 folio 没有任何部分被提交到 bio。
        // 这通常发生在它是一个“干净”的 folio 时。
        // 它不会有 bio 回调, 所以我们必须手动结束其回写状态。
        if (folio_test_writeback(folio))
            folio_end_writeback(folio);
    }
    // ...
}

```

逻辑阐述: 首先我们判断当前的 Folio 是否还被锁住。如果它没有进入任何提交流程（比如没有脏范围），它就不会被解锁，所以我们必须手动 `folio_unlock(folio)`。然后，我们根据记录实际提交量的 `submitted` 变量判断，如果没有任何部分被提交，就意味着它不会通过 `bio` 回调来结束回写状态，因此我们必须手动调用 `folio_end_writeback(folio)` 来完成生命周期，避免其永久“空悬”。

为了让 `!submitted` 判断生效，我们还必须修复一个原生 F2FS 代码中的遗漏：在原地更新的 IO 提交流程中，没有正确标记 `submitted` 状态。

补充原地更新路径的提交流程标记

```

int f2fs_submit_page_bio(struct f2fs_io_info *fio)
{
    // ...
    if (is_read_io(bio_op(bio)))
        f2fs_submit_read_bio(fio->sbi, bio, fio->type);
    else
        f2fs_submit_write_bio(fio->sbi, bio, fio->type);

    fio->submitted = true;

    return 0;
}

```

2. 修正子页索引计算错误 我们修复了 `f2fs_do_write_data_page` 中块地址查询的逻辑，通过 `folio_page_idx(folio, fio->page)` 加上 Folio 的基地址 `folio->index`，来计算出当前子页的正确全局索引，而不是错误地只使用基地址。

`f2fs_do_write_data_page` 中正确的索引计算

```

int f2fs_do_write_data_page(struct f2fs_io_info *fio)
{
    // ...
    if (need_inplace_update(fio) &&
        f2fs_lookup_read_extent_cache_block(inode,

```

```
folio->index + folio_page_idx(folio, fio->page)

,

    &fio->old_blkaddr)) {
    if (!f2fs_is_valid_blkaddr(fio->sbi, fio->old_blkaddr,
        DATA_GENERIC_ENHANCE))
        return -EFSCORRUPTED;

    ipu_force = true;
    fio->need_lock = LOCK_DONE;
    goto got_it;
}

/* Deadlock due to between page->lock and f2fs_lock_op */
if (fio->need_lock == LOCK_REQ && !f2fs_trylock_op(fio->sbi))
    return -EAGAIN;

err = f2fs_get_dnode_of_data(&dn,

folio->index + folio_page_idx(folio, fio->page)

,
LOOKUP_NODE);
if (err)
    goto out;
fio->old_blkaddr = dn.data_blkaddr;
// ...
}
```

3. 修正 bio 提交流程中的页大小与偏移 最后，我们修复了 `f2fs_submit_page_bio` 函数，确保向 bio 添加页面时，使用正确的页面大小（`PAGE_SIZE`）和在 Folio 内的精确偏移，彻底解决上游补丁引入的数据范围错误问题。

`f2fs_submit_page_bio` 中正确的 bio 添加方式

```
int f2fs_submit_page_bio(struct f2fs_io_info *fio)
{
    struct bio *bio;
    struct folio *fio_folio = page_folio(fio->page);
    struct folio *data_folio = fio->encrypted_page ?
        page_folio(fio->encrypted_page) : fio_folio;
    if (!f2fs_is_valid_blkaddr(fio->sbi, fio->new_blkaddr,
        fio->is_por ? META_POR : (__is_meta_io(fio) ?
        META_GENERIC : DATA_GENERIC_ENHANCE)))
        return -EFSCORRUPTED;
```

```
trace_f2fs_submit_folio_bio(data_folio, fio);

/* Allocate a new bio */
bio = __bio_alloc(fio, 1);

f2fs_set_bio_crypt_ctx(bio, fio_folio->mapping->host,
    fio_folio->index, fio, GFP_NOIO);

bio_add_folio_nofail(bio, data_folio, PAGE_SIZE,
    folio_page_idx(data_folio, fio->encrypted_page ?
    fio->encrypted_page : fio->page) << PAGE_SHIFT);

// ...
}
```

逻辑阐述:

- 将错误的 `folio_size` 改回了正确的 `PAGE_SIZE`。
- 新增了 `bio_add_folio_nofail` 的第四个参数 (偏移量), 通过 `folio_page_idx(...)<<PAGE_SHIFT` 计算得出, 精确地告诉 `bio` 我们要添加的是 `data_folio` 中的哪一页, 从而解决了数据范围错误的问题。

3.7 垃圾回收 GC

3.7.1 功能背景

闪存的物理特性与挑战

- **写入限制:** NAND 闪存设备有一个根本性的物理限制: 在向一个存储单元 (页, Page) 写入新数据之前, 必须先擦除其所在的整个块 (Erase Block)。一个擦除块通常包含数百个页。
- **写放大 (Write Amplification):** 如果文件系统天真地在原地更新数据 (例如, 只修改一个 4KB 文件), 它将不得不读取整个擦除块 (如 2MB) 到内存, 修改那 4KB 数据, 然后擦除整个块, 最后再将整个 2MB 数据写回。这个过程导致实际写入磁盘的数据量远大于用户请求的数据量, 即“写放大”。这会急剧消耗闪存的有限擦写寿命 (P/E Cycles) 并降低性能。

F2FS 的日志结构设计与 GC 的诞生

- **日志结构文件系统 (LFS):** 为了克服上述挑战, F2FS 采用了日志结构 (Log-structured) 的设计。它从不原地更新数据, 而是将所有修改 (包括新数据和旧数据的更新版本) 作

为”日志”顺序追加到闪存的干净区域。这种方式将大量随机的小 IO 聚合成连续的顺序大 IO，极大地提升了写入性能并保护了闪存寿命。

- **“无效数据”的产生**: 这种设计的副作用是，当文件被修改或删除时，旧版本的数据块并不会被立即擦除，它们只是在文件系统的元数据中被标记为”无效”。随着时间的推移，文件系统中会散布着大量不再被任何活动文件引用的无效数据块，占用了宝贵的存储空间。
- **垃圾回收 (Garbage Collection) 的使命**: GC 机制就是为了解决这个问题而生的。它的核心任务是定期扫描文件系统，找到那些混合了有效和无效数据的”脏”段 (Segment)。然后，它会将其中仍然有效的活数据块 (Live Blocks) 复制 (搬运) 到一个新的、干净的段中，并更新元数据指向这些新位置。一旦所有活数据都被搬走，原来的”脏”段就只剩下无效数据，可以被安全地整个擦除，从而回收空间以备后续写入。可以说，GC 是 F2FS 维持长期稳定性和可用空间的生命线。

3.7.2 重大问题分析

在主内核中，Matthew Wilcox 先生的补丁已经将 GC 的接口从 `page` 迁移到了 `folio`。然而，这种迁移是初步的，其实现仍然基于”一个 `folio` 等于一个 4KB 页”的陈旧假设，这在引入 `largefolios` 后引发了两个严重问题。

问题 1: 高阶 Folio 协同与写放大

当 GC 处理一个高阶 `folio` (例如 64KB) 时，它也必须与我们为 `iomap` 设计的 `f2fs_iomap_folio_state` 机制协同工作。具体来说：

- **私有标志位冲突**: GC 过程可能会设置一些状态标志 (如 `PG_private_gcging`)，这些标志需要被安全地存储在 `f2fs_iomap_folio_state` 中，以避免与 `iomap` 自身的状态冲突。
- **写放大风险**: 如果 GC 只是因为 `folio` 中的一个 4KB 子页需要被回收，就将整个 64KB 的 `folio` 标记为脏并回写，这将导致严重的写放大。因此，GC 路径也必须具备逐块脏页追踪的能力，精确地只回写那些真正需要被搬运的数据。

问题 2: 因接口误用导致的双重解锁 (Double Unlock)

这是一个更为隐蔽和危险的 bug，源于 F2FS GC 路径中一个令人困惑的接口使用惯例。在 GC 上下文中，`fio->encrypted_page` 字段的语义被复用，它实际上代表”任何需要后处理的页”，而不仅仅是加密页。GC 会将被选中的数据读到这个 `page` 中，然后将其搬运到元数据 `inode` 对应的 `meta_mapping` 中。这个过程导致了灾难性的后果：

Bug 复现路径：

- (a) **数据预读**: GC 调用 `ra_data_block` 来读取需要被搬运的数据。
- (b) **I/O 提交**: 在 `f2fs_submit_page_bio` 中，由于 `fio->encrypted_page` 存

在, bio 实际上被关联到了 meta_mapping 中的那个新 page, 而不是原始的 folio。

- (c) **I/O 完成与崩溃:** 当 I/O 完成时, 会调用 `f2fs_iomap_finish_folio_read`。这个回调函数是针对 **原始数据 folio** 的。然而, 在 (a) 步骤中, 原始数据 folio 已经被 `f2fs_folio_put` 解锁了。因此, 当 `f2fs_iomap_finish_folio_read` 再次尝试对这个 folio 调用 `folio_end_read` 时, 就发生了对已解锁 folio 的第二次解锁, 导致内核崩溃。

ra_data_block 中的错误根源

```
static int ra_data_block(struct inode *inode, pgoff_t index)
{
    ...
    struct f2fs_io_info fio = { ... };
    struct folio *folio; // 这是原始数据 folio
    ...
    folio = f2fs_grab_cache_folio(mapping, index, true);
    ...
    // 关键点1: 将一个从 meta_mapping 获取的 page 赋值给 fio->encrypted_page
    // 这个 page 实际上是数据搬运的目的地
    fio.encrypted_page = f2fs_pagecache_get_page(META_MAPPING(sbi),
        dn.data_blkaddr,
        FGP_LOCK | FGP_CREAT, GFP_NOFS);
    ...
    err = f2fs_submit_page_bio(&fio); // 提交 I/O
    ...
    // 关键点2: 在这里, 原始数据 folio 被解锁并释放了引用计数!
    f2fs_folio_put(folio, true);
    ...
}
```

f2fs_submit_page_bio 中的 I/O 转向

```
int f2fs_submit_page_bio(struct f2fs_io_info *fio)
{
    ...
    struct folio *fio_folio = page_folio(fio->page);
    // data_folio 变成了 encrypted_page 所属的 folio
    struct folio *data_folio = fio->encrypted_page ?
        page_folio(fio->encrypted_page) : fio_folio;
    ...
    // 旧的、错误的实现: 总是添加整个 folio
    // bio_add_folio_nofail(bio, data_folio, folio_size(data_folio), 0);
    ...
}
```


f2fs_iomap_finish_folio_read 中的致命一击

```
void f2fs_iomap_finish_folio_read(struct folio *folio, ... )
{
    ...
    // 此时的 folio 是已经被 ra_data_block 释放的 folio
    // 再次调用 folio_end_read 会导致双重解锁
    folio_end_read(folio, uptodate);
    ...
}
```

解决方案**步骤 1: 实现精确的脏页标记**

我们在数据页的搬运函数 `move_data_page` 中，加入了对 `folio` 阶数（order）的判断。

- 如果是 0 阶 `folio`，我们沿用旧逻辑，直接调用 `folio_mark_dirty`。
- 如果是高阶 `folio`，则调用 `iomap_set_range_dirty`，它会利用 `f2fs_iomap_folio_state` 来精确标记出当前正在被 GC 的那个子页所对应的范围是脏的，从而避免了对整个 `folio` 的不必要回写。

步骤 2: 改造 I/O 提交逻辑，传递子范围信息

这是解决双重解锁问题的关键。我们利用改造后的、支持范围的 `f2fs_io_info` 结构体，向 `f2fs_submit_page_bio` 传递了精确的块数量和页内偏移。

通过这个改动，我们告诉块设备层，我们只关心 `data_folio` 中从 `fio->idx` 开始、长度为 `fio->cnt` 个块的区域。这样，即使 `data_folio` 是一个高阶 `folio`，I/O 操作也只会针对其正确的子集进行，从而修复了之前错误的 I/O 范围。

步骤 3: 同步改造 GC 的读路径

GC 不仅涉及写，还涉及读。因此，我们也必须改造 GC 读取数据块的函数 `f2fs_submit_page_read`，使其在向 `bio` 添加页面时，也能正确传递子页在 `folio` 内的偏移。

这一系列改动形成了一个完整的闭环，确保了 F2FS 的垃圾回收机制在读、写、脏页标记等所有关键路径上，都能正确、高效地处理高阶 `folio`，从根本上解决了写放大和双重解锁的严重问题。

move_data_page 中的精确脏页标记

```
static int move_data_page( ... )
{
    ...
    if (gc_type == BG_GC) {
        if (folio_test_writeback(folio)) {
            err = -EAGAIN;
            goto out;
        }
        f2fs_set_folio_private_gcing(folio);

        if (folio_order(folio) == 0) {
            folio_mark_dirty(folio);
        } else {
            iomap_set_range_dirty(&folio->page,
                (u64)folio->index << PAGE_SHIFT, PAGE_SIZE);
        }
    } else {
        ...
    }
    ...
}
```

f2fs_submit_page_bio 的修正

```
int f2fs_submit_page_bio(struct f2fs_io_info *fio)
{
    ...
    struct folio *data_folio = fio->encrypted_page ?
        page_folio(fio->encrypted_page) : page_folio(fio->page);
    ...
    bio = __bio_alloc(fio, 1);
    ...

    // 核心改动：
    // 1. 长度使用 fio->cnt (回退到 1 个块)，而不是整个 folio_size()
    // 2. 偏移量使用 fio->idx, 精确指定 folio 内的子页位置
    bio_add_folio_nofail(bio, data_folio,
        F2FS_BLK_TO_BYTES(fio->cnt ? fio->cnt : 1),
        fio->idx << PAGE_SHIFT);

    ...
}
```

f2fs_submit_page_read 的修正

```
static int f2fs_submit_page_read(struct inode *inode, struct folio *folio,
                                block_t blkaddr, blk_opf_t op_flags, pgoff_t index,
                                bool for_write)
{
    ...
    /* wait for GCed page writeback via META_MAPPING */
    f2fs_wait_on_block_writeback(inode, blkaddr);

    // 核心改动：添加第四个参数，即页内字节偏移
    // (index - folio->index) 计算出子页在 folio 中的相对页索引
    if (!bio_add_folio(bio, folio, PAGE_SIZE,
                      (index - folio->index) << PAGE_SHIFT)) {

        ...
        return -EFAULT;
    }
    ...
    f2fs_submit_read_bio(sbi, bio, DATA);
    return 0;
}
```

项目性能测试

为了全面评估本项目中 f2fs 文件系统对 largefolios 支持所带来的性能影响，我们设计并执行了一套严谨的性能测试方案。我们使用业界著名的存储性能测试工具 fio，针对普通文件、空洞（sparse）文件以及压缩文件这三种典型场景，分别进行了缓存读（bufferedread）与缓存写（bufferedwrite）的基准测试。

测试平台覆盖了两种具有代表性的环境：标准的 x86-64 架构下的 QEMU 虚拟机，以及前沿的 ARM 架构裸金属设备——树莓派 5。特别地，在树莓派 5 上的测试基于最新的 16KB 页面大小配置进行，其测试结果对于未来在 ARM 架构上推广 16KB 基页技术具有重要的参考价值和开创性意义。

本报告将首先详细介绍我们的测试环境、核心测试脚本与方法论，随后展示在两大平台上的具体性能测试结果并进行深入分析，最后介绍用于数据后处理与可视化的辅助脚本。

4.1 测试环境与方法论

精准的性能测试依赖于一套自动化、可复现且能有效隔离干扰的测试流程。为此，我们编写了一系列脚本来统一调度测试、设置环境、采集数据。

4.1.1 核心测试调度脚本：run_benchmark.py

整个测试流程由核心调度脚本 run_benchmark.py 驱动。该脚本负责编排整个测试矩阵，包括遍历不同的块大小（BlockSizes）、读写模式以及多队列深度（QueueDepth），并为每组测试配置执行指定次数的迭代，以获取稳定、可靠的性能数据。

该脚本的核心职责包括：

- **参数解析与矩阵生成：**解析用户输入（如内核版本标识、测试类型、目标文件等），并与预定义的块大小列表（4k, 128k, 1M, 10M, 100M, 1G）组合，生成完整的测试任务队列。
- **环境“写栅栏”设置：**在进行任何读写测试之前，调用 `set_write_fence()` 函数。此函数的目的是为了最大限度地减少内核后台任务对测试结果的干扰。在标准的 Linux 内核中，当应用程序向页缓存（PageCache）写入数据后，这些“脏页”会由后台的回写（writeback）机制在稍后的某个时间点异步地刷写到磁盘。这种异步回写行为会占用 CPU 和 I/O 带宽，从而对我们测量的纯缓存读写性能造成干扰。`set_write_fence()` 通过修改 `sysctl` 内核参数（如 `vm.dirty_ratio`, `vm.dirty_background_ratio`, `vm.dirty_writeback_centisecs`），临时性地“关闭”这个自动回写机制，确保在测试期间所有写操作都停留在内存中，从而精确测量缓存性能。测试结束后，`restore_write_fence()` 函数会负责将这些参数恢复原状。
- **只读测试环境隔离：**在执行“冷读”测试前，脚本会调用 `remount()` 函数，将目标文件系统以只读（read-only）模式重新挂载。这样做是为了彻底杜绝在清理缓存（调用 `sync` 命令）时可能引发的元数据更新（如访问时间 `atime`），确保读测试的纯粹性。
- **迭代调用 fio：**对测试矩阵中的每一个测试点，脚本会循环调用底层的 `fio_test.sh` 脚本执行 10 次，并将结果分别记录。这种多次迭代取平均值的做法，旨在平滑由 CPU 调度抖动或其他瞬时系统活动带来的数据波动，使最终结果更具统计意义。

以下是 `run_benchmark.py` 脚本的完整代码：

核心测试调度脚本：run_benchmark.py

```
#!/usr/bin/env python3
import subprocess
import argparse
import psutil
import sys
import math
from pathlib import Path

DEFAULT_FILE = "/mnt/f2fs/lf.c"
DEFAULT_QD_SET = [1]
# --- 测试矩阵定义 ---
BLOCK_SIZES = ["1M", "10M", "100M", "1G", "4k", "128k"]
ITERATIONS = 10 # 每组测试运行10次

# --- 脚本配置 ---
FIO_SCRIPT_PATH = "./fio_test.sh"
RESULTS_DIR = "fio_results"
# --- 全局变量，用于保存原始内核参数 ---
original_vm_settings = {} # <--- 关键修复：在这里初始化全局字典
def get_mount_point(target: str) -> str:
    """返回目标文件所在的 mount point，依赖 findmnt(8)。"""
    return subprocess.check_output(
        ["findmnt", "-no", "TARGET", "--target", target], text=True
```

```

    ).strip()
def remount(target_mp: str, mode: str):
    """mode 取 'ro' 或 'rw'。始终带 noatime、nodiratime 避免 atime update。"""
    opts = f"remount,{mode},noatime,nodiratime"
    print(f" [mount] sudo mount -o {opts} {target_mp}")
    subprocess.run(["sudo", "mount", "-o", opts, target_mp], check=True)

def set_write_fence():
    """
    为整个测试套件设置“写回屏障”。
    保存原始值，然后设置能阻止脏页回写的参数。
    """
    global original_vm_settings
    print("\n--- [FENCE UP] Setting kernel parameters to prevent writeback
    ... ---")
    params_to_change = {
        "vm.dirty_ratio": "100",
        "vm.dirty_background_ratio": "100",
        "vm.dirty_writeback_centisecs": "0"
    }
    try:
        # 1. 保存原始值
        for param in params_to_change.keys():
            result = subprocess.run(["sysctl", "-n", param], capture_output=
True, text=True, check=True)
            original_vm_settings[param] = result.stdout.strip()
            print(f" Original settings saved: {original_vm_settings}")

        # 2. 设置新值
        for param, value in params_to_change.items():
            print(f" Setting {param} = {value}")
            subprocess.run(["sudo", "sysctl", "-w", f"{param}={value}"],
check=True, capture_output=True)

        print("--- Kernel writeback fence is ACTIVE. ---")

    except (subprocess.CalledProcessError, FileNotFoundError) as e:
        print(f"[!!!] CRITICAL ERROR: Failed to set kernel parameters: {e}",
file=sys.stderr)
        print("[!!!] Aborting tests.", file=sys.stderr)
        restore_write_fence()
        sys.exit(1)

def restore_write_fence():
    """
    恢复原始的内核脏页回写参数。
    """

```

```

global original_vm_settings
if not original_vm_settings:
    return

print("\n--- [FENCE DOWN] Restoring original kernel parameters ... ---")
try:
    for param, value in original_vm_settings.items():
        print(f" Restoring {param} = {value}")
        subprocess.run(["sudo", "sysctl", "-w", f"{param}={value}"],
            check=True, capture_output=True)
    print("--- Kernel parameters restored successfully. ---")
    original_vm_settings = {}
except (subprocess.CalledProcessError, FileNotFoundError) as e:
    print(f"[!!!] CRITICAL ERROR: Failed to restore kernel parameters: {e}", file=sys.stderr)
    print("[!!!] Please restore them manually:", file=sys.stderr)
    for param, value in original_vm_settings.items():
        print(f" sudo sysctl -w {param}={value}", file=sys.stderr)
    sys.exit(1)

def get_system_memory_gb():
    """获取系统总内存，并以GB为单位返回一个整数标签"""
    mem_bytes = psutil.virtual_memory().total
    gb_rounded = math.ceil(mem_bytes / (1024**3))
    return f"{gb_rounded}G"

def run_single_test(kernel_ver, mem_config, rw_mode, bs, file_path, ioengine, qd):
    """
    qd: 对 psync 映射为 numjobs; 对 io_uring 映射为 iodepth
    """
    if ioengine == "psync":
        numjobs = qd
        iodepth = 1
    else:
        numjobs = 1
        iodepth = qd

    print(f" -> Running: bs={bs}, mode={rw_mode}, kernel={kernel_ver}, mem={mem_config}, "
        f"file={file_path}, engine={ioengine}, qd={qd} (iodepth={iodepth}, numjobs={numjobs})")

    title = f"{kernel_ver}_{mem_config}_mem"
    extra_fio_flags = []
    if rw_mode == 'r':
        extra_fio_flags += ["--readonly=1",
            "--allow_file_create=0",

```

```

                                "--unlink=0"]

try:

    subprocess.run(
        [
            "sudo", "bash", FIO_SCRIPT_PATH,
            title, bs, rw_mode,
            f"file={file_path}",
            f"ioengine={ioengine}",
            f"iodepth={iodepth}",
            f"numjobs={numjobs}",
            f"qd={qd}",
            *extra_fio_flags,
        ],
        check=True,
        capture_output=False,
        text=True
    )

except subprocess.CalledProcessError as e:
    print(f" [!] ERROR running test for bs={bs}, qd={qd}.")
    print(f" [!] STDERR: {e.stderr}")
    return False
return True

def main():
    parser = argparse.ArgumentParser(description="Automated FIO benchmark orchestrator.")
    parser.add_argument("--name", "-n", required=True, help="Kernel version identifier (e.g., 'vanilla', 'iomap_v1').")
    parser.add_argument("--type", "-t", required=True, choices=['read', 'write', 'sync', 'r', 'w', 's'], help="Test type: read/write/sync.")
    parser.add_argument("--file", "-f", default=DEFAULT_FILE, help=f"Target file for FIO (default: {DEFAULT_FILE})")
    parser.add_argument("--ioengine", "-e", choices=["psync", "io_uring", "libaio"], default="psync", help="fio ioengine.")
    parser.add_argument("--qdsset", default="1", help="Comma-separated queue depths to sweep.")
    parser.add_argument("--qdwrite", action="store_true", help="Also sweep queue depths for WRITE tests.")
    args = parser.parse_args()

    try:
        qd_list = [int(x) for x in args.qdsset.split(",") if x.strip()]
    except ValueError:
        raise SystemExit("Invalid --qdsset. Use like: 1,2,4,8,16")

```

```

mem_config = get_system_memory_gb()
print(f"--- Detected System Memory: {mem_config} ---")
Path(RESULTS_DIR).mkdir(exist_ok=True)

# is_write_test = args.type in ['w', 'write']
need_fence=True

try:
    if need_fence:
        set_write_fence()

    print(f"\n≡ Starting Benchmark for Kernel: {args.name}, Type: {
args.type} ≡")

    if args.type in ['read', 'r']:
        mount_point = get_mount_point(args.file)
        remount(mount_point, "ro")          # ----- ro,noatime
    -----
    print("Running COLD READ tests (cache cleared before each run)
... ")
    for bs in BLOCK_SIZES:
        print(f"\n[+] Testing Block Size: {bs}")
        for qd in qd_list:
            for i in range(ITERATIONS):
                print(f" - Iteration {i+1}/{ITERATIONS}")
                run_single_test(args.name, mem_config, 'r', bs, args
.file,args.ioengine, qd)
            remount(mount_point, "rw")
        elif args.type in ['w', 'write']:
            print("Running HOT WRITE tests (on cached file)... ")
            print("\n[+] Preparing cached file by performing an initial
write ... ")
            if not run_single_test(args.name+"_cold", mem_config, 'w', '1M',
args.file, args.ioengine, qd_list[0]):
                print("[!] Failed to prepare cached file. Aborting.")
            return

    sweep_list = qd_list if args.qdwrite else [qd_list[0]]
    for bs in BLOCK_SIZES:
        print(f"\n[+] Testing Block Size: {bs}")
        for qd in sweep_list:
            for i in range(ITERATIONS):
                print(f" - Iteration {i+1}/{ITERATIONS}")
                run_single_test(args.name, mem_config, 'w', bs, args
.file, args.ioengine, qd)

    elif args.type in ['sync', 's']:

```



```

print("Running SYNC WRITE tests (fsync after every write) ... ")
sweep_list = qd_list if args.qdwrite else [qd_list[0]]
for bs in BLOCK_SIZES:
    print(f"\n[+] Testing Block Size: {bs}")
    for qd in sweep_list:
        for i in range(ITERATIONS):
            print(f"  - Iteration {i+1}/{ITERATIONS}")
            run_single_test(args.name, mem_config, 's', bs, args
.file, args.ioengine, qd)

finally:
    if need_fence:
        restore_write_fence()

print("\n=== Benchmark Finished! ===")
print(f"All results are in the '{RESULTS_DIR}' directory.")

if __name__ == "__main__":
    main()

```

4.1.2 底层测试执行脚本：fio_test.sh & clear_cache.sh

run_benchmark.py 脚本的每次 run_single_test() 调用，都会通过 subprocess 模块启动一个 fio_test.sh 子进程。这个 bash 脚本是实际与 fio 命令交互的执行层。它负责接收来自上层脚本的参数（如实验标题、块大小、读写模式等），并组装成一个完整的 fio 命令。

一个关键设计是，它会生成一个包含所有测试参数和时间戳的、具有唯一性的日志文件名，例如 iomap_v1_8G_mem_write_bs-1M_20240101-123000.log。这种命名规范极大地便利了后续的数据解析。

此外，对于读测试，该脚本会首先调用 clear_cache.sh。这是一个非常简单的脚本，其核心命令是 sudotee/proc/sys/vm/drop_caches<<<3，作用是清空系统的页缓存、目录项和 inode 缓存，确保接下来的读操作是从磁盘（或在我们的测试中，是未被缓存的虚拟块设备）发起，从而实现“冷读”测试。

底层测试执行与缓存清理脚本

```

#!/bin/bash

# 脚本功能：运行FIO读/写测试，并将结果保存到唯一的、描述性的文件中。
# 用法： ./fio_test.sh <experiment_title> [block_size] [r|w]
# 示例 1（默认读，1M bs）： ./fio_test.sh vanilla_kernel
# 示例 2（指定读，4k bs）：  ./fio_test.sh vanilla_kernel 4k r
# 示例 3（指定写，128k bs）： ./fio_test.sh modified_kernel 128k w

# --- 基础配置（可以根据需要修改） ---
FIO_TARGET_FILE="/mnt/f2fs/lf.c"
RESULTS_DIR="fio_results"

```

```

CLEAR_CACHE_SCRIPT="./clear_cache.sh"
DEFAULT_BS="1M"
DEFAULT_RW_MODE="r" # <--- 默认模式现在是 'r' (read)
IOENGINE="psync"
IODEPTH=1
NUMJOBS=1
USER_QD="" # 仅用于输出命名
USE_FSYNC_EVERY_IO=0
# --- 全局变量, 用于保存原始内核参数 ---
for kv in "${@:4}"; do
    case "$kv" in
        file=*) FIO_TARGET_FILE="{kv#file=}" ;;
        ioengine=*) IOENGINE="{kv#ioengine=}" ;;
        iodepth=*) IODEPTH="{kv#iodepth=}" ;;
        numjobs=*) NUMJOBS="{kv#numjobs=}" ;;
        qd=*) USER_QD="{kv#qd=}" ;;
        *) echo "警告: 未知参数 $kv" ;;
    esac
done
OLD_DIRTY_RATIO=""
OLD_DIRTY_BG_RATIO=""

# --- 清理函数: 无论脚本如何退出, 都会被调用以恢复内核参数 ---
function cleanup_and_restore_thresholds {
    echo # 换行
    # 只有当我们成功修改过参数后才需要恢复
    if [ -n "$OLD_DIRTY_RATIO" ]; then
        echo "--- Restoring original kernel dirty page thresholds ... ---"
        sudo sysctl -w vm.dirty_ratio="$OLD_DIRTY_RATIO"
        sudo sysctl -w vm.dirty_background_ratio="$OLD_DIRTY_BG_RATIO"
        echo "Kernel parameters restored."
    fi
}

# --- 关键的安全保障: 设置一个“陷阱”(trap) ---
# 无论脚本是正常结束(EXIT), 还是被中断(INT)或终止(TERM),
# 都会自动调用上面的 cleanup_and_restore_thresholds 函数。
trap cleanup_and_restore_thresholds EXIT INT TERM

# 1. 检查用户是否提供了实验标题 (第一个参数)
if [ -z "$1" ]; then
    echo "错误: 请提供一个实验标题作为第一个参数。"
    echo "用法: $0 <experiment_title> [block_size] [r|w]"
    exit 1
fi

# 2. 准备变量

```

```

EXPERIMENT_TITLE=$1

# 检查是否提供了块大小（第二个参数），否则使用默认值
BS=${2:-$DEFAULT_BS}

# 检查是否提供了读写模式（第三个参数），并将其转换为fio能识别的格式
USER_RW_MODE=${3:-$DEFAULT_RW_MODE}
case "$USER_RW_MODE" in
    r|read)
        RW_MODE="read"
        MODE_TAG="read"
        ;;
    w|write)
        RW_MODE="write"
        MODE_TAG="write"
        ;;
    s|sync)
        RW_MODE="write"          # fio 的 rw 仍然是 write
        MODE_TAG="sync"         # 文件名里体现为 sync
        USE_FSYNC_EVERY_IO=1    # 每次 write 后 fsync
        ;;
    *)
        echo "错误：无效的模式 '${USER_RW_MODE}'。使用 r|w|s (read|write|sync)。”
        exit 1
        ;;
esac

# 3. 如果是写测试，则启动“安全围栏”
if [ "$RW_MODE" = "write" ]; then
    # echo "--- Write test detected. Setting up 'safe fence' ... ---"
    # 保存原始值
    OLD_DIRTY_RATIO=$(cat /proc/sys/vm/dirty_ratio)
    OLD_DIRTY_BG_RATIO=$(cat /proc/sys/vm/dirty_background_ratio)
    # echo "Original values: dirty_ratio=${OLD_DIRTY_RATIO},
    dirty_background_ratio=${OLD_DIRTY_BG_RATIO}"

    # 设置非常高的临时值
    # echo "Setting high temporary thresholds to prevent writeback ..."
    # sudo sysctl -w vm.dirty_background_ratio=90
    # sudo sysctl -w vm.dirty_ratio=90
fi

TIMESTAMP=$(date +%Y%m%d-%H%M%S)
# 将读写模式也加入到文件名中！
OUTPUT_FILENAME="${EXPERIMENT_TITLE}_${RW_MODE}_bs-${BS}_${TIMESTAMP}.log"
OUTPUT_FILE_PATH="${RESULTS_DIR}/${OUTPUT_FILENAME}"

# 3. 创建结果目录（如果不存在）
mkdir -p "${RESULTS_DIR}"

```

```

echo "===== "
echo "准备执行FIO测试 ..."
echo "  - 实验标题: ${EXPERIMENT_TITLE}"
echo "  - 读写模式: ${RW_MODE} (输入: '${USER_RW_MODE}')"
echo "  - 块大小:   ${BS}"
echo "  - 引擎:     ${IOENGINE}"
echo "  - iodepth:  ${IODEPTH}"
echo "  - numjobs:  ${NUMJOBS}"
echo "  - 文件:     ${FIO_TARGET_FILE}"
echo "  - 时间戳:   ${TIMESTAMP}"
echo "  - 输出:     ${OUTPUT_FILE_PATH}"
echo "===== "

# 5. 清理系统缓存 (你可以取消这里的注释来启用它)
echo "正在清理系统缓存 ..."
if [ "${RW_MODE}" = "read" ]; then
    if [ -f "${CLEAR_CACHE_SCRIPT}" ]; then
        # if [ "${RW_MODE}" = "write" ]; then
            # rm -f "${FIO_TARGET_FILE}"
            # echo "旧的测试文件 ${FIO_TARGET_FILE} 已删除。"
        # fi
        sudo "${CLEAR_CACHE_SCRIPT}"
        vmtouch "${FIO_TARGET_FILE}"
    fi
else
    echo "警告: 缓存清理脚本 ${CLEAR_CACHE_SCRIPT} 未找到, 跳过此步骤!"
fi

# sleep 2
# 组装额外 fio 选项
EXTRA_OPTS=( )
if [ "${USE_FSYNC_EVERY_IO}" -eq 1 ]; then
    EXTRA_OPTS+=( --fsync=1 )    # 每次写后立刻 fsync; 如需 fdatsync 改成 --
    fdatsync=1
fi

# 6. 执行FIO命令
echo "开始运行FIO ..."
fio --name="buffered-${RW_MODE}" \
    --filename="${FIO_TARGET_FILE}" \
    --rw="${RW_MODE}" \
    --bs="${BS}" \
    --io_size=100M \
    --ioengine=psync \
    --numjobs=1 \
    --group_reporting \
    --allow_file_create=0 \
    "${EXTRA_OPTS[@]}" \
    --output="${OUTPUT_FILE_PATH}"

```

```
# 7. 测试完成
echo "-----"
echo "测试完成！"
echo "结果已保存在：${OUTPUT_FILE_PATH}"
echo "-----"

#!/bin/bash
# clear_cache.sh
echo "Syncing filesystem to ensure all pending data is written ..."
sync
echo "Dropping PageCache, dentries and inodes ..."
# 必须用 sudo 或 root 权限运行
echo 3 | sudo tee /proc/sys/vm/drop_caches
echo "Caches dropped. System is ready for a cold read test."
```

4.2 性能测试结果与分析

我们将在本节详细展示并分析在 QEMU 虚拟机和树莓派 5 两个平台上的测试结果。其中,“noiomap”代表未应用本项目优化的基线内核,“iomap”代表应用了 largefolios 优化的内核版本。

4.2.1 QEMU 虚拟机性能测试

测试平台及配置

字段	配置
硬件平台	Intel(R) Xeon(R) Gold 6240, 16 GiB RAM, PCIe 3.0
操作系统	Ubuntu 22.04 LTS
虚拟化层	qemu-system-aarch64 (8 vCPU, 8 GiB RAM)
页面大小	4 KiB (CONFIG_ARM64_4K_PAGES)
存储后端	virtio-blk, 虚拟 f2fs 文件系统镜像, cache=none
测试数据集	普通文件 1 GiB; 空洞文件 1 GiB; 压缩文件 100 MiB

测试性能结果

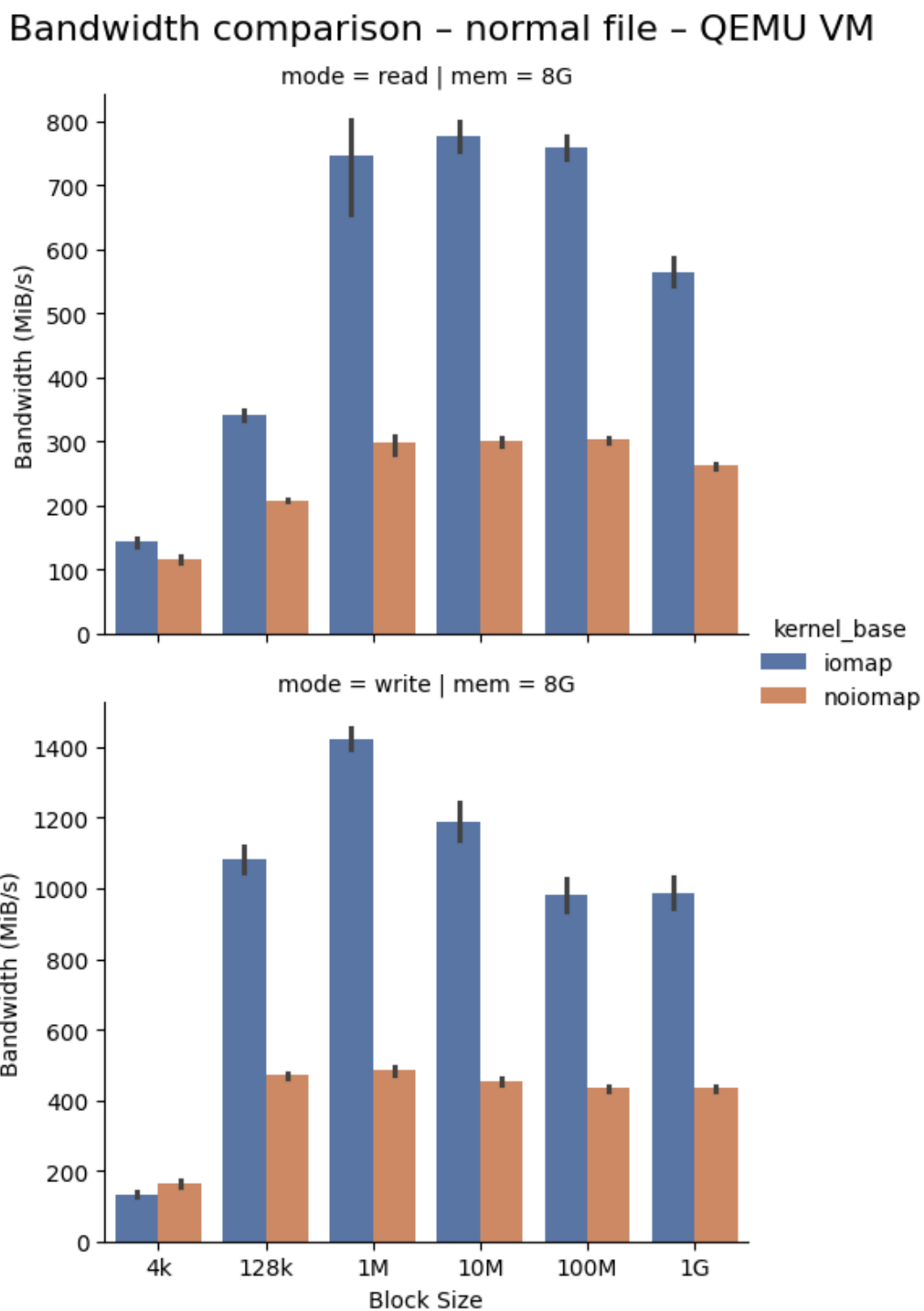


图 1: 普通文件带宽-QEMU VM 缓存读/写, 4 KiB page

Bandwidth comparison – sparse file (hole) – QEMU VM

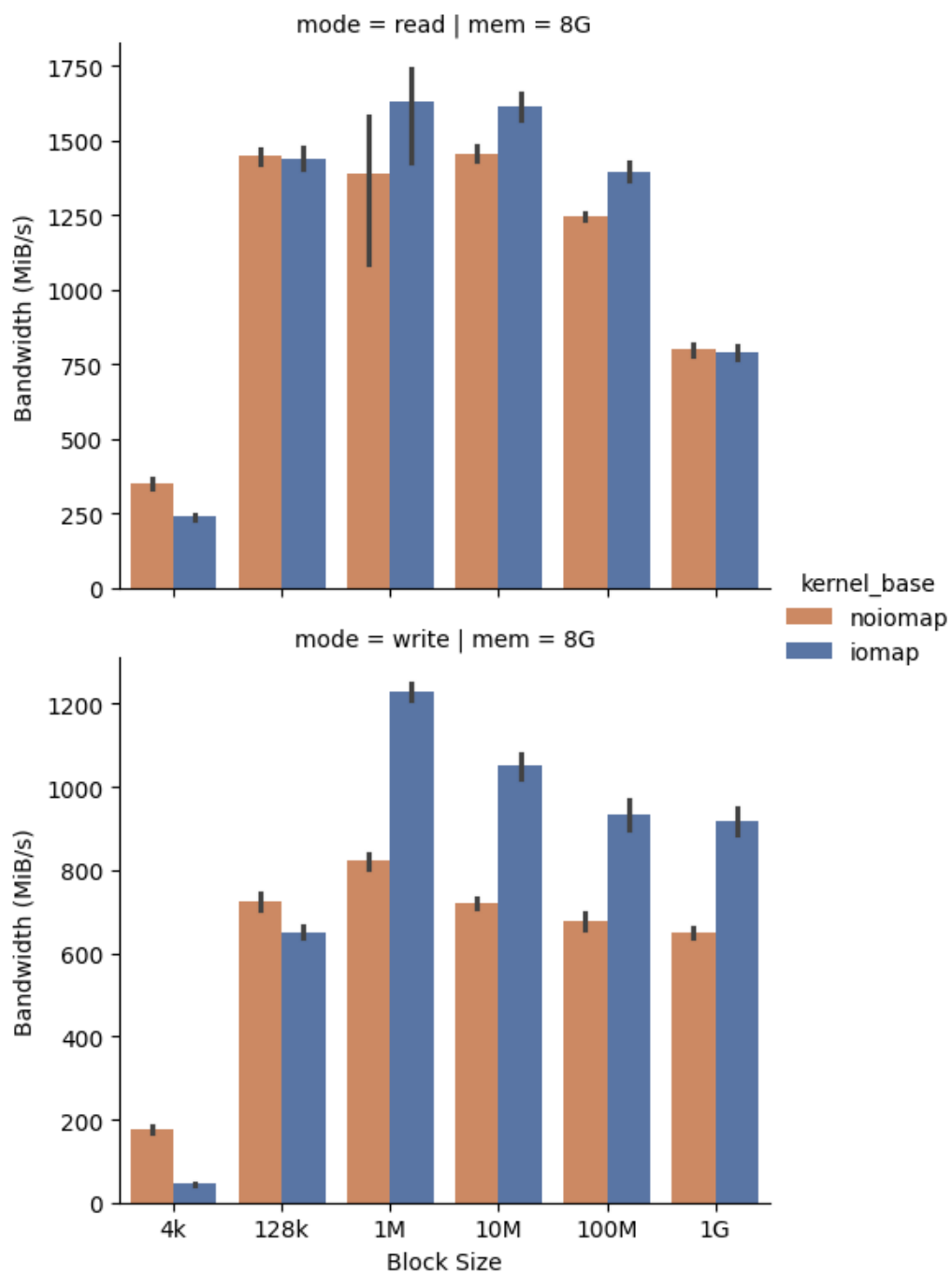


图 2: 空洞文件带宽-QEMU VM 缓存读/写, 4 KiB page

Bandwidth comparison - compressed file - QEMU VM

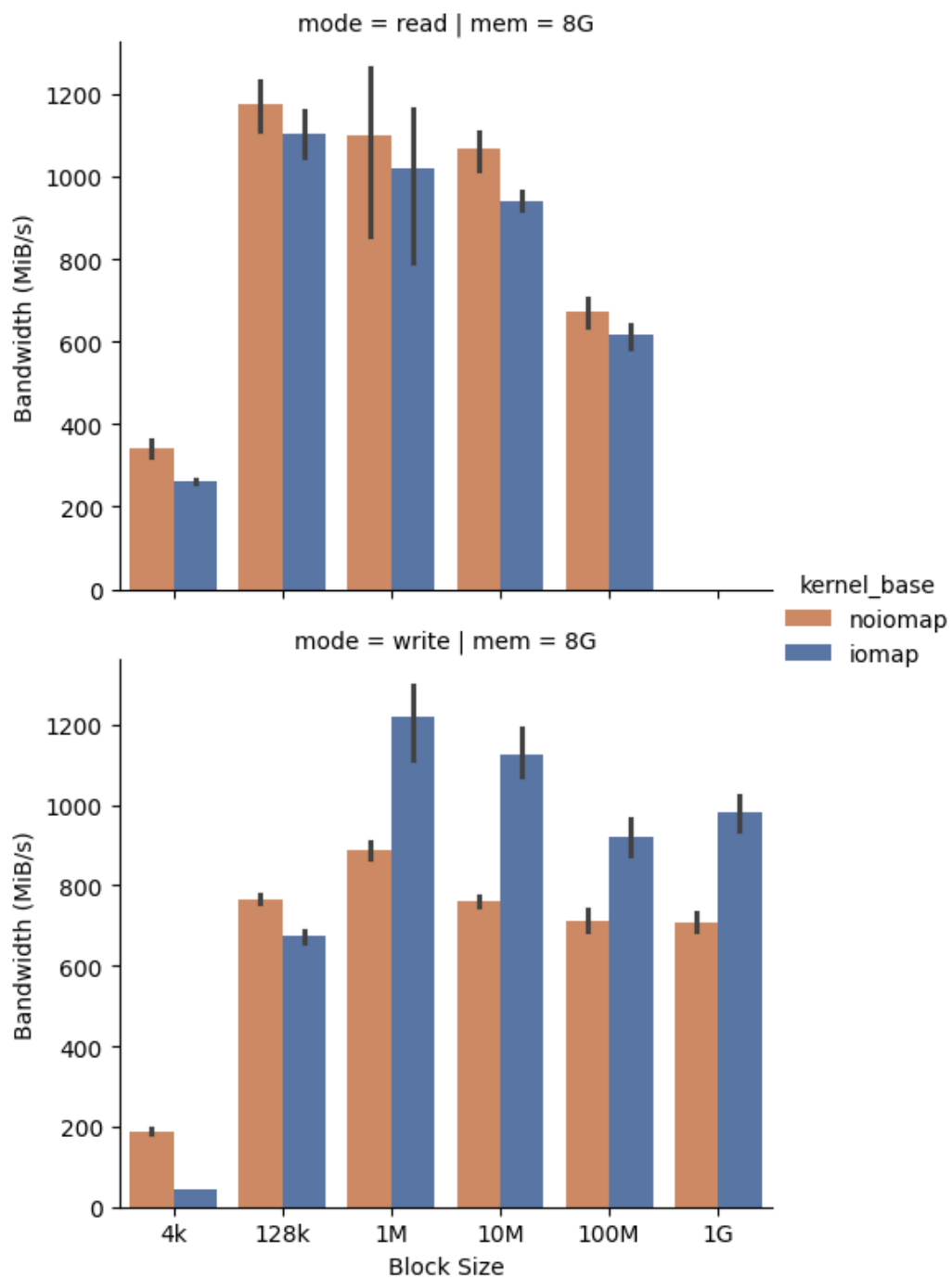


图 3: 压缩文件带宽-QEMU VM 缓存读/写, 4 KiB page

性能提升总结与分析

在 QEMU 虚拟机（4KB 页大小）环境下，我们的 largefolios 优化展现了显著的性能提升，尤其是在大块 I/O 的场景下。

- **普通文件 (Normal File):**
 - **缓存读:** 从 qemu 普通文件读写性能结果图的上半部分可以看出，在所有测试的块大小下，iomap 内核的读取性能均优于或持平于基线内核。随着块大小的增加，性能优势愈发明显，在 1M 块大小下达到峰值，性能提升最高可达 **2.5 倍**！
 - **缓存写:** 写入性能的提升更为惊人。如 qemu 普通文件读写性能结果图下半部分所示，当块大小大于等于 128k 时，iomap 内核的写入带宽远超基线。在 1M 块大小处，性能提升达到最大，接近 **3 倍**！这充分证明了 largefolios 通过减少元数据操作、合并 I/O 请求，极大地提高了大文件写入效率。
 - **问题分析:** 值得注意的是，在 4k 块大小的写入测试中，iomap 内核的性能相较于基线有轻微的劣化。这可能是因为对于小而碎的 I/O，创建和管理 largefolios 的额外开销，超过了其带来的收益。
- **空洞文件 (Sparse File):**
 - 如 qemu 空洞文件结果图所示，对于空洞文件，写入性能的提升同样显著，最高可达 **1.5 倍**。这是因为 largefolios 能够更有效地表示和处理文件中未分配实体的“空洞”区域，减少了不必要的元数据查找和 I/O 开销。
- **压缩文件 (Compressed File):**
 - 对于 f2fs 内置压缩的文件，如 qemu 压缩文件结果图所示，写入性能最高也获得了约 **1.5 倍** 的提升。这表明我们的优化与文件系统的压缩特性可以良好地协同工作。与普通文件类似，在 4k 块大小下也观察到了轻微的性能劣化。

4.2.2 树莓派 5 性能测试

测试平台及配置

字段	配置
硬件平台	Broadcom BCM2712 (Cortex-A76 4C @ 2.4 GHz), 4 GiB RAM, PCIe 2.0
操作系统	Raspberry Pi OS 64-bit
虚拟化层	—(裸机)
页面大小	16 KiB
存储后端	NVMe SSD (256 GB, 格式化为 F2FS 分区)
测试数据集	同上：普通 1 GiB；稀疏 1 GiB；压缩 100 MiB

测试性能结果

Bandwidth comparison - normal file - Raspberry Pi 5

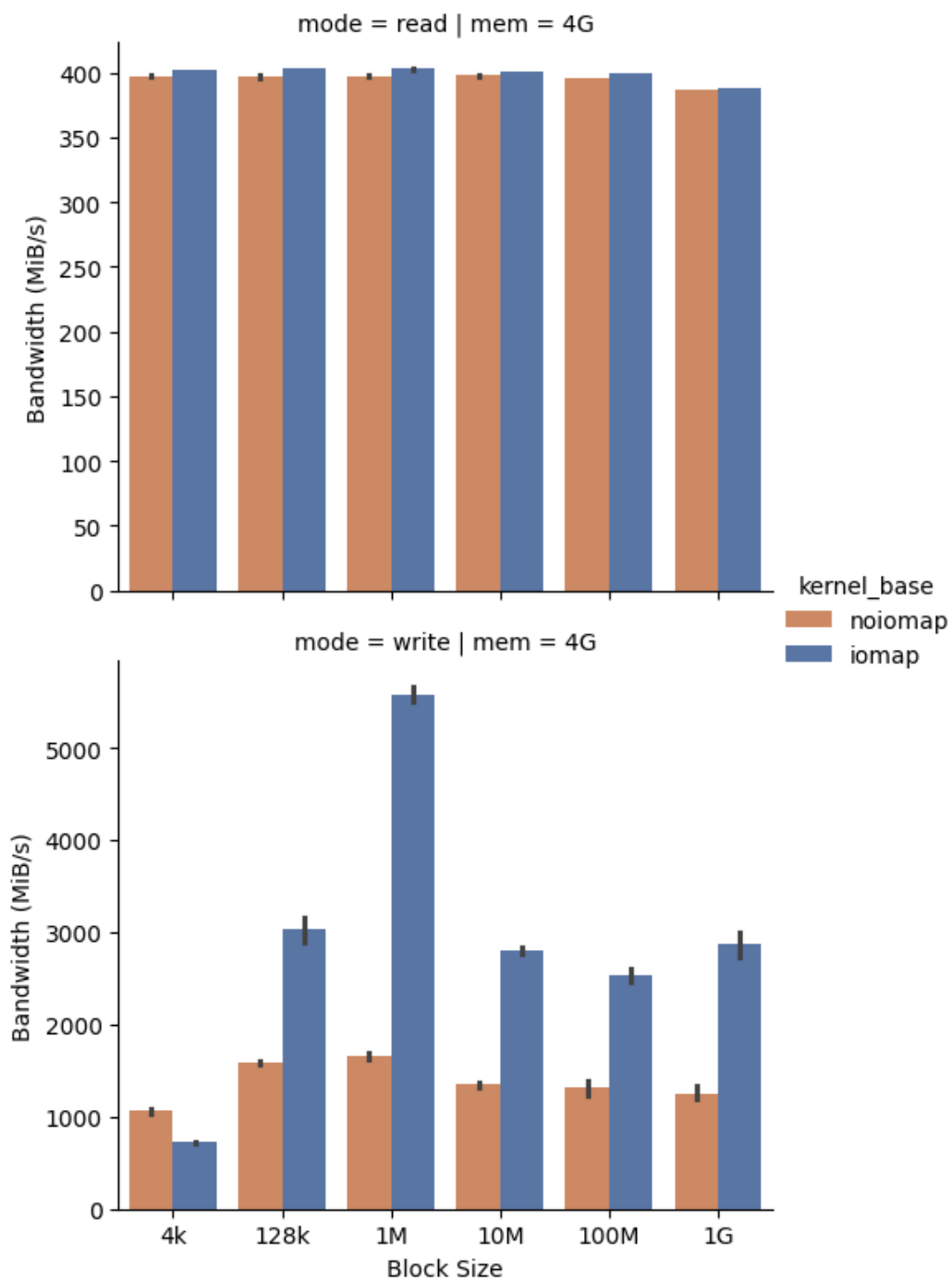


图 4: 普通文件带宽-树莓派 5 缓存读/写, 16 KiB page

Bandwidth comparison - sparse file (hole) - Raspberry Pi 5

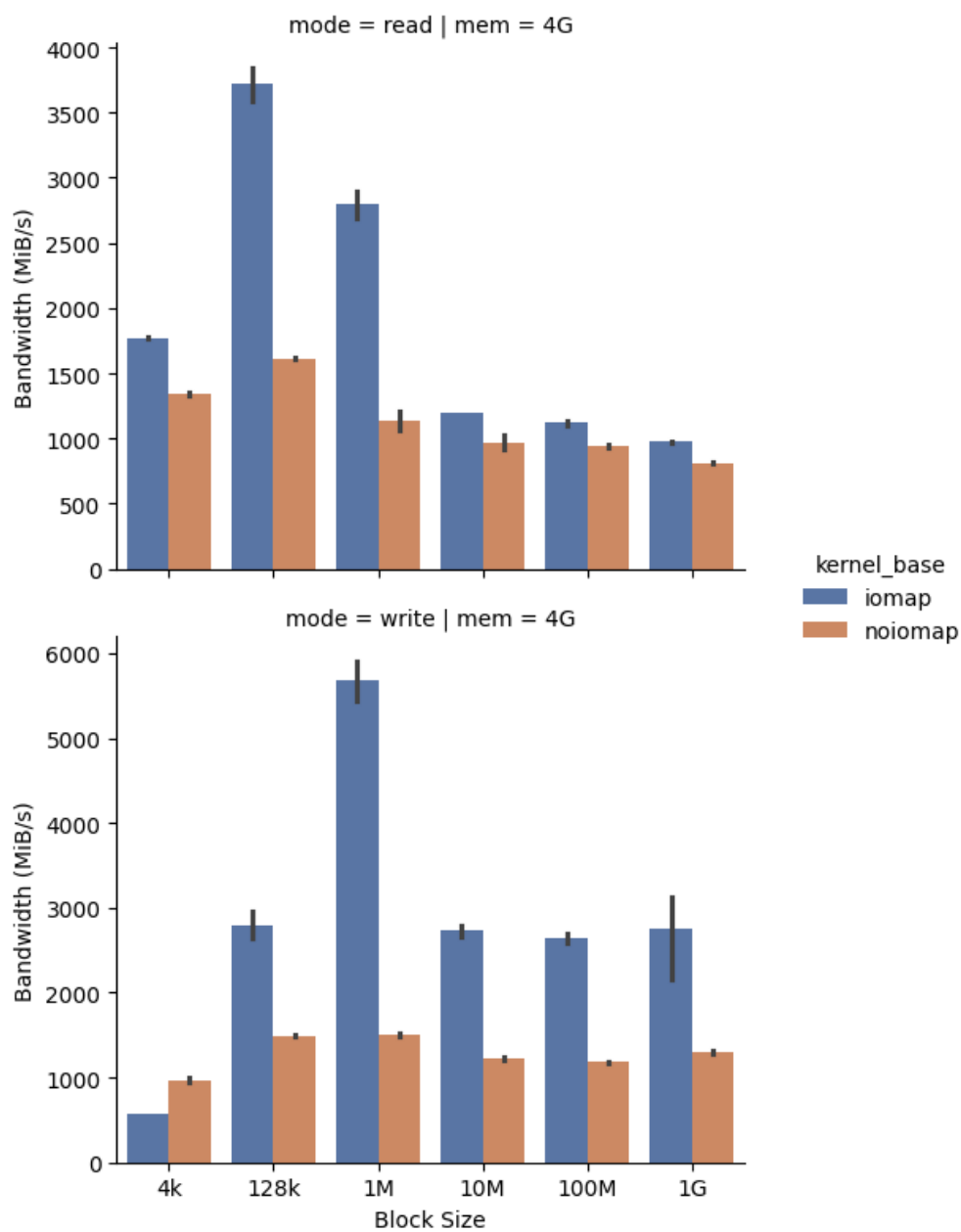


图 5: 空洞文件带宽-树莓派 5 缓存读/写, 16 KiB page

Bandwidth comparison – compressed file – Raspberry Pi 5

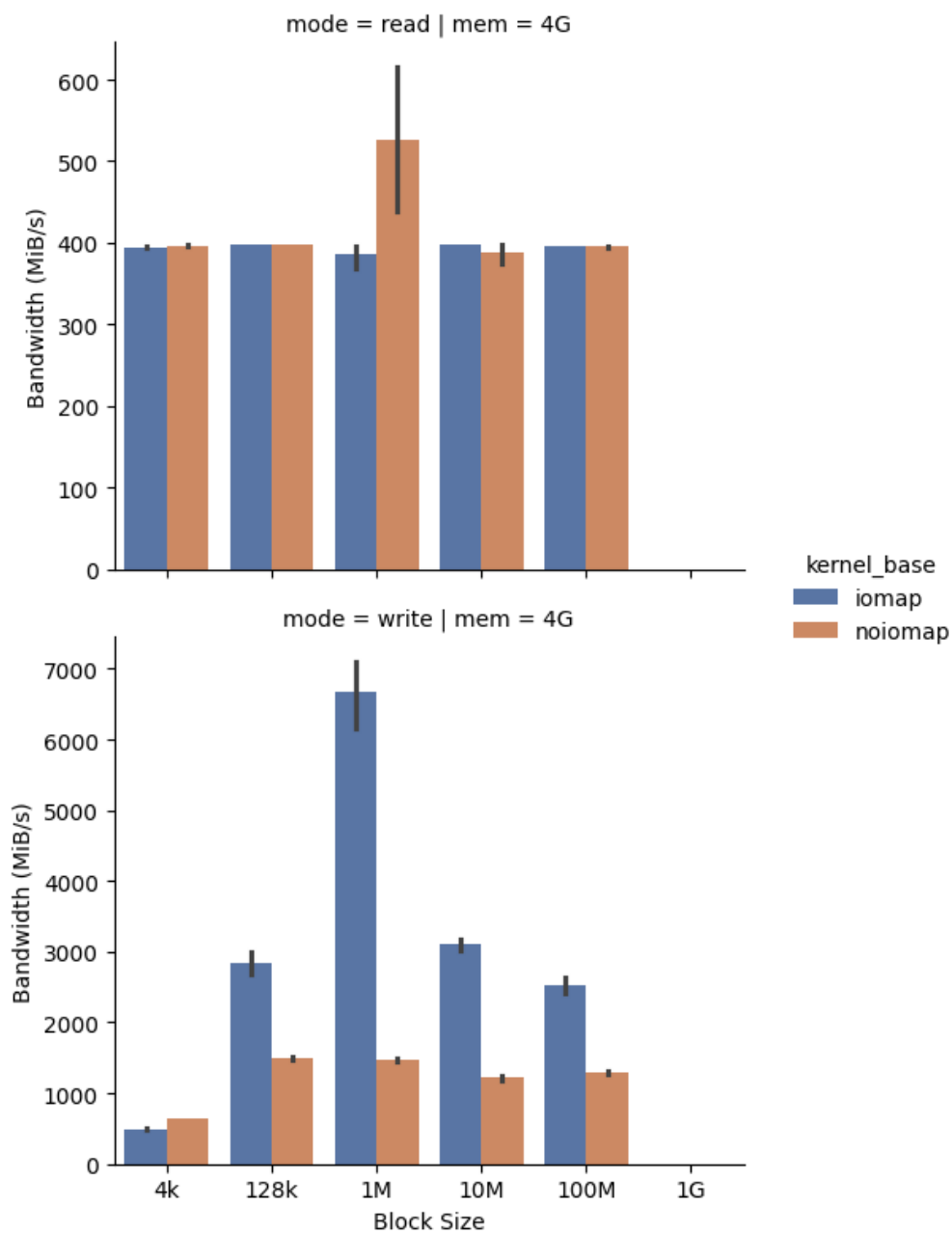


图 6: 压缩文件带宽-树莓派 5 缓存读/写, 16 KiB page

性能提升总结与分析

在搭载了 16KB 基页的树莓派 5 裸金属环境上，largefolios 的优势被进一步放大，尤其是在写入和处理稀疏数据的场景下，性能提升极为显著。

- **普通文件 (Normal File):**

- 缓存写：如树莓派普通文件结果图下半部分所示，写入性能再次成为亮点，最高可提升 2.5 倍。更大的基页大小与 largefolios 结合，使得内存管理的效率更高，单次 I/O 操作能够处理更多的数据，从而带来了巨大的吞吐量提升。
- 缓存读：读性能有小幅提升，但并未像写性能那样取得数量级的飞跃。这主要是因为测试平台的树莓派 5 使用的是 PCIe 2.0 接口，其顺序读取带宽存在物理上限，成为了性能瓶颈。即使 CPU 和内存子系统能够更快地处理数据，最终也受限于 I/O 总线的传输能力。

- **空洞文件 (Sparse File):**

- 这是本次测试中性能提升最惊人的部分。如图树莓派空洞文件结果图所示，缓存读性能最高提升了 2.5 倍，而缓存写性能最高提升了将近 4 倍！这强有力地证明了，在 16KB 页大小的环境下，largefolios 极大地优化了对稀疏数据的处理逻辑，无论是读取跳过空洞，还是写入并创建新的数据块，效率都得到了根本性的改善。

- **压缩文件 (Compressed File):**

- 如树莓派压缩文件结果图所示，压缩文件的写入性能也获得了巨大的飞跃，最高提升可达 4 倍！这表明我们的优化在 16KB 页大小下与压缩功能形成了完美的协同效应。

- **共性分析：**与 QEMU 环境类似，在所有写入测试中，4k 块大小的场景下 iomap 内核依然表现出轻微的性能劣化。这进一步印证了我们的猜想，即对于远小于页大小（16KB）的 I/O 请求，largefolios 机制的介入可能引入了不成比例的额外开销。

4.3 数据处理与可视化脚本

为了从 fio 生成的大量原始日志文件中提取有效信息并进行可视化，我们还开发了两个 Python 辅助脚本。

4.3.1 日志解析脚本：parse_fio_logs.py

在多轮迭代测试后，fio_results 目录下会产生数百个独立的 .log 文件。手动处理这些文件是不现实的。parse_fio_logs.py 脚本的作用就是自动化地完成这一过程。

它会遍历结果目录下的所有日志文件，并使用正则表达式从每个文件中提取关键信息：

- **元数据：**从文件名中解析出内核版本、内存配置、测试模式、块大小等。
- **性能指标：**从文件内容中提取带宽 (BW)、每秒 I/O 操作数 (IOPS)、平均延迟 (clatavg)、CPU 使用率 (usr% 和 sys%) 等核心数据。

该脚本具有良好的健壮性，能够处理单位后缀（如 k、m）和不同时间单位（usec、msec），最终将所有解析出的结构化数据汇总并输出到一个单一的 results.csv 文件中，为后续的数据分析和绘图奠定基础。

FIO 日志解析脚本: parse_fio_logs.py

```
#!/usr/bin/env python3
"""
parse_fio_logs.py · v5 · bullet-proof

* 兼容文件名中的连字符
* BW/bw、IOPS 大小写不敏感
* IOPS 支持 k / m 后缀并换算为「次/秒」
* clat 同时支持 usec / msec, 无则写 None
* CPU 行只要在同一行里出现 usr= 和 sys= 就能抓到
"""

import re, csv, math
from pathlib import Path

RESULTS_DIR = "./fio_results"
OUTPUT_CSV = "./results.csv"

# ----- 小工具 ----- #

def _parse_with_suffix(value: str) -> float:
    """把 '212k' / '15.3m' / '987' 统一转成 float"""
    if value.lower().endswith('k'):
        return float(value[:-1]) * 1_000
    if value.lower().endswith('m'):
        return float(value[:-1]) * 1_000_000
    return float(value)

# ----- 解析单个 log ----- #

def parse_fio_log(path: Path):
    txt = path.read_text()
    fname = path.name

    # 1) 文件名元数据 -----
    m = re.match(
        r"^(?P<kernel>[\w-]+)_(?P<mem>\d+G)_mem_"
        r"(?P<mode>read|write|sync)_bs-(?P<bs>[\w\d]+)_(?P<ts>[\d-]+)\.log$",
        fname)
    if not m:
        print(f"[skip] 文件名不匹配: {fname}")
        return None
    meta = m.groupdict()
```

```

# 2) 性能指标 -----
# BW / IOPS
bw_rg = re.search(r'BW[=:]((\d+\.?\d*)[GM]iB/s)', txt, re.I)
iops_rg = re.search(r'IOPS=(\d+\.?\d*[kKmM]?)', txt, re.I)

# clat 行 (可选)
clat_rg = re.search(r'clat \((usec|msec)\):.*?avg= *([\d.]+)', txt)
p99_rg = re.search(r'99\.00th=\\s*([\d.]+)]', txt)

# CPU 一行
cpu_rg = re.search(
    r'^.*cpu.*usr=([\d.]+)%.*sys=([\d.]+)%', txt,
    flags=re.I | re.M)

# ----- 校验 & 容错 ----- #
if not (bw_rg and iops_rg and cpu_rg):
    print(f"[warn] 必要字段缺失 → {fname}")
    return None # 跳过这份

# 3) 单位换算 -----
bw = float(bw_rg.group(1))
if "GiB/s" in bw_rg.group(0):
    bw *= 1024 # 统一成 MiB/s

iops = _parse_with_suffix(iops_rg.group(1)) # 转成 “次/秒”

# clat 可能缺
clat_avg = None
clat_p99 = None
if clat_rg:
    clat_avg = float(clat_rg.group(2))
    if clat_rg.group(1) == 'msec':
        clat_avg *= 1000 # 统一成微秒
if p99_rg:
    clat_p99 = float(p99_rg.group(1))
    if "clat percentiles (msec)" in txt:
        clat_p99 *= 1000

cpu_usr = float(cpu_rg.group(1))
cpu_sys = float(cpu_rg.group(2))

return {**meta,
        "bw_mib_s": bw,
        "iops": iops,
        "clat_avg_us": clat_avg,
        "clat_p99_us": clat_p99,
        "cpu_usr_pct": cpu_usr,
        "cpu_sys_pct": cpu_sys}

```

```
# ----- 主程序 ----- #

def main():
    rows = [ r for p in Path(RESULTS_DIR).glob("*.log")
              if (r := parse_fio_log(p)) ]

    if not rows:
        print("Nothing parsed - 检查文件格式? ")
        return

    with open(OUTPUT_CSV, "w", newline="") as fp:
        w = csv.DictWriter(fp, fieldnames=rows[0].keys())
        w.writeheader(); w.writerows(rows)

    print(f"✓ Parsed {len(rows)} logs → {OUTPUT_CSV}")

if __name__ == "__main__":
    main()
```

4.3.2 结果可视化脚本: visualize_results.py

最后, visualize_results.py 脚本负责将 results.csv 文件中的数据转化为直观的图表。该脚本使用强大的 pandas 库进行数据处理和聚合, 使用 seaborn 和 matplotlib 库进行绘图。

它的智能之处在于:

- **环境自动识别:** 通过检查输入 CSV 文件的路径, 脚本能自动判断测试是在 QEMU 还是树莓派 (Pi) 上进行的, 并为图表添加相应的标题后缀。
- **数据自动分组:** 脚本会自动将内核名称 (如 iomap_hole, noiomap_com) 拆分为“文件类型” (normal, hole, com) 和“内核基线” (iomap, noiomap) 两个维度, 从而可以对不同文件类型分别生成对比图。
- **批量生成图表:** 对于每种文件类型, 脚本都会自动生成三类图表: 带宽对比柱状图、带宽稳定性箱线图以及 CPU 使用率分解图, 最终输出本报告中展示的所有性能结果图。

测试结果可视化脚本: visualize_results.py

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Visualize fio CSV results
- 自动识别运行环境 (Raspberry Pi 5 / QEMU VM)
- 将 kernel 字段拆分为:
    file_type: normal | hole | com
    kernel_base: iomap | noiomap
- 对每种 file_type 各自生成 3 张图:
    bandwidth_<type>.png
```



```

        bandwidth_stability_<type>.png
        cpu_usage_<type>.png
"""
import argparse
from pathlib import Path

import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# ----- 命令行参数 -----
ap = argparse.ArgumentParser(description="Visualize fio CSV results")
ap.add_argument("-i", "--input", required=True, help="输入 CSV 文件路径")
ap.add_argument("-o", "--outdir", default="plots", help="输出图片目录, 默认 plots")
args = ap.parse_args()

INPUT_CSV = args.input
PLOTS_DIR = Path(args.outdir)
PLOTS_DIR.mkdir(parents=True, exist_ok=True)

# ----- 环境标签识别 -----
ENV_TAGS = []
path_parts = [p.lower() for p in Path(INPUT_CSV).resolve().parts]
if any("pi" in p for p in path_parts):
    ENV_TAGS.append("Raspberry Pi 5")
if any("qemu" in p for p in path_parts):
    ENV_TAGS.append("QEMU VM")
env_suffix = f" - {' & '.join(ENV_TAGS)}" if ENV_TAGS else ""

# ----- 读取与轻量清洗 -----
df = pd.read_csv(INPUT_CSV, low_memory=False)

# 1) 删掉夹在中间的“重复表头”行
if "kernel" in df.columns:
    df = df[df["kernel"] != "kernel"].copy()

# 2) 数值列转为 float
num_cols = [
    "bw_mib_s", "iops", "clat_avg_us", "clat_p99_us",
    "cpu_usr_pct", "cpu_sys_pct"
]
for c in num_cols:
    if c in df.columns:
        df[c] = pd.to_numeric(df[c], errors="coerce")

# ----- 衍生列: file_type / kernel_base -----
def split_kernel(k: str):

```

```

k = k.strip()
if k.endswith("_hole"):
    return "hole", k.replace("_hole", "")
elif k.endswith("_com"):
    return "com", k.replace("_com", "")
return "normal", k

df[["file_type", "kernel_base"]] = df["kernel"].apply(
    lambda k: pd.Series(split_kernel(k))
)
mode_order = ["read", "write"]
if "mode" in df.columns:
    df["mode"] = pd.Categorical(df["mode"],
                                categories=mode_order, ordered=True)

# ----- 分类顺序 -----
bs_order = ["4k", "128k", "1M", "10M", "100M", "1G"]
if "bs" in df.columns:
    df["bs"] = pd.Categorical(df["bs"], categories=bs_order, ordered=True)

file_type_order = ["normal", "hole", "com"]
file_type_label = {
    "normal": "normal file",
    "hole": "sparse file (hole)",
    "com": "compressed file"
}

# 固定颜色: iomap = 蓝, noiomap = 橙
PALETTE = {"iomap": "#4c72b0", "noiomap": "#dd8452"}

# ----- 循环生成各类文件的图 -----
for ft in file_type_order:
    sub = df[df["file_type"] == ft].copy()
    if sub.empty:
        continue # 该类型没数据

    lbl = file_type_label[ft]

# 1. 带宽柱状图 -----
g = sns.catplot(
    data=sub,
    kind="bar",
    x="bs", y="bw_mib_s",
    hue="kernel_base",
    col="mem", row="mode",
    palette=PALETTE,
    height=4, aspect=1.25,
    sharey=False
)

```

```

g.set_axis_labels("Block Size", "Bandwidth (MiB/s)")
g.fig.suptitle(f"Bandwidth comparison - {lbl}{env_suffix}",
               fontsize=16, y=1.02)
g.savefig(PLOTS_DIR / f"bandwidth_{ft}.png", bbox_inches="tight")
plt.close(g.fig)

# 2. 带宽箱线图 -----
g = sns.catplot(
    data=sub,
    kind="box",
    x="bs", y="bw_mib_s",
    hue="kernel_base",
    col="mem", row="mode",
    palette=PALETTE,
    height=4, aspect=1.25,
    sharey=False
)
g.set_axis_labels("Block Size", "Bandwidth (MiB/s)")
g.fig.suptitle(f"Bandwidth stability (box, 10 runs) - {lbl}{env_suffix}",
               fontsize=16, y=1.02)
g.savefig(PLOTS_DIR / f"bandwidth_stability_{ft}.png", bbox_inches="
tight")
plt.close(g.fig)

# 3. CPU usr/sys 并列柱 -----
cpu_long = (
    sub.groupby(["kernel_base", "mem", "mode", "bs"])
        [["cpu_usr_pct", "cpu_sys_pct"]]
        .mean()
        .reset_index()
        .melt(
            id_vars=["kernel_base", "mem", "mode", "bs"],
            value_vars=["cpu_usr_pct", "cpu_sys_pct"],
            var_name="cpu_type", value_name="pct"
        )
)
g = sns.catplot(
    data=cpu_long,
    kind="bar",
    x="bs", y="pct",
    hue="cpu_type",
    col="kernel_base", # 左右分栏: iomap / noiomap
    row="mem",
    palette="pastel",
    height=4, aspect=1.25
)
g.set_axis_labels("Block Size", "CPU usage (%)")

```

```
g.fig.suptitle(f"Average CPU breakdown - {lbl}{env_suffix}",
               fontsize=16, y=1.02)
g.savefig(PLOTS_DIR / f"cpu_usage_{ft}.png", bbox_inches="tight")
plt.close(g.fig)

print("🎉 Done! 图表已输出至:", PLOTS_DIR.resolve())
```

开发环境搭建与 workflow

本章将详细介绍本项目从零开始搭建 QEMU 虚拟开发环境、树莓派物理测试环境以及高效的 Git 版本控制 workflow 的全过程，并重点分享在此过程中遇到的关键问题与解决方案。

5.1 QEMU 虚拟开发环境

5.1.1 基本 workflow 搭建

QEMU 提供了一个灵活、高效且可快照的虚拟环境，是内核开发与调试的首选。我们的基础环境搭建流程主要参考了知乎博主“无人知晓”分享的宝贵经验，并在此基础上进行了适配与扩展。

第一步：交叉编译环境准备

(本节内容引自知乎文章：[《qemu 搭建 arm64 linux kernel 环境》](#)，作者：无人知晓)

1. **安装交叉编译工具链**: 为了在 x86_64 主机上编译 aarch64 架构的内核，我们需要安装相应的工具链。在 Ubuntu 22.04 系统上，可以通过以下命令安装：
2. **下载并编译内核**:
 - 从 The Linux Kernel Archives 下载内核源码并解压。
 - 配置并编译内核。注意，CROSS_COMPILE 的值需要与您安装的工具链前缀完全匹配。

安装 aarch64 交叉编译工具链

```
sudo apt-get install gcc-12-aarch64-linux-gnu
```

编译 aarch64 内核

```
# 设置环境变量以简化后续命令
export ARCH=arm64
export CROSS_COMPILE=aarch64-linux-gnu-

# 使用默认配置生成 .config 文件
make defconfig

# 编译内核镜像 (Image) 和设备树 (dtbs)
```

```
# vmlinux 文件用于后续 GDB 调试
make all -j$(nproc)
```

第二步：构建基于 Ubuntu 的根文件系统

(本节内容引自知乎文章：《[QEMU 构建基于 ubuntu 的根文件系统](#)》，作者：无人知晓)

BusyBox 虽然轻量，但功能有限。为了获得更完整的开发调试体验，我们选择构建一个基于 Ubuntu 22.04 的根文件系统。

1. **创建镜像并解压基础包：**
2. **Chroot 并配置内部环境：** 为了在 x86 主机上配置 aarch64 的系统，需要 qemu-user-static。

创建并准备根文件系统镜像

```
# 下载 Ubuntu aarch64 基础包
wget http://cdimage.ubuntu.com/ubuntu-base/releases/22.04/release/ubuntu-
base-22.04-base-arm64.tar.gz

# 创建一个 4G 大小的空镜像文件
dd if=/dev/zero of=ubuntu.img bs=1M count=4096

# 将其格式化为 ext4 文件系统
mkfs.ext4 ubuntu.img

# 挂载镜像并将基础包解压进去
mkdir -p rootfs
sudo mount -t ext4 ubuntu.img rootfs/
sudo tar -xzf ubuntu-base-22.04-base-arm64.tar.gz -C rootfs/
```

使用 qemu-user-static 和 chroot 配置环境

```
# 安装 qemu-user-static
sudo apt-get install qemu-user-static

# 将其复制到即将 chroot 的环境中
sudo cp /usr/bin/qemu-aarch64-static rootfs/usr/bin/

# chroot 到 rootfs 目录中
sudo chroot rootfs
```

```
# --- 在 chroot 环境内执行 ---  
# 更新软件源并安装必要工具  
apt update  
apt install udev vim net-tools ssh ethtool ifupdown rsyslog  
  
# 添加用户并设置密码  
useradd -s '/bin/bash' -m -G adm,sudo myuser  
passwd myuser  
  
# 退出 chroot  
exit
```

5.1.2 高级 workflow 与遇到的问题

问题 1: 多版本内核并行开发与异地编译

为了进行性能基准测试，我需要同时维护和编译多个不同版本的内核（如主线开发版、我的补丁版、用于对比的稳定版）。为保持源码树的干净，我采用了异地编译（Out-of-Tree Build）的策略。

问题 2: 复杂的多实例 QEMU 管理

开发过程中，经常需要一边运行带调试符号的内核进行 GDB 调试，另一边运行优化后的内核进行性能测试。手动管理多个 QEMU 实例的命令行参数、镜像和 PID 非常繁琐。为此，我编写了一个强大的多实例管理脚本。

核心功能解析：

- **实例隔离**: 为每个实例（如 vm1, vm2）创建独立的目录，存放其差分镜像、共享目录和 PID 文件，互不干扰。
- **写时复制 (CoW) 镜像**: 使用 qemu-img 基于一个只读的基准系统镜像创建 qcow2 差分镜像。这极大地节省了磁盘空间，并能快速创建和销毁实例。
- **灵活配置**: 通过命令行参数可以方便地为不同实例指定不同的内核目录 (-k) 和内存大小 (-m)。
- **生命周期管理**: 提供了 start, stop, cleanup 三个命令，实现了对虚拟机实例从创建、运行到彻底销毁的全生命周期管理。
- **主机-虚拟机文件共享**: 利用 -virtfs 参数实现了两个共享目录，一个用于通用文件交换，另一个专门用于动态加载和卸载内核模块 (.ko 文件)，极大地方便了调试。

问题 3: 虚拟机内 F2FS 自动挂载

每次启动虚拟机后手动挂载 F2FS 分区并加载模块非常低效。为了实现自动化，我编写了一个 systemd 服务，在开机后自动完成所有准备工作。

服务单元解析：

- **依赖关系:** After=local-fs.target 确保在本地文件系统准备好之后运行。
- **核心操作:** 在 ExecStart 中, 依次执行创建挂载点、以特定压缩选项挂载 F2FS 磁盘 (/dev/vdb)、以及设置一些便于调试的内核参数 (如 panic_on_oops)。
- **服务类型:** Type=oneshot 和 RemainAfterExit=yes 表明这是一个一次性任务, 执行成功后服务状态保持为 active。

异地编译脚本示例

```
make O=../../f2fs_head ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu-
olddefconfig Image -j8 &> ../../f2fs_head/makebenchlog.txt
# make O=../../f2fs_bench ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu-
olddefconfig Image -j8 &> ../../f2fs_bench/makebenchlog.txt
make O=../../f2fs_release ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu-
olddefconfig Image -j8 &> ../../f2fs_release/makebenchlog.txt
```

多实例 QEMU 管理脚本 qemu-run.sh

```
#!/bin/bash
# --- 基础配置 (可以根据需要修改) ---
# 基准镜像 (只读), 请确保这些文件存在
BASE_ROOT_IMG="/media/tower/DATA/ubuntu_2404.img"
BASE_F2FS_IMG="/usr/local/share/lff2fs.img"
# --- 默认值设置 ---
DEFAULT_KERDIR="/media/tower/DATA/f2fs_head" # 默认内核目录
DEFAULT_MEM="4G"
# 宿主机共享目录的基准路径
SHARE=/usr/local/share
# 实例文件存放目录
INSTANCE_DIR=$SHARE/qemu_instances
# --- 初始化变量为默认值 ---
KERDIR="$DEFAULT_KERDIR"
MEM="$DEFAULT_MEM"
# --- 主逻辑 --- 因为涉及到了多个虚拟机示例了, 我们对每个虚拟机示例都得知道要进行的操作, 并且
# 需要指定
# 唯一的实例名
COMMAND=$1
INSTANCE_NAME=$2

# 检查命令和实例名
if [[ "$COMMAND" != "start" && "$COMMAND" != "stop" && "$COMMAND" != "
cleanup" ]] || [[ -z "$INSTANCE_NAME" ]] || [[ "$1" = "-h" ]] || [[ "$1
" = "--help" ]]; then
    usage
    exit 0
fi
```

```

# 实例相关路径
INSTANCE_PATH="${INSTANCE_DIR}/${INSTANCE_NAME}"
INSTANCE_ROOT_IMG="${INSTANCE_PATH}/root.qcow2"
INSTANCE_F2FS_IMG="${INSTANCE_PATH}/f2fs.qcow2"
INSTANCE_SHARED_DIR="${INSTANCE_PATH}/shared_with_qemu"
INSTANCE_MOD_DIR="${INSTANCE_PATH}/modshare"
PID_FILE="${INSTANCE_PATH}/qemu.pid"
# --- 使用说明函数 ---
usage() {
    echo "QEMU 实例管理脚本"
    echo ""
    echo "用法: $0 <命令> [实例名] [选项]"
    echo ""
    echo "命令:"
    echo "  start <实例名>    启动一个新的或已存在的虚拟机实例"
    echo "  stop <实例名>     停止一个正在运行的虚拟机实例"
    echo "  cleanup <实例名>  停止实例并删除其所有文件（差分镜像、共享目录等）"
    echo ""
    echo "选项（仅用于 'start' 命令）:"
    echo "  -k, --kerdir DIR    指定内核所在的目录（默认: ${DEFAULT_KERDIR}）"
    echo "  -m, --mem SIZE      指定分配给虚拟机的内存大小（默认: ${DEFAULT_MEM}）"
    echo "  -h, --help          显示此帮助信息并退出"
    echo ""
    echo "示例:"
    echo "  # 启动名为 'vm1' 的实例（首次启动会自动创建文件）"
    echo "  $0 start vm1"
    echo ""
    echo "  # 启动 'vm2', 并分配 16G 内存, 使用 'linux_next' 内核"
    echo "  $0 start vm2 -m 16G -k linux_next"
    echo ""
    echo "  # 停止 'vm1'"
    echo "  $0 stop vm1"
    echo ""
    echo "  # 彻底清理 'vm2'（删除所有相关文件）"
    echo "  $0 cleanup vm2"
}

# --- 命令处理 ---
case $COMMAND in
    start)
        # --- 解析 start 命令的额外参数 ---
        shift 2 # 移过 command 和 instance_name
        KERDIR="${DEFAULT_KERDIR}"
        MEM="${DEFAULT_MEM}"
        while [[ $# -gt 0 ]]; do
            key="$1"
            case $key in

```



```

        -k|--kerdir) KERDIR="$2"; shift 2 ;;
        -m|--mem) MEM="$2"; shift 2 ;;
        *) echo "错误: 'start' 命令的未知选项 '$1'"; usage; exit 1 ;;
    esac
done

# --- 检查基准镜像是否存在 ---
if [ ! -f "$BASE_ROOT_IMG" ] || [ ! -f "$BASE_F2FS_IMG" ]; then
    echo "错误: 找不到基准镜像 '$BASE_ROOT_IMG' 或 '$BASE_F2FS_IMG'。"
    echo "请将它们放在脚本所在目录, 或修改脚本中的 BASE*_IMG 变量。"
    exit 1
fi

# --- 准备实例环境 (如果不存在) ---
if [ ! -d "$INSTANCE_PATH" ]; then
    echo "首次启动实例 '${INSTANCE_NAME}', 正在创建所需文件 ..."
    mkdir -p "$INSTANCE_PATH"
    mkdir -p "$INSTANCE_SHARED_DIR"
    mkdir -p "$INSTANCE_MOD_DIR"
    echo "复制基准共享目录文件到实例共享目录 ..."
    cp -r $SHARE/shared_with_qemu $INSTANCE_SHARED_DIR
    cp -r $SHARE/modshare $INSTANCE_SHARED_DIR
    echo "创建 CoW 根文件系统镜像 ..."
    qemu-img create -f qcow2 -b "${BASE_ROOT_IMG}" -F raw "
$INSTANCE_ROOT_IMG"

    echo "创建 CoW f2fs 数据盘镜像 ..."
    qemu-img create -f qcow2 -b "${BASE_F2FS_IMG}" -F raw "
$INSTANCE_F2FS_IMG"

    echo "环境准备完毕。"
fi

if [ -f "$PID_FILE" ] && ps -p $(cat "$PID_FILE") > /dev/null; then
    echo "错误: 实例 '${INSTANCE_NAME}' 似乎已在运行 (PID: $(cat "
$PID_FILE"))。"
    exit 1
fi

# --- 打印最终配置并启动 QEMU ---
echo "===== QEMU 配置 (${INSTANCE_NAME}) ====="
echo "内核目录: ${KERDIR}"
echo "内存大小: ${MEM}"
echo "根文件系统: ${INSTANCE_ROOT_IMG} (基于 ${BASE_ROOT_IMG})"
echo "F2FS 数据盘: ${INSTANCE_F2FS_IMG} (基于 ${BASE_F2FS_IMG})"
echo "共享目录 1: ${INSTANCE_SHARED_DIR}"
echo "共享目录 2: ${INSTANCE_MOD_DIR}"
echo "PID 文件: ${PID_FILE}"

```

```

echo "===== "

# 使用 exec 会让 qemu 进程替换掉当前的 shell 进程
qemu-system-aarch64 \
    -smp 8 \
    -machine virt,virtualization=true,gic-version=3 \
    -nographic \
    -m size=${MEM} \
    -mem-prealloc \
    -cpu cortex-a72 \
    -kernel ${KERDIR}/arch/arm64/boot/Image \
    -netdev user,id=eth0 -device virtio-net-device,netdev=eth0 \
    -drive format=qcow2,file=${INSTANCE_ROOT_IMG},if=virtio,id=
rootdisk \
    -drive format=qcow2,file=${INSTANCE_F2FS_IMG},if=virtio,id=
f2fsdisk \
    -virtfs local,path=${INSTANCE_SHARED_DIR},mount_tag=hostshare,
security_model=passthrough,id=hostshare \
    -virtfs local,path=${INSTANCE_MOD_DIR},mount_tag=modshare,
security_model=passthrough,id=modshare \
    -append "noinitrd root=/dev/vda rw console=ttyAMA0 nokaslr
loglevel=8 crashkernel=auto sysrq_always_enabled" \
    -pidfile ${PID_FILE} \
    -s
;;

stop)
    if [ ! -f "$PID_FILE" ]; then
        echo "错误：找不到实例 '${INSTANCE_NAME}' 的 PID 文件。它可能未在运行或
已被清理。"
        exit 1
    fi
    PID=$(cat "$PID_FILE")
    echo "正在停止实例 '${INSTANCE_NAME}' (PID: ${PID}) ... "
    if ps -p $PID > /dev/null; then
        kill $PID
        # 等待进程结束
        while ps -p $PID > /dev/null; do
            sleep 1
        done
        echo "实例已停止。"
        rm -f "$PID_FILE"
    else
        echo "警告：PID ${PID} 不存在。可能实例已经关闭。正在清理 PID 文件。"
        rm -f "$PID_FILE"
    fi
;;

```

```

cleanup)
    echo "警告：这将永久删除实例 '${INSTANCE_NAME}' 的所有数据！"
    read -p "包括：${INSTANCE_PATH} 目录及其所有内容。确定吗？(y/N) " confirm
    if [[ "$confirm" != "y" && "$confirm" != "Y" ]]; then
        echo "操作已取消。"
        exit 0
    fi

    # 首先尝试正常停止
    if [ -f "$PID_FILE" ]; then
        $0 stop "$INSTANCE_NAME"
    fi

    echo "正在删除实例目录：${INSTANCE_PATH}"
    rm -rf "$INSTANCE_PATH"
    echo "实例 '${INSTANCE_NAME}' 已被彻底清理。"
    ;;
esac

# -drive format=raw,file=smallf2fs.img,if=virtio,id=smallf2fsdisk \

# -drive format=raw,file=xfss.img,if=virtio,id=xfssdisk
# kasan_multi_shot=1 page_owner=on lockdep_debug=1 "

```

systemd 服务单元 /etc/systemd/system/f2fs-setup.service

```

[Unit]
Description=Mount modshare and setup F2FS with modules
After=local-fs.target
Before=multi-user.target

[Service]
Type=oneshot
RemainAfterExit=yes
ExecStart=/bin/bash -c '\
    # 3. 创建 F2FS 挂载点并挂载 \
    mkdir -p /mnt/f2fs && \
    mount -t f2fs -o compress_algorithm=lz4,compress_extension=txt /dev/vdb /
    mnt/f2fs && \
    echo 8 > /proc/sysrq-trigger && \
    echo 1 > /proc/sys/kernel/panic_on_oops && \
    echo "F2FS mounted successfully"'

ExecStop=/bin/bash -c '\
    umount /mnt/f2fs 2>/dev/null || true && \
    rmmod f2fs 2>/dev/null || true && \
    umount /lib/modules/$(uname -r) 2>/dev/null || true'

```

```
TimeoutSec=60

[Install]
WantedBy=multi-user.target
```

5.2 Git 版本控制 workflow

5.2.1 从挣扎到规范：workflow 的演进

- **初期遇到的问题**: 在项目初赛阶段, 我试图将整个包含我修改的内核源码树提交到 GitLab。由于内核代码仓库极其庞大, 这导致推送 (push) 操作几乎无法完成, 严重影响了开发效率和版本备份。
- **导师指引下的变革**: 在导师的指导下, 我切换到了业界标准的内核开发 workflow。我不再尝试提交整个代码树, 而是:
 1. Fork 了上游的 f2fs-dev 内核源码仓库到我自己的 GitHub 账号下。
 2. 在我的 Fork 中创建了专门的开发分支。
 3. 只提交我的代码修改 (patches), 而不是整个仓库。

这次转变不仅解决了提交困难的问题, 更重要的是, 它保留了完整的 Git 历史, 使得我的工作能够清晰地与上游社区的演进对齐。

5.2.2 高级技巧：在动态变化中维护代码

面临的挑战：与上游社区同步

Linux 内核开发极其活跃, 是一个永远在动态变化的过程。就在决赛开发期间, f2fs 的维护者 Matthew Wilcox 先生向上游推送了一系列补丁, 其中一部分与我正在开发的代码产生了冲突。这要求我必须能熟练地处理代码合并与重构。

掌握的“后悔药”与代码重构技巧

整个过程让我深刻理解了 Git “永远都有后悔药可吃” 的设计哲学, 并熟练掌握了一系列高级操作来维护一个清晰、逻辑性强的提交历史。

- **交互式变基 (git rebase -i)**: 这是最重要的工具。我用来:
 - **拆分 Commit**: 将一个大的功能实现 (如 buffered read/write) 拆分成多个逻辑独立的、原子性的提交。
 - **合并 Commit (squash)**: 将一些小的修复或调试性提交合并到其所属的功能提交中, 保持主干历史的整洁。

- 编辑 Commit (edit): 修改提交信息或对某个历史提交进行微调。
- **精准暂存 (git add -p)**: 在拆分 Commit 时, 使用该命令可以交互式地选择要暂存的代码块 (hunks), 实现对文件的部分提交, 非常精准。
- **日常“后悔药”**:
 - git restore --staged <file>: 取消暂存。
 - git rebase --abort: 当变基过程出现严重问题时, 放弃本次 rebase, 回到初始状态。
 - git stash: 临时保存工作区改动, 切换到其他分支修复紧急问题。
 - git cherry-pick: 从其他分支“采摘”某个特定的提交应用到当前分支。
 - git commit --amend: 修改最新一次的提交。

通过这一过程, 我成功地将我的代码变基到了最新的 f2fs-dev 分支上, 并利用条件编译等技巧巧妙地解决了冲突。现在, 我已经掌握了专业的软件开发、版本管理和备份流程, 对内核社区的协同工作方式有了深入的理解, 初步迈入了内核开发者的行列。

5.3 树莓派物理开发环境

5.3.1 基本 workflow 搭建

硬件与前沿技术背景

- **测试平台**: 我选购了 **树莓派 5 (4GB RAM, 256GB NVMe SSD)** 作为物理测试平台。它是一款性能强大的 aarch64 设备。
- **16KB 页面大小的重大意义**: 树莓派 5 官方系统的一大特性是使用了 **16KB** 的基础页面大小。这一点至关重要, 因为它与当前业界, 特别是安卓生态系统的发展前沿完全契合。Google 正在推动未来的安卓系统默认使用 16KB 页面, 以匹配硬件厂商推出的 16KB 物理块大小的闪存设备。因此, 在树莓派上针对 16KB 页面大小进行性能测试和优化, 其结果具有极高的现实参考价值和前瞻性。

部署流程

1. **格式化 16KB 块大小的 F2FS 分区**: 树莓派官方 Ubuntu 22.04 源中的 f2fs-tools 版本较旧, 不支持格式化 16KB 块大小。为此, 我从官方 Git 仓库拉取了最新的 f2fs-tools 源码, 在我的 x86 主机上交叉编译出 aarch64 版本的二进制文件。然后, 我将树莓派的 SSD 通过 USB 转接器挂载到主机上, 使用这个新编译的 mkfs.f2fs 成功格式化出了 16KB 块大小的 F2FS 分区。
2. **内核编译与部署策略**: 为了确保系统的稳定性和驱动兼容性, 我采取了稳健的策略:
 - (a) Fork 树莓派官方的内核源码仓库 (版本 6.15)。

- (b) 首先，不加任何修改地编译官方内核，并将其部署到树莓派上，确保可以成功启动。
- (c) 验证成功后，在我的 Fork 中开辟新分支，只修改文件系统（fs/f2fs/）相关的代码，不触碰任何驱动或核心架构代码。

这个策略保证了我的内核能够基于一套官方验证过的稳定驱动运行，极大地降低了调试难度。

3. **自动化部署脚本**：手动复制内核镜像、设备树和模块到 SD 卡或通过网络传输非常繁琐且容易出错。为此，我编写了一个全自动的部署脚本。

树莓派内核自动化部署脚本 `deploy-rpi.sh`

```
#!/usr/bin/env bash
# Deploy a custom RPi5 kernel to /boot/firmware and modules to /usr/lib/
# modules (on the Pi)
# Requirements on host: built kernel tree, sshpass, scp/ssh
set -Eeuo pipefail

### === 配置区（按需修改） ===
# 内核源码目录（已完成配置并能正常 make 的目录）
KERNEL_SRC="${KERNEL_SRC:-$PWD}"

# 交叉编译前缀（如果本机是 x86_64 而编 aarch64 内核）
CROSS_COMPILE="${CROSS_COMPILE:-aarch64-linux-gnu-}"
ARCH="${ARCH:-arm64}"

# 树莓派 5 (BCM2712) 要放到 /boot/firmware 的镜像名
KERNEL_IMAGE_NAME="${KERNEL_IMAGE_NAME:-mykernel_2712.img}"

# 目标树莓派
RPI_HOST="${RPI_HOST:-raspberrypi.local}"
RPI_USER="${RPI_USER:-pi}"
RPI_PASS="${RPI_PASS:-pi}"
RPI_BOOT="${RPI_BOOT:-/boot/firmware}" # 你的系统就是 /boot/firmware
RPI_MODBASE="${RPI_MODBASE:-/usr/lib/modules}" # Ubuntu/RPiOS 下 /lib/modules
# -> /usr/lib/modules

# 本地打包输出目录（安全区，不触碰系统目录）
STAGING="${STAGING:-/tmp/rpi-kernel-out}"
### === 配置区结束 ===

command -v sshpass >/dev/null || { echo "需要安装 sshpass"; exit 1; }

echo "=> 编译内核 Image / modules / dtbs"
make -C "$KERNEL_SRC" ARCH="$ARCH" CROSS_COMPILE="$CROSS_COMPILE" -j"${nproc}"
) Image modules dtbs
```

```

KREL="$(make -sC "$KERNEL_SRC" ARCH="$ARCH" CROSS_COMPILE="$CROSS_COMPILE"
    kernelrelease)"
echo "=> Kernel release: $KREL"

echo "=> 准备打包目录: $STAGING"
rm -rf "$STAGING"
mkdir -p "$STAGING/boot/firmware/overlays" "$STAGING/usr/lib/modules"

echo "=> 安装全部模块到打包目录（不影响主机）"
make -C "$KERNEL_SRC" ARCH="$ARCH" modules_install \
    INSTALL_MOD_PATH="$STAGING" INSTALL_MOD_STRIP=1

echo "=> 放置内核镜像到打包目录: $KERNEL_IMAGE_NAME"
cp -fv "$KERNEL_SRC/arch/arm64/boot/Image" \
    "$STAGING/boot/firmware/$KERNEL_IMAGE_NAME"

echo "=> 收集 DTB 和 overlays（尽量兼容路径差异）"
# DTB 到 /boot/firmware 根目录
while IFS= read -r -d '' f; do
    cp -fv "$f" "$STAGING/boot/firmware/"
done < <(find "$KERNEL_SRC/arch" -type f \( -name "bcm27*.dtb" -o -name "
    bcm2712*.dtb" \) -print0 2>/dev/null)

# overlays 到 /boot/firmware/overlays
while IFS= read -r -d '' f; do
    cp -fv "$f" "$STAGING/boot/firmware/overlays/"
done < <(find "$KERNEL_SRC/arch" -type f -path "*/overlays/*.dtbo" -print0
    2>/dev/null)

# README（可选）
ov_readme="$(find "$KERNEL_SRC/arch" -type f -path "*/overlays/README*" |
    head -n1 || true)"
[ -n "$ov_readme:-" ] && cp -fv "$ov_readme" "$STAGING/boot/firmware/
    overlays/"

echo "=> 本地打包完成结构: "
tree -L 3 "$STAGING" || find "$STAGING" -maxdepth 3 -type f | sed "s#^
    $STAGING/# #"

TS="$(date +%Y%m%d-%H%M%S)"
REMOTE_TMP="/tmp/kupd-$TS"

echo "=> 在树莓派上创建临时目录，并校验 /lib/modules 软链"
sshpass -p "$RPI_PASS" ssh -o StrictHostKeyChecking=no "$RPI_USER@$RPI_HOST"
    bash -lc "set -e
sudo mkdir -p '$REMOTE_TMP/boot' '$REMOTE_TMP/modules'
if [ ! -L /lib/modules ]; then
    echo 'ERROR: 远端 /lib/modules 不是软链接，退出以保护系统' >&2

```

```

    exit 1
fi
echo '远端检查通过: /lib/modules 是软链接 -> ' \$(readlink -f /lib/modules)
uname -a
"

echo "==> 传输 boot 文件到远端 $REMOTE_TMP/boot"
sshpass -p "$RPI_PASS" scp -o StrictHostKeyChecking=no -r \
"$STAGING/boot/firmware/"* \
"$RPI_USER@$RPI_HOST:$REMOTE_TMP/boot/"

echo "==> 传输 modules 到远端 $REMOTE_TMP/modules"
sshpass -p "$RPI_PASS" scp -o StrictHostKeyChecking=no -r \
"$STAGING/usr/lib/modules/"* \
"$RPI_USER@$RPI_HOST:$REMOTE_TMP/modules/"

echo "==> 远端执行: 滚动备份并覆盖安装 (cp 覆盖, 不用 rsync/tar) "
sshpass -p "$RPI_PASS" ssh -o StrictHostKeyChecking=no "$RPI_USER@$RPI_HOST"
    bash -lc "set -e
KREL='$KREL'; RPI_BOOT='$RPI_BOOT'; RPI_MODBASE='$RPI_MODBASE'; REMOTE_TMP='
    $REMOTE_TMP'

# 1) 备份当前 /boot/firmware 到带时间戳目录, 保留最近 3 份
TS2=\$(date +%Y%m%d-%H%M%S)
if [ -d "\"\$RPI_BOOT\""]; then
    echo '备份: '\$RPI_BOOT' -> /boot/firmware.mybak.\$TS2
    sudo cp -a "\"\$RPI_BOOT\" \" /boot/firmware.mybak.\$TS2\"
    sudo bash -lc 'ls -ldt /boot/firmware.mybak.* 2>/dev/null | tail -n +4 |
        xargs -r rm -rf'
fi

# 2) 覆盖安装 boot 文件 (允许覆盖 mykernel_2712.img)
echo '安装 boot 文件到 '\$RPI_BOOT
sudo cp -a "\"\$REMOTE_TMP/boot/\\"* "\"\$RPI_BOOT/\\"

# 3) 安装 modules (复制到 /usr/lib/modules, 绝不动 /lib/modules 的软链实体)
echo '安装 modules 到 '\$RPI_MODBASE
sudo mkdir -p "\"\$RPI_MODBASE\"
sudo cp -a "\"\$REMOTE_TMP/modules/\\"* "\"\$RPI_MODBASE/\\"

# 4) 为新内核做 depmod (只针对新版本, 不动当前运行内核)
echo 'depmod for '\$KREL
sudo depmod -a "\"\$KREL\" || true

# 5) 清理临时目录
sudo rm -rf "\"\$REMOTE_TMP\"
echo '部署完成. 新内核: '\$KREL' 镜像: '\$RPI_BOOT'/'$KERNEL_IMAGE_NAME'
"

```



```
echo "=> 全部完成。现在可以重启树莓派："
echo "ssh $RPI_USER@$RPI_HOST 'sudo reboot'"
```

5.3.2 遇到的重大问题：动态 IP 与网络穿透

问题的根源：不稳定的局域网 IP

在校园网环境中，树莓派通过 Wi-Fi 或有线连接获取到的 IP 地址是动态分配的，每次重启或重连后都可能改变。这使得我无法在 WSL 开发环境中通过一个固定的 IP 地址来 SSH 连接或使用 SCP 传输文件，极大地阻碍了开发流程。

失败的尝试

我曾尝试使用 Windows 系统自带的移动热点功能，让树莓派连接此热点以获得固定 IP。然而，该热点创建的网络与我的 WSL 所在的 Hyper-V 虚拟网络处于不同的局域网段。尽管我手动配置了 WSL 的路由表试图进行转发，但在排除了防火墙问题后，两者之间依然无法 ping 通。我推测这可能是 Hyper-V 虚拟交换机本身的限制所致。

最终解决方案：使用 WireGuard 搭建虚拟私有网络

为了彻底解决这个问题，我租用了一台具有公网 IP 的云服务器(VPS)，并使用 WireGuard 在我的 WSL、VPS 和树莓派之间构建了一个星形拓扑的虚拟私有网络 (VPN)。

工作原理：

- VPS 作为中心节点（中转服务器），它拥有一个固定的公网 IP 和一个固定的 VPN 内网 IP (如 10.0.0.1)。
- 我的 WSL 和树莓派都作为客户端连接到这个中心节点。它们各自也被分配一个固定的 VPN 内网 IP (如 WSL 为 10.0.0.2，树莓派为 10.0.0.3)。
- 流量通过 WireGuard 的加密隧道，经由 VPS 进行中转。这样，无论我的 WSL 和树莓派在物理世界中的 IP 地址如何变化，它们在 VPN 这个“虚拟局域网”中的 IP 地址永远是固定的。

配置流程：

1. **在三台设备上都安装 WireGuard**
2. **生成密钥**：在每台设备上生成一对公钥和私钥。
3. **配置 VPS (服务器)**：

```
[Interface]
Address = 10.0.0.1/24
ListenPort = 51820
PrivateKey = <VPS的私钥>
PostUp = iptables -A FORWARD -i %i -j ACCEPT; iptables -t nat -A
        POSTROUTING -o eth0 -j MASQUERADE
PostDown = iptables -D FORWARD -i %i -j ACCEPT; iptables -t nat -D
        POSTROUTING -o eth0 -j MASQUERADE

# Peer 1: WSL
[Peer]
PublicKey = <WSL的公钥>
AllowedIPs = 10.0.0.2/32

# Peer 2: Raspberry Pi
[Peer]
PublicKey = <树莓派的公钥>
AllowedIPs = 10.0.0.3/32
```

4. 配置 WSL (客户端 1):

```
[Interface]
Address = 10.0.0.2/32
PrivateKey = <WSL的私钥>
DNS = 8.8.8.8

[Peer]
PublicKey = <VPS的公钥>
Endpoint = <VPS的公网IP>:51820
AllowedIPs = 0.0.0.0/0, ::/0 # 路由所有流量
PersistentKeepalive = 25
```

5. 配置树莓派 (客户端 2):

```
[Interface]
Address = 10.0.0.3/32
PrivateKey = <树莓派的私钥>
DNS = 8.8.8.8

[Peer]
PublicKey = <VPS的公钥>
Endpoint = <VPS的公网IP>:51820
AllowedIPs = 0.0.0.0/0, ::/0 # 路由所有流量
PersistentKeepalive = 25
```

项目总结和未来展望

在 AI 训练带来海量数据吞吐需求，以及安卓生态积极推进 16KB 页面作为新标准的双重驱动下，一场由大页内存引领的深刻变革已是大势所趋。

我们有理由相信，随着 Large Folios 技术的进一步推广，操作系统长期以来固守 4KB 页面的时代可能终于要告一段落了。我们为 F2FS 文件系统所做的优化工作，正是对这一历史趋势的积极响应。

我们的核心技术愿景，是深度融合 Large Folios 与 Iomap 这两项内核关键技术，为上层应用构建一条从物理存储直达计算核心的“数据高速公路”，最大限度地消除 I/O 路径上的障碍。展望未来，我们计划将相关补丁持续推向上游社区，并积极参与中国 Linux 内核开发者大会等会议与同行进行深入探讨，最终目标是推动该技术优化在亿万安卓设备上实现产业落地，让我们的工作成果真正惠及广大用户。

修改函数

方法	返回值	方法描述
f2fs_cluster_is_empty	bool	检查压缩簇是否为空, 适配 folio
f2fs_release_folio	bool	释放一个 folio, 适配 large folio
f2fs_iomap_ops	const struct iomap_ops	导出的 iomap 操作集, 适配新 I/O 路径
f2fs_do_write_data_page	int	执行数据页的写操作, 适配 folio
f2fs_map_blocks	int	映射文件逻辑块到物理块, 适配 iomap
f2fs_prepare_compress_overwrite	int	准备压缩并覆写, 适配 folio
f2fs_read_multi_pages	int	读取多个页面, 修改为基于 folio 操作
f2fs_submit_page_bio	int	提交页面 bio, 适配 folio
f2fs_write_multi_pages	int	写入多个页面, 修改为基于 folio 操作
f2fs_write_single_data_page	int	写入单个数据页, 修改为基于 folio 操作
__has_merged_page	bool	检查 bio 中是否已合并页面, 适配 folio
cluster_may_compress	bool	判断压缩簇是否可以被压缩, 适配 folio
f2fs_cluster_is_full	bool	检查压缩簇是否已满, 适配 folio
f2fs_dirty_meta_folio	bool	标记元数据 folio 为脏页
f2fs_dirty_node_folio	bool	标记节点 folio 为脏页
map_is_mergeable	bool	检查块映射是否可合并, 适配 folio
__f2fs_write_data_pages	int	内部函数, 写入数据页, 适配 folio
__get_segment_type_6	int	获取段类型, 适配 folio 场景
f2fs_preallocate_blocks	int	预分配数据块, 适配 iomap
f2fs_write_cache_pages	int	回写缓存中的页面, 修改为基于 folio
f2fs_write_compressed_pages	int	写入压缩页面, 修改为基于 folio
f2fs_write_raw_pages	int	写入原始页面, 修改为基于 folio
gc_data_segment	int	对数据段进行垃圾回收, 适配 folio
move_data_block	int	移动数据块, 适配 folio
move_data_page	int	移动数据页, 修改为基于 folio
prepare_compress_overwrite	int	准备压缩并覆写, 适配 folio
cluster_idx	pgoff_t	计算给定索引所在的簇索引, 适配 folio
f2fs_buffered_write_iter	ssize_t	buffered write 的核心迭代函数, 适配 iomap
f2fs_new_inode	struct inode *	创建新 inode, 适配 iomap
offset_in_cluster	unsigned int	计算页在簇内的偏移, 适配 folio
cancel_cluster_writeback	void	取消簇的回写状态, 适配 folio
do_write_page	void	执行页写入, 修改为基于 folio
f2fs_drop_rpages	void	丢弃预读页面, 修改为基于 folio
f2fs_finish_read_bio	void	完成读 bio 的回调, 适配 folio
f2fs_handle_step_decompress	void	处理解压步骤, 适配 folio
f2fs_put_rpages_wbc	void	释放预读页面, 修改为基于 folio
f2fs_submit_write_bio	void	提交写 bio, 适配 folio
f2fs_write_end_io	void	写 IO 完成回调, 适配 folio
set_cluster_dirty	void	设置簇为脏, 适配 folio
set_cluster_writeback	void	设置簇的回写状态, 适配 folio

修改函数 (续)

方法	返回值	方法描述
f2fs_compress_write_end_io	void	压缩写 IO 完成回调, 适配 folio
f2fs_decompress_end_io	void	解压 IO 完成回调, 适配 folio
f2fs_invalidate_folio	void	使 folio 失效, 适配 large folio
f2fs_outplace_write_data	void	执行 out-of-place 数据写入, 适配 folio
f2fs_submit_page_write	void	提交页写操作, 适配 folio
f2fs_update_dirty_folio	void	更新 folio 的脏页状态

新增函数

方法	返回值	方法描述
cluster_i_idx	pgoff_t	获取 inode 中指定索引的簇号
do_read_multi_folios	int	执行多个 folio 的读取操作
end_idx_of_cluster	pgoff_t	获取簇的结束索引
f2fs_buffered_read_compress_iomap_begin	int	iomap 框架下压缩文件 buffered read 的起始操作
f2fs_buffered_read_iomap_begin	int	iomap 框架下普通文件 buffered read 的起始操作
f2fs_buffered_write_atomic_iomap_begin	int	iomap 框架下原子写操作的起始准备
f2fs_buffered_write_iomap_begin	int	iomap 框架下 buffered write 的起始操作
f2fs_buffered_write_iomap_end	int	iomap 框架下 buffered write 的结束操作
f2fs_clear_folio_private_all	void	清除 folio 的所有私有数据和标志
f2fs_clear_folio_private_data	void	清除 folio 的私有数据
f2fs_cluster_is_partial_full	bool	检查压缩簇是否部分满
f2fs_compress_ctx_add_folio	void	向压缩上下文中添加一个 folio
f2fs_compress_iomap_readahead	void	iomap 框架下压缩文件的预读实现
f2fs_compress_readahead_iter	int	iomap 预读迭代器, 处理压缩文件
f2fs_compress_readpage_iter	int	iomap 读页迭代器, 处理压缩文件
f2fs_do_read_multi_folios	int	执行多个 folio 的读取
f2fs_do_read_single_folio_iomap	int	iomap 框架下读取单个 folio
f2fs_flush_compress_ctx_read	int	刷新压缩上下文, 提交读 bio
f2fs_folio_get_private	unsigned long	获取 folio 的私有数据
__f2fs_get_compressed_pblks_info	int	获取压缩文件的物理块信息
f2fs_get_folio_private_data	unsigned long	获取 folio 私有数据中的数据部分
f2fs_ifs_alloc	int	为 folio 分配 iomap folio state (ifs)
f2fs_ifs_block_is_uptodate	bool	检查 ifs 中的块是否是 up-to-date
f2fs_ifs_clear_range_uptodate	void	清除 ifs 中指定范围的 up-to-date 状态
f2fs_ifs_dirty_bytes_pending_ptr	atomic_t *	获取 ifs 中待处理脏字节数的指针
f2fs_ifs_free	void	释放 folio 的 iomap folio state
f2fs_ifs_iomap_longs	int	计算 ifs 中用于 iomap 的 long 数量
f2fs_ifs_print_uptodate_status	void	打印 ifs 的 up-to-date 状态 (调试用)
f2fs_ifs_private_flags_ptr	unsigned long *	获取 ifs 私有标志的指针
f2fs_ifs_total_longs	int	计算 ifs 所需的总 long 数量
f2fs_init_readpage_ctx	void	初始化读页上下文
f2fs_iomap_buffered_write	ssize_t	基于 iomap 的 buffered write 实现
f2fs_iomap_find_dirty_range	void	在 folio 中查找脏数据范围
f2fs_iomap_finish_folio_read	void	完成 folio 的读操作, 处理 iomap 状态

新增函数 (续)

方法	返回值	方法描述
f2fs_set_folio_private_data	void	设置 folio 的私有数据
f2fs_should_use_buffered_iomap	bool	判断是否应使用 iomap buffered I/O 路径
f2fs_submit_read_bio_sync	int	同步提交读 bio
f2fs_wait_cluster_uptodate	int	等待簇数据变为 up-to-date
f2fs_write_cache_folios	int	回写缓存中的多个 folio
f2fs_write_single_data_folio	int	写入单个数据 folio
f2fs_write_split_folio_range	void	回写指定范围内的脏 folio
i_end_idx_of_cluster	pgoff_t	获取 inode 中指定索引所在簇的结束索引
ifs_to_f2fs_ifs	void	将通用 ifs 指针转换为 f2fs_iomap_folio_state
inc_page_count_multiple	void	批量增加页面计数
is_f2fs_ifs	bool	检查 folio 是否使用了 f2fs iomap folio state
offset_i_in_cluster	unsigned int	获取 inode 中指定索引在簇内的偏移
set_cluster_write_pending	void	设置簇为写 pending 状态